

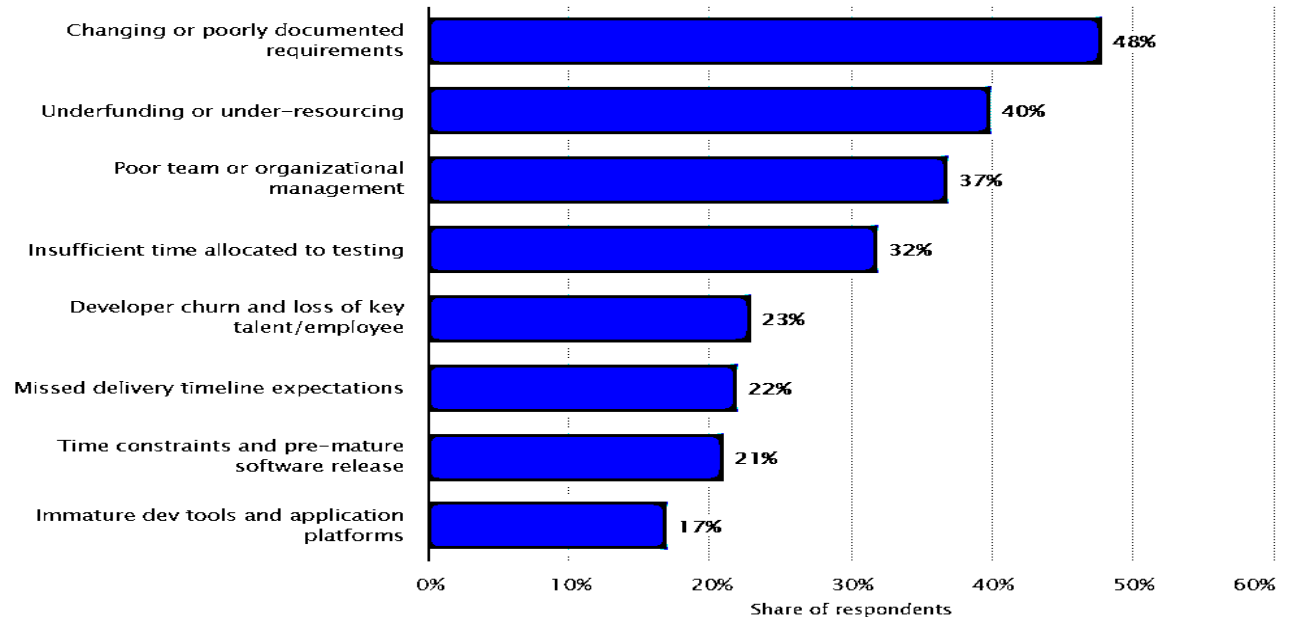
COSC 205

Agile Software Development



Challenges in Project Development

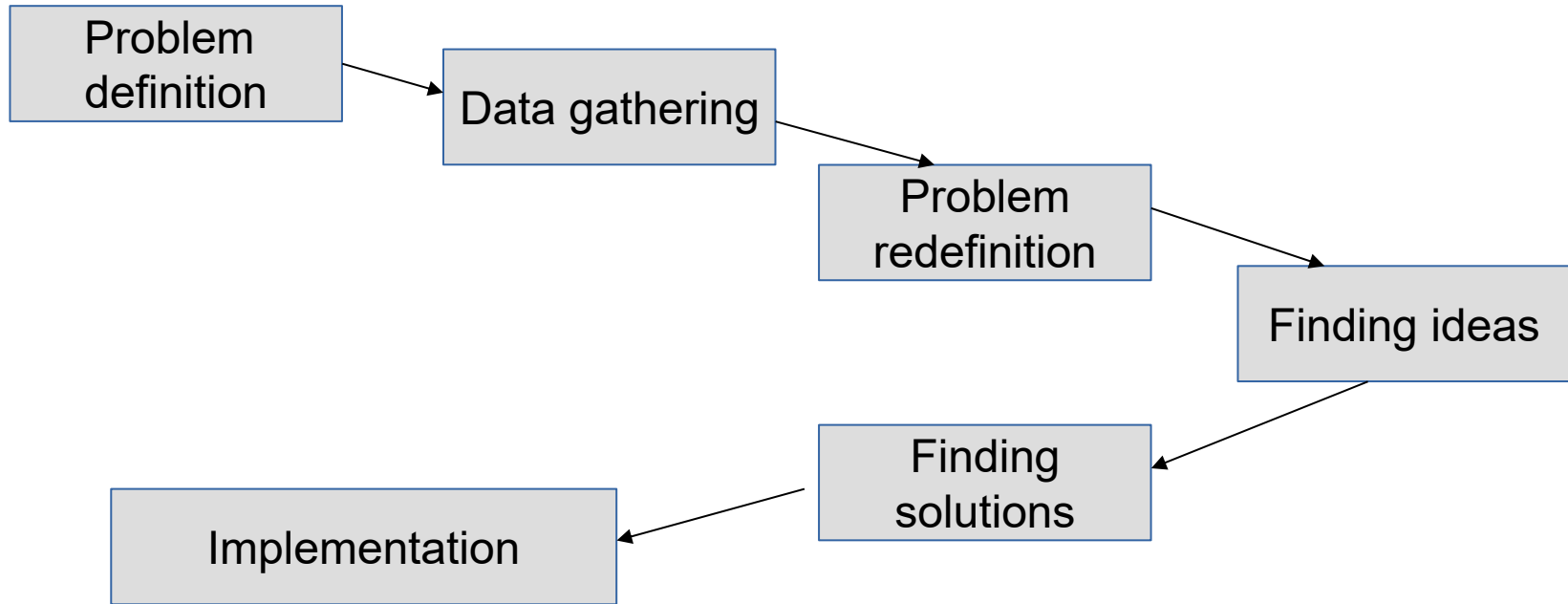
Project Challenged Factors	% of Responses
1. Lack of User Input	12.8%
2. Incomplete Requirements & Specifications	12.3%
3. Changing Requirements & Specifications	11.8%
4. Lack of Executive Support	7.5%
5. Technology Incompetence	7.0%
6. Lack of Resources	6.4%
7. Unrealistic Expectations	5.9%
8. Unclear Objectives	5.3%
9. Unrealistic Time Frames	4.3%
10. New Technology	3.7%
Other	23.0%



Challenges

- **Quality Problems** (missing or inappropriate functionality, incorrect requirements analysis, external events change the environment, poor interface design, requirements changed before project delivery, ineffective help)
- **Productivity Problems** (implementation is not feasible, impossible targets, time constraints, requirements dynamics, poor project control, cost overrun, late delivery)
- **Installation and Operation Problems** (poor installation, operational issues, poor response times, unreliable operations)

Lifecycle of a Project



Planning

Analysis

Design

Implementation

System

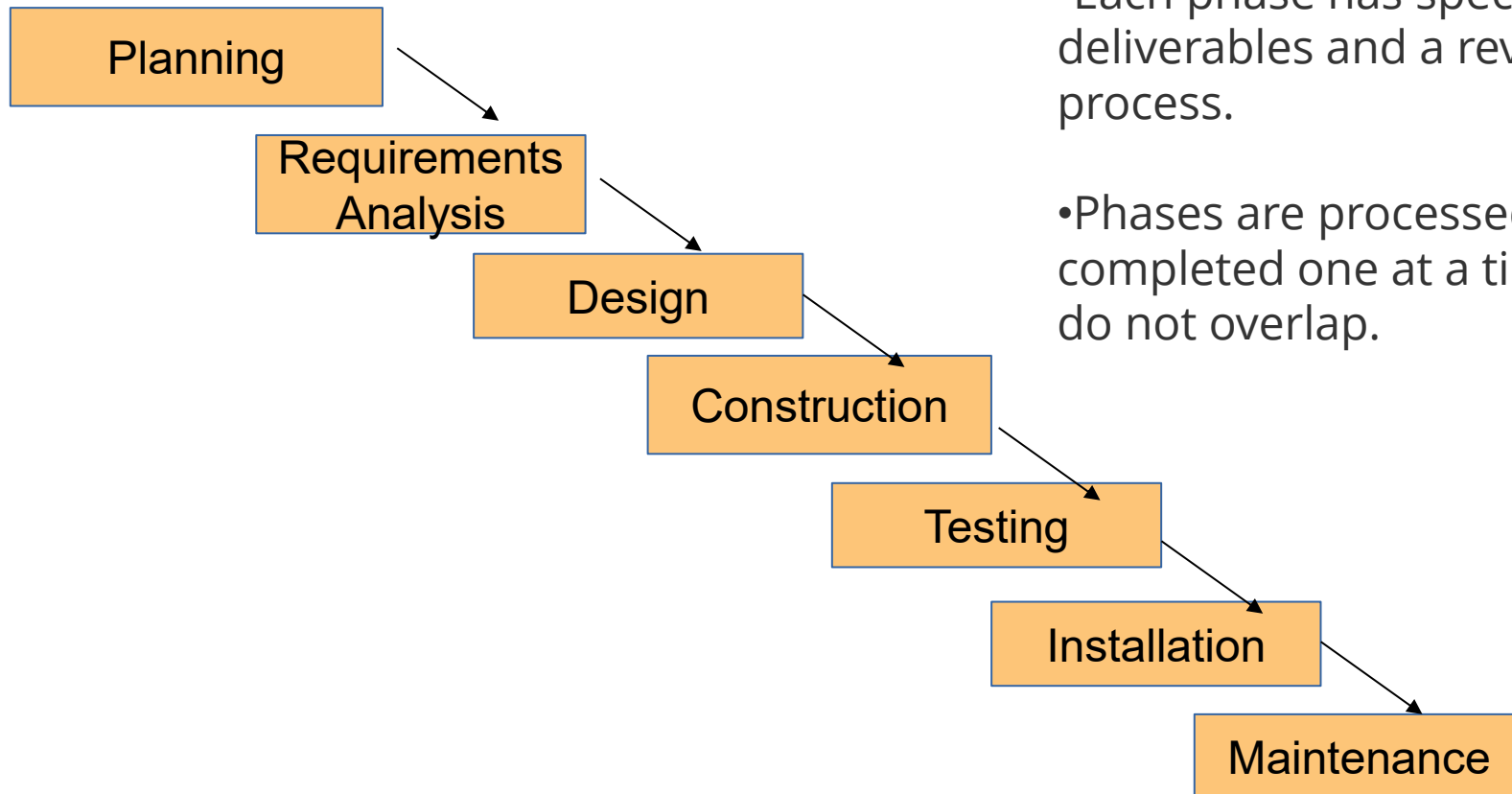
Software Development Life Cycle

Rapid Software Development

- Rapid development and delivery is now the most important requirement for companies
 - Business operate in a fast environment where requirements are constantly changing, so our software must evolve quickly as well
- Two ideologies:
 - Plan-driven development
 - Agile development

Plan-driven Development

- One of the more common plan-driven development ideologies is the waterfall process model



- Each phase has specific deliverables and a review process.

- Phases are processed and completed one at a time. Phases do not overlap.

Waterfall Process Model

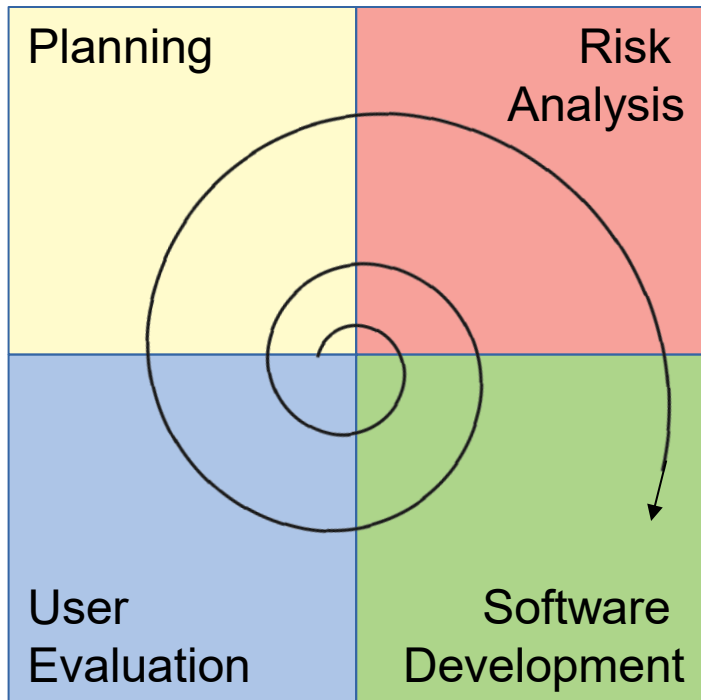
- Advantages:
 - early identification of system requirements;
 - minimizes the changes.
- Disadvantages:
 - no working software is produced until late during the life cycle;
 - high amounts of risk and uncertainty;
 - takes long time to deliver the system.

Agile Development (Iterative Process Model)

- The concept of iterative development: software development is organized into a series of small projects called iterations.
- Each iteration has its analysis, design, implementation, and testing. At the end of each iteration, the customer input is taken.
- If a customer did not agree, then the loss is minimal (1-4 weeks) as compared to waterfall process model.
- Program specification, design and implementation are interleaved
- Frequent delivery of new versions for evaluation
- Minimal documentation – focus on working code

Iterative Process Models

- Iterative and Incremental



Agile Development Values

- Individuals and interactions over processes and tools
- Working software over comprehensive documentation
- Customer collaboration over contract negotiation
- Responding to change over following a plan
- Agile principles <http://agilemanifesto.org/principles.html>
- Agile manifesto <https://agilemanifesto.org/>

Agile Development

- Advantages:
 - risk mitigation;
 - team learning;
 - improved quality;
 - user involvement;
 - eliminating much of the modeling and documentation overhead.
- Disadvantages:
 - lack of actual documentation;
 - challenges in managing the life cycle;
 - challenges in development large mission-critical systems.

Plan-driven vs Agile Development

- Plan-driven development
 - A plan-driven approach to software engineering is based around separate development stages with the outputs to be produced at each of these stages planned in advance.
 - Iteration occurs within activities.
- Agile development
 - Specification, design, implementation and testing are interleaved and the outputs from the development process are decided through a process of negotiation during the software development process.

Plan-driven vs Agile Development - Exercise

- Either in groups or by yourself brainstorm some situations where companies would might prefer plan-driven approach over agile development and vice versa.

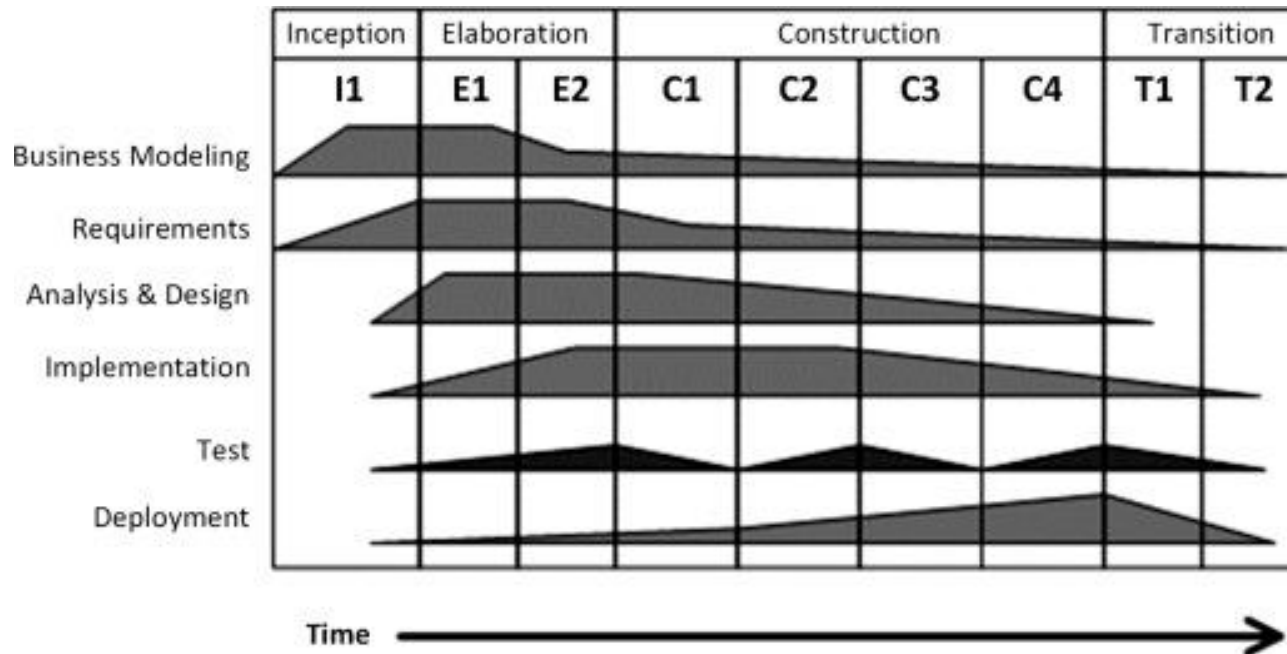
Exercise: Plan-driven – Sample Scenarios

- Well-Defined Requirements:
 - Construction projects, especially large-scale structures like skyscrapers, often follow a plan-driven approach. The requirements are well-defined, and the project typically involves extensive upfront planning, design, and engineering.
- Fixed Scope and Budget:
 - Government contracts for infrastructure projects, where the scope, budget, and timelines are fixed, often favor a plan-driven approach. These projects require detailed planning and adherence to predetermined specifications.
- Extensive Upfront Planning:
 - The manufacturing of medical devices, especially those subject to regulatory approval, typically follows a plan-driven approach. Stringent quality control processes, documentation requirements, and adherence to regulatory standards are critical.
- Very large companies:
 - In environments where the development team is spread out, where constant meetings cannot occur plan driven is sometimes the better approach

Exercise: Agile Development – Sample Scenarios

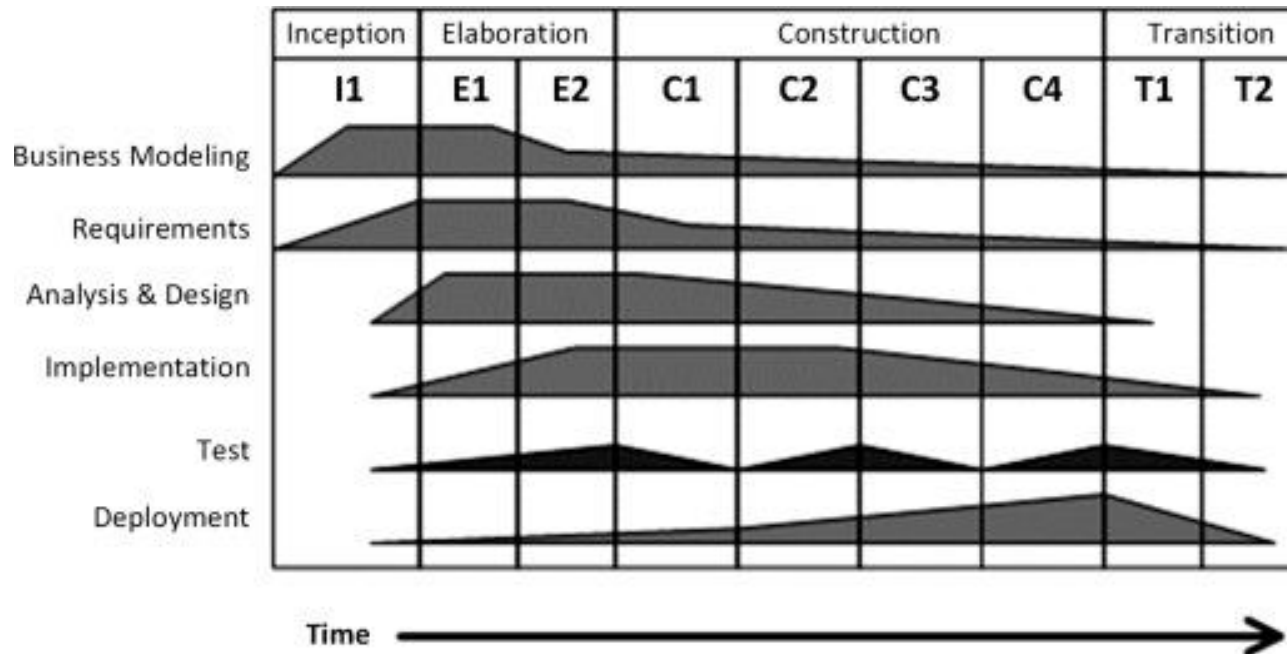
- Changing Requirements:
 - Most obvious is software development. The dynamic nature of software requirements, the need for quick iterations, and the importance of user feedback make agile practices well-suited for this industry.
- Rapid Time-to-Market:
 - For projects where time-to-market is a critical factor, especially in competitive and dynamic industries like startups
- Highly Collaborative Environments:
 - In projects where close collaboration between stakeholders, developers, and end-users is crucial, agile methodologies excel. Agile encourages continuous communication and feedback, fostering collaboration and quick decision-making.

Unified Software Development Process Phases



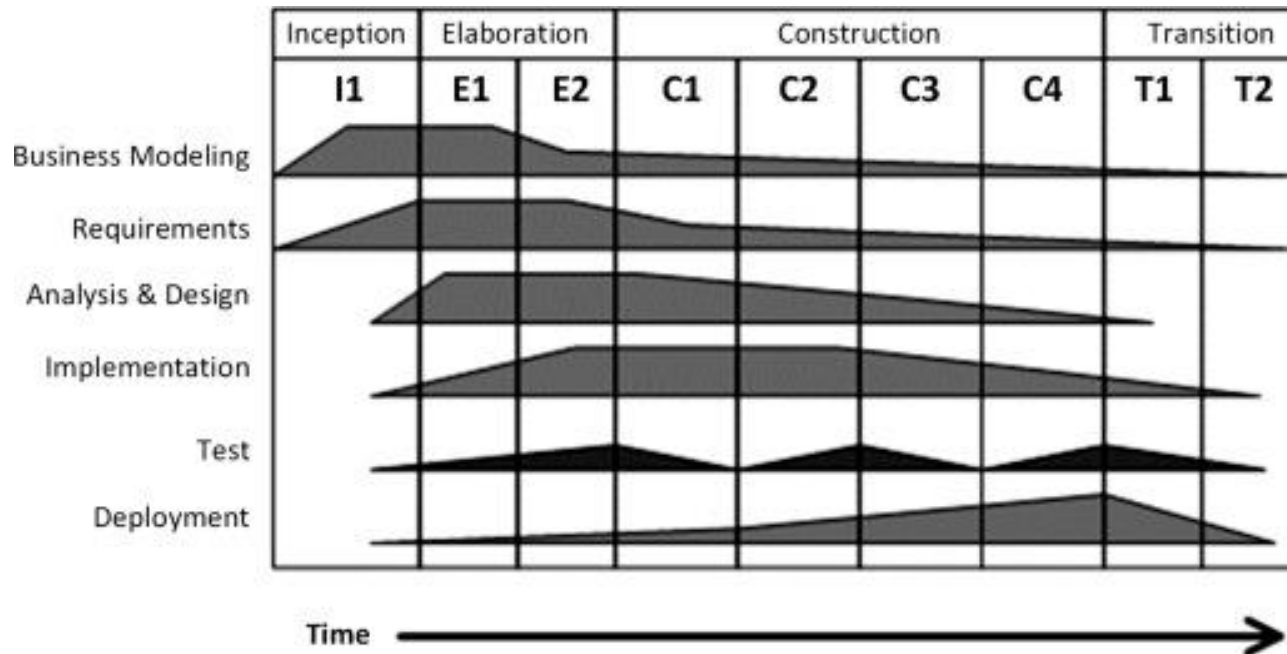
- **Inception:** establish a common vision and basic scope for the project; analysis of ~10% of the use cases, analysis of the critical non-functional requirement, creation of a business case, rough estimate for schedule, rough unreliable range of cost.

Unified Software Development Process Phases



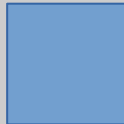
- **Elaboration:** refine the vision, identify and describe requirements, finalize the scope, determine structure of the system, design and implement core architecture, and functions, resolve high risks.

Unified Software Development Process Phases



- **Construction:** build the system, implement low risk, predictable, and easier elements and prepare for deployment.
- **Transition:** complete testing and deployment

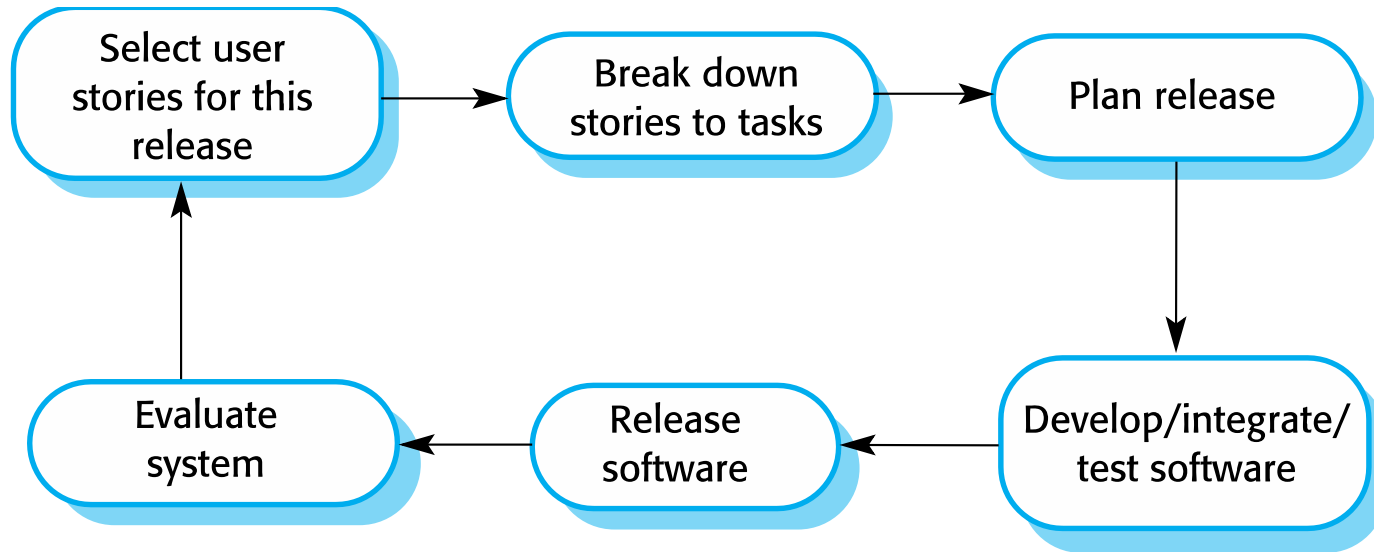
Unified Software Development Process

	Phases							
	Inception		Elaboration		Construction		Transition	
Requirements								
Analysis								
Design								
Implementation								
Test								

Extreme Programming

- Extreme Programming (XP) is an agile software development methodology that emphasizes collaboration, flexibility, and customer satisfaction.
- XP is designed to accommodate changing requirements and foster a dynamic and responsive approach to software development.
- New versions may be built several times per day;
- Increments are delivered to customers every 2 weeks;
- All tests must be run for every build and the build is only accepted if tests run successfully.

Extreme Programming



Significance of Extreme Programming

- Incremental development is supported through small, frequent system releases.
- Customer involvement means full-time customer engagement with the team.
- People not process through pair programming, collective ownership and a process that avoids long working hours.
- Change supported through regular system releases.
- Maintaining simplicity through constant refactoring of code.

Extreme Programming Practices

- Key practices
 - User stories for specification
 - Refactoring
 - Test-first development
 - Pair programming

User Stories

- In XP, a customer or user is part of the XP team and is responsible for making decisions on requirements.
- User requirements are expressed as user stories or scenarios.
- These are written on cards and the development team break them down into implementation tasks. These tasks are the basis of schedule and cost estimates.
- The customer chooses the stories for inclusion in the next release based on their priorities and the schedule estimates.
- User stories are short, simple descriptions of a feature or functionality told from the perspective of the person (or role) who desires the new capability.
- They are a common practice in agile development methodologies, providing a user-centric approach to defining requirements and capturing the intended value of a feature.

Components of User Stories

- Role:
 - The role represents the person or entity for whom the feature is intended. It could be an end-user, a customer, or any stakeholder involved in the product.
- Feature:
 - The feature is a concise and specific description of the functionality or capability that the user desires. It outlines what the system should do or what problem it should solve. Features are often written in plain language to ensure clarity and understanding among all team members.
- Reason:
 - The reason provides the motivation or benefit behind the desired feature. It answers the question of why the feature is important or valuable to the user and the overall project. Understanding the reason helps the development team prioritize and focus on delivering features that provide the most significant impact.

User Stories in Practice

- Once you get to your actual project you will work with your customer to develop user stories
- During your XP Spike, the team will write the user stories, refine them and estimate them

How to make User Stories?

- Identify Stakeholders
 - Gather key stakeholders, including end-users, customers, and anyone with a vested interest in the product.
- Understand User Needs
 - Engage in conversations with stakeholders to understand their needs, expectations, and goals.
- Define Roles
 - Identify the primary roles or personas involved in using the product. Each user story should focus on the perspective of a specific role to maintain clarity and relevance.
- Use the "As a [Role], I want [Feature], So That [Reason]" Format:
 - As a customer, I want to be able to track my order status so that I can plan for its delivery.
- Keep Stories Independent and Small
 - User stories should be small, specific, and independent to facilitate better prioritization and incremental development. Avoid creating overly complex or interconnected stories.

What to do after making User Stories?

- **Prioritize User Stories:**
 - Collaborate with the product owner and stakeholders to prioritize user stories based on business value, user impact, and project goals. Prioritization helps the team focus on the most important features first.
- **Add Acceptance Criteria:**
 - Detail the acceptance criteria for each user story. Acceptance criteria define the conditions that must be met for a story to be considered complete. They serve as a shared understanding of what it means for a feature to be done.
- **Refine and Iterate:**
 - Regularly revisit and refine user stories, especially during backlog grooming sessions. As the team gains more information or the project evolves, user stories may need to be adjusted or split into smaller stories.

User Stories Exercise

- Say a Bookstore Owner wants to create an Online Bookstore and gives you this project description:
- **Overview:** Develop an online bookstore platform that allows users to browse, search, and purchase books. The platform should provide an intuitive user experience, support multiple genres, and include features for user accounts, reviews, and recommendations.
- Either in groups or by yourself identify potential user stories for various roles

User Stories Exercise – Sample User Stories

- As a visitor, I want to see a homepage showcasing featured books, bestsellers, and new releases so that I can quickly discover popular titles and promotions.
- As a registered user, I want to create an account and log in so that I can save my preferences, track my orders, and leave reviews.
- As a book enthusiast, I want to search for books by title, author, or genre so that I can easily find and explore books of interest.
- As a shopper, I want to view detailed information about a book, including its description, price, and availability, so that I can make an informed purchase decision.
- As a user, I want to add books to my shopping cart, review my selections, and proceed to checkout easily.
- As a book club organizer, I want the option to purchase multiple copies of the same book in a single order to facilitate group orders.
- As a Bookstore Owner, I want to have an easy-to-manage inventory system, so I can efficiently update book availability, add new titles, and track sales.
- As a Finance Manager, I want to generate sales reports and track revenue, so I can analyze the bookstore's financial performance and make informed business decisions.

User Stories Estimates

- Make estimates out loud in front of other people of how long you think it will take you to complete the task
- Ex: "I could do that in about two hours"
 - The way to improve is to continue to make estimates and note the difference between your estimates and actual times
- After making your estimate go away and work on the task
 - Then see how long it took you to complete the task and use that as data going forward
- By the end of the spike/sprint you will have made a lot of estimates and getting better at estimating the tasks
- It is important to do these estimates as we get further on in the project lifecycle

Product Backlog

- The product backlog is an ordered list of "requirements" that is maintained for a product.
- It contains Product Backlog Items ordered by the Product Owner based on considerations like risk, business value, dependencies, date needed, etc.
- Higher-priority user stories appear at the top of the Product Backlog, ensuring that the team works on the most valuable features first
- User stories in the Product Backlog undergo iterative refinement as more details become available or as the project evolves. This process helps ensure that the development team has a clear understanding of what needs to be implemented in each sprint.

Sprint Backlog

- Before each sprint, the team selects a set of user stories from the Product Backlog to work on. These selected user stories form the Sprint Backlog for the upcoming sprint.
- The sprint backlog is the list of work the Development Team must address during the next sprint.
- The list is derived by selecting stories/features from the top of the product backlog until the Development Team feels it has enough work to fill the sprint.
- This is done by the Development Team asking "Can we also do this?" and adding stories/features to the sprint backlog.
- The Development Team should keep in mind the velocity of its previous Sprints (total story points completed from each of the last sprints stories) when selecting stories/features for the new sprint, and use this number as a guide line of how much "effort" they can complete.

Upcoming

- Quiz on Wed
- Lab Assignment 1 – XP Spike 1

References

- Object-Oriented Systems Analysis and Design Using UML by Simon Bennett, Ray Farmer (2010) ISBN: 978-0077125363
- Dennis, Alan, et al. System Analysis and Design. Wiley, 2013.
- Object-Oriented Analysis and Design with Applications. Pearson Education (US), 2011.
- "Object Oriented Analysis and Design Tutorial." [Www.tutorialspoint.com](http://www.tutorialspoint.com), Tutorials Point, www.tutorialspoint.com/object_oriented_analysis_design/.
- Craig Larman, Introduction to Object-Oriented Analysis and Design and Iterative Development, Pearson, 2004
- Omeecode. "Object Oriented Analysis and Design." CodeProject - For Those Who Code, CodeProject, www.codeproject.com/Articles/1137299/Object-Oriented-Analysis-and-Design.
- "Reasons for Software Project Failure Globally 2015 | Statistic." Statista, www.statista.com/statistics/627648/worldwide-software-developer-survey-project-failure/.

COSC 205

Agile Project Management

Pair Programming



Agile Project Management

- The principal responsibility of software project managers is to manage the project so that the software is delivered on time and within the planned budget for the project.
- The standard approach to project management is plan-driven. Managers draw up a plan for the project showing what should be delivered, when it should be delivered and who will work on the development of the project deliverables.
- Agile project management requires a different approach, which is adapted to incremental development and the practices used in agile methods.

Scrum

- Scrum is an agile method that focuses on managing iterative development rather than specific agile practices.
- There are three phases in Scrum.
 - The initial phase is an outline planning phase where you establish the general objectives for the project and design the software architecture.
 - This is followed by a series of sprint cycles, where each cycle develops an increment of the system.
 - The project closure phase wraps up the project, completes required documentation such as system help frames and user manuals and assesses the lessons learned from the project.

Scrum Team

- Product Owner
 - Holds the vision for the product and prioritizes the product backlog
 - Represents the stakeholders and is the voice of the customer
 - Can also be a developer, but not the Scrum master as well.
- Scrum Master
 - Responsible for ensuring that the Scrum process is followed and guides the team in the effective use of Scrum.
 - Their purpose is to help solve anything blocking the team, so that they can continue to the delivery at the end of the sprint.
- Development Team
 - Responsible for delivering a potentially shippable product at the end of each sprint.
 - Good to have a mix of different skills and expertise on the team.
 - The team is self-organizing, and works with the management rather than for.

Scrum Terminology

- Product Backlog
 - An emergent, ordered list of what is needed to improve the product and includes the product goal.
- Sprint Backlog
 - The set of product backlog items selected for the sprint by the developers (team members), plus a plan for delivering the increment and realizing the sprint goal.
- Increment
 - A sum of usable sprint backlog items completed by the developers in the sprint that meets the definition of done, plus the value of all the increments that came before. Each increment is a recognizable, visibly improved, operating version of the product.
- Sprint
 - The heartbeat of scrum. Each sprint should bring the product closer to the product goal and is a month or less in length.
 - A development iteration. Sprints are usually 2-4 weeks long.

Scrum – Sprint Cycle

- Scrum teams work in short timeframes called sprints. Sprints can be as short as one week or as long as one month. Sprints happen one right after the other, with no breaks, to maintain a steady project cadence.
- Each sprint begins with a plan (sprint planning) and ends with a review of the work completed (sprint review) and an additional look back at the way in which the team worked together (sprint retrospective).
- During each sprint, the entire scrum team works together to complete one or more increments of a larger overall product or project.
- The idea is to deliver small batches of functionality that stakeholders can see and inspect at the end of every sprint.
- Then, based on that feedback, the scrum team can adapt their plans for the next batch of functionality.
- By learning early what works and what doesn't and whether an increment matches stakeholder expectations, the scrum team is ultimately able to deliver a full product that both satisfies and also delights customers.

Scrum – Sprint Cycle

- The starting point for planning is the product backlog, which is the list of work to be done on the project.
- The selection phase involves all of the project team who work with the customer to select the features and functionality from the product backlog to be developed during the sprint.
- Once these are agreed, the team organize themselves to develop the software.
- During this stage the team is isolated from the customer and the organization, with all communications channeled through the so-called 'Scrum master'.
- The role of the Scrum master is to protect the development team from external distractions.
- At the end of the sprint, the work done is reviewed and presented to stakeholders. The next sprint cycle then begins.

Scrum – Sprint Cycle

▪ **Sprint Planning**

- The entire scrum team establishes the sprint goal, what can be done, and how the chosen work will be completed. Planning should be timeboxed to a maximum of 8 hours for a month-long sprint, with a shorter timebox for shorter sprints.

▪ **Sprint Review**

- The entire scrum team inspects the sprint's outcome with stakeholders and determines future adaptations. Stakeholders are invited to provide feedback on the increment.

▪ **Sprint Retrospective**

- The scrum team inspects how the last sprint went regarding individuals, interactions, processes, tools, and definition of done. The team identifies improvements to make the next sprint more effective and enjoyable. This is the conclusion of the sprint.
- Two main questions are asked in the sprint retrospective: What went well during the sprint? What could be improved in the next sprint?

Scrum – Daily Scrum

- The 'Scrum master' is a facilitator who arranges daily meetings, tracks the backlog of work to be done, records decisions, measures progress against the backlog and communicates with customers and management outside of the team.
- The whole team attends short daily meetings (daily scrums) where all team members share information, describe their progress since the last meeting, problems that have arisen and what is planned for the following day.
- This means that everyone on the team knows what is going on and, if problems arise, can re-plan short-term work to cope with them.

Scrum – Daily Scrum

- Each day during the sprint, a project status meeting occurs. This is called a daily scrum or a daily standup. This meeting has specific guidelines:
 - The meeting starts precisely on time.
 - All are welcome, but normally only the core roles speak.
 - The meeting length is set (timeboxed) to 15 minutes.
 - The meeting should happen at the same location and same time every day.
- During the meeting, each team member answers three questions:
 - What have you done since yesterday?
 - What are you planning to do today?
 - Any impediments/stumbling blocks?
 - Scrum Master generally leads this meeting but Product Owner can step in if needed

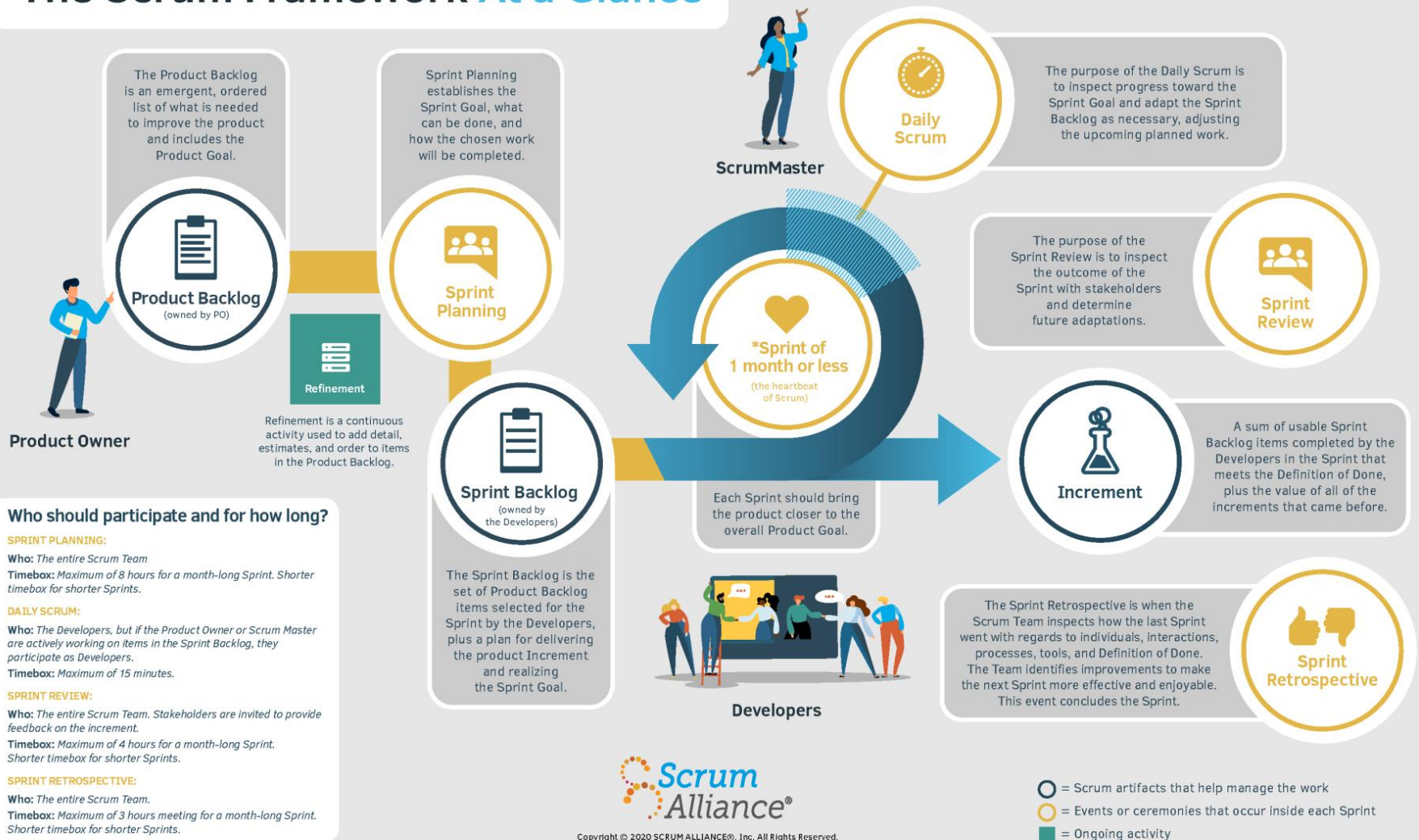
Scrum Benefits

- The product is broken down into a set of manageable and understandable chunks.
- Unstable requirements do not hold up progress.
- The whole team have visibility of everything and consequently team communication is improved.
- Customers see on-time delivery of increments and gain feedback on how the product works.
- Trust between customers and developers is established and a positive culture is created in which everyone expects the project to succeed.

Scrum

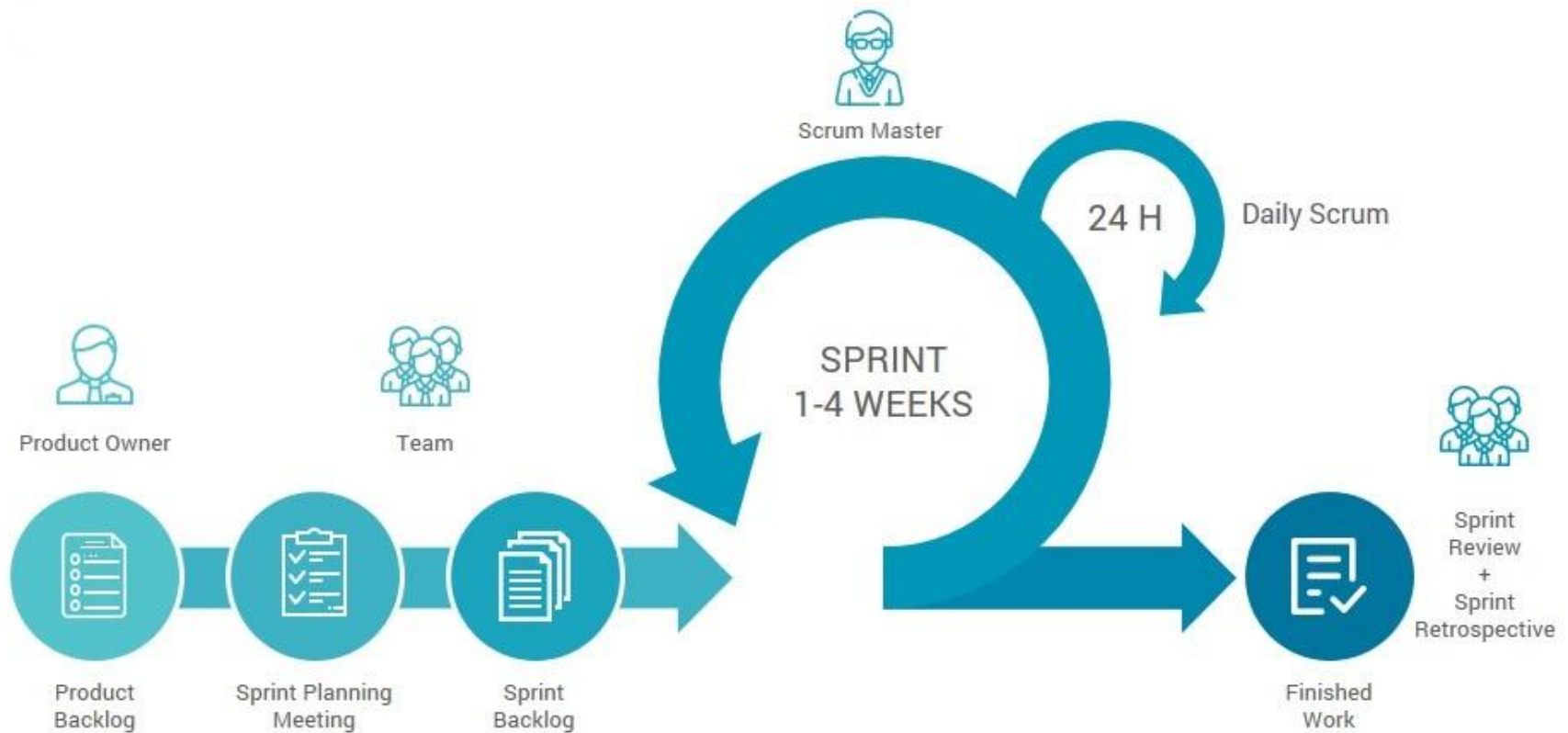
Version 5.0

The Scrum Framework At a Glance



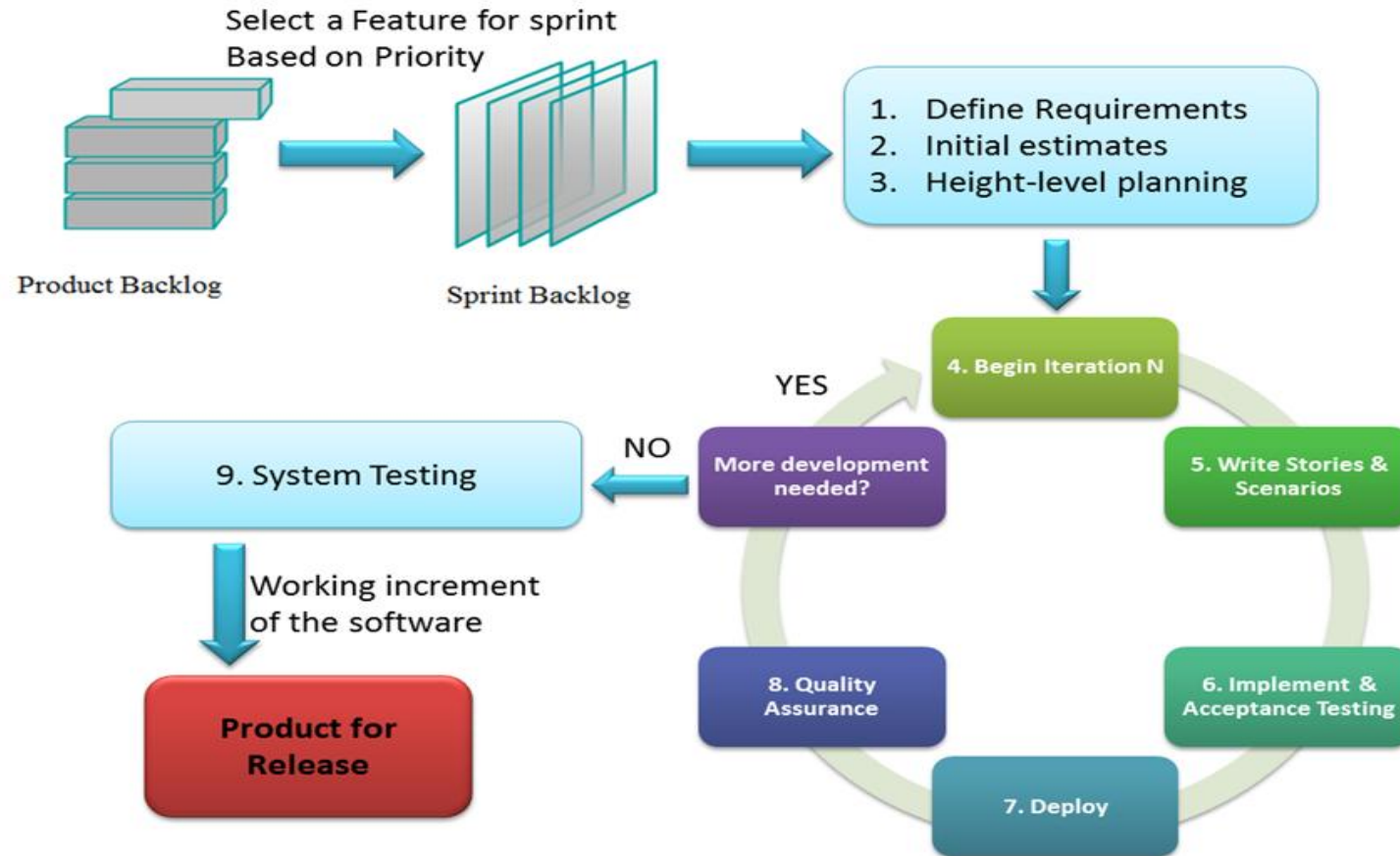
Scrum

Scrum Framework



<https://www.verbindungszentrum.com/scrum-en/>

Scrum



<https://clickhowto.com/how-to-understand-agile-scrum-methodology-and-its-uses/>

XP - Pair Programming

- Pair programming involves programmers working in pairs, developing code together.
- This helps develop common ownership of code and spreads knowledge across the team.
- It serves as an informal review process as each line of code is looked at by more than 1 person.
- It encourages refactoring as the whole team can benefit from improving the system code.

XP - Pair Programming

- In pair programming, programmers sit together at the same computer to develop the software.
- Pairs are created dynamically so that all team members work with each other during the development process.
- The sharing of knowledge that happens during pair programming is very important as it reduces the overall risks to a project when team members leave.
- Pair programming is not necessarily inefficient and there is some evidence that suggests that a pair working together is more efficient than 2 programmers working separately.

XP - Pair Programming Roles

- Two distinct roles in a pair
- **Driver**
 - Controls keyboard and mouse
 - Responsible for creating the code the pair is currently working on
 - Ex: If they are writing a method the driver types in the method signature and starts working on the method body
 - Actively talking to the co-pilot
- **Copilot**
 - Actively paying attention to everything going on
 - Watches what the driver is doing and makes syntactical corrections and suggestions
 - At a moment's notice co-pilot should be able to take over for the driver
 - Should continuously process and evaluate the information passing before them

XP - Pair Programming Swapping Roles

- When the situation dictates the copilot and driver should swap roles
- Switching roles is a fundamental and necessary part of pairing
- It stimulates the flow of ideas, and it keeps the pairs mentally fresher by changing their activities
- So, as a co-pilot you should always be looking for queues or scenarios where it is appropriate to switch roles

XP - Pair Programming Exercise

- Working in pairs assign one person to be the driver and other to be the co-pilot (after 5 minutes switch roles)
- Working with a partner, one pencil, one sheet of paper (or on Microsoft paint), and no external reference, draw, to the best of your recollection, a map of Canada
 - Bonus points for listing the province names and capital cities

XP - Pair Programming in Practice

- Question:
- Could you and the other two team members have worked on your tasks individually instead of collaboratively? If so, do you think the work would have been completed sooner? Why or why not?

XP - Pair Programming in Practice

- You possibly and probably would have finished sooner
- But a key facet of XP is that every line of code that is delivered is written by two people sitting at the same keyboard working together. Hence, no code written by one programmer gets delivered to the client
- Why is this important?

XP – Benefits of Pair Programming

- Pair programming provides advantages that individual programming cannot
 - Real-Time Code Reviews
 - Avoiding Distractions
 - Managing for Two
 - Knowledge and Information Migration

XP – Real-Time Code Reviews

- Driver can concentrate on producing while the copilot can focus on catching mistakes
- Usually typos, cognitive dropouts and invalid assumptions

XP – Avoiding Distractions

- Time you set aside for pairing is a valuable commodity
- When you are working by yourself oftentimes you get distracted by other things (extracurricular activities, family etc...)
 - But when pair programming you have set out time where you and your partner write sustainable code and have trust your partner is giving you their full time as well

XP – Managing for Two

- Forces thought and evaluation of possible solutions
- Two programmers working on the same problem agree on the same approach, then it increases their confidence that the approach is reasonable
 - If they disagree then it naturally forces discussion

XP – Knowledge and Information Migration

- Witness others on your team code and start to pick up different programming habits and techniques
 - Ex: Shortcuts, software skills, coding practices
- Additionally, knowledge of the developing software also spreads across the team
 - Helps to promote collective code ownership
 - This is very helpful in cases where someone might be sick and not able to make the meeting so another person can step up and explain the code to the rest of the group

XP - Pair Programming in Practice

- Each programmer signs up for their tasks for the week and estimate them
- Think of how much faster you would work if you could get your classmate to put everything aside and help you with your tasks
- So, you pair up and complete a task you estimate to complete in 6 hours in 4 hours
- Hence, a 6-hour task for the group cost the group 8 hours (4 from you, and 4 from partner)
- Now you have finished your tasks. But, that doesn't mean you can take the rest of the week off. So, you pair with another student and help him finish his task. You learn some new coding technique along the way as well.

Upcoming

- XP Spike 2 - Lab Assignment 2 (has been released last Friday) due this Friday noon

References

- Object-Oriented Systems Analysis and Design Using UML by Simon Bennett, Ray Farmer (2010) ISBN: 978-0077125363
- Dennis, Alan, et al. System Analysis and Design. Wiley, 2013.
- Object-Oriented Analysis and Design with Applications. Pearson Education (US), 2011.
- "Object Oriented Analysis and Design Tutorial." [Www.tutorialspoint.com](http://www.tutorialspoint.com), Tutorials Point, www.tutorialspoint.com/object_oriented_analysis_design/.
- Craig Larman, Introduction to Object-Oriented Analysis and Design and Iterative Development, Pearson, 2004
- Omeecode. "Object Oriented Analysis and Design." CodeProject - For Those Who Code, CodeProject, www.codeproject.com/Articles/1137299/Object-Oriented-Analysis-and-Design.
- "Reasons for Software Project Failure Globally 2015 | Statistic." Statista, www.statista.com/statistics/627648/worldwide-software-developer-survey-project-failure/.

COSC 205

Refactoring

Test Driven Development

Synergies



Refactoring

- Conventional wisdom in software engineering is to design for change
- It is worth spending time and effort anticipating changes as this reduces costs later in the life cycle
- XP, however, maintains that this is not worthwhile as changes cannot be reliably anticipated
- Rather, it proposes constant code improvement (refactoring) to make changes easier when they have to be implemented

Refactoring

- Programming team look for possible software improvements and make these improvements even where there is no immediate need for them
- This improves the understandability of the software and so reduces the need for documentation
- Changes are easier to make because the code is well-structured and clear
- You can think of refactoring as 'preventative maintenance' that reduces the problems of future change
- Refactoring involves modifying a program to improve its structure, reduce its complexity or make it easier to understand
- When you refactor a program, you should not add functionality but rather concentrate on program improvement

Examples of Refactoring

- Re-organization of a class hierarchy to remove duplicate code
- Tidying up and renaming attributes and methods to make them easier to understand
- The replacement of inline code with calls to methods that have been included in a program library
- Practical example
 - A longer method has a part of it used to calculate the subtotal, a part of it to calculate the tax, and a part of it to calculate the total
 - Separate them into three methods called `calculateSubTotal()`, `calculateSalesTax()` and `calculateTotal()`
 - Now your code is both more readable and more modular

'Bad smells' in program code

- Duplicate code
 - The same or very similar code may be included at different places in a program
 - This can be removed and implemented as a single method or function that is called as required
- Long methods
 - If a method is too long, it should be redesigned as a number of shorter methods
- Switch (case) statements
 - These often involve duplication, where the switch depends on the type of a value
 - The switch statements may be scattered around a program

Refactoring with the Project

- Early on you might not see many opportunities for refactoring especially when pair programming
 - But maybe you hold code review sessions with the whole group once a week
 - More eyes looking will produce more suggestions of how to improve
- Another important aspect of refactoring is having tests written for your code
 - This way when you undergo refactoring you know you aren't breaking anything because the tests will continue to run
 - Could be scenarios however, where you must refactor your test code as well

Refactoring – Exercise 1

- Consider a simple method that calculates the area of a rectangle.

```
public class RectangleCalculator {  
    public double calculateArea(double length, double width) {  
        double a; // 'a' stands for area  
        a = length * width; // Calculation of area  
        return a;  
    }  
}
```

- How can we refactor this?

Refactoring – Exercise 1 (Solution)

```
public class RectangleCalculator {  
    public double calculateArea(double length, double width) {  
        return length * width;  
    }  
}
```


Refactoring – Exercise 2

- Consider some methods that take in some String parameters to check if they are null and empty

```
public void processUserData(String name) {  
    if (name != null && !name.isEmpty()) {  
        // process user data  
    } else {  
        System.out.println("Invalid");  
    }  
}  
  
public void processProductData(String productName) {  
    if (productName != null && !productName.isEmpty()) {  
        // process user data  
    } else {  
        System.out.println("Invalid");  
    }  
}
```

How can we refactor this?

Refactoring – Exercise 2 (Solution)

- Notice how both methods just check to see if the string is both not null and not empty

```
public void processUserData(String name) {  
    if checkIfEmpty(name) {  
        // process user data  
    } else {  
        System.out.println("Invalid");  
    }  
}  
  
public void processProductData(String productName) {  
    if checkIfEmpty(productName){  
        // process user data  
    } else {  
        System.out.println("Invalid");  
    }  
}  
  
public boolean checkIfEmpty(String name) {  
    if (name != null && !name.isEmpty()){  
        return true;  
    } else return false;  
}
```

Test-first Development (Test-driven Development)

- Test-Driven Development (TDD) is a fundamental practice in Extreme Programming (XP)
- TDD involves writing tests before implementing the actual code, with the aim of ensuring that the code meets the specified requirements and behaves as expected
- The TDD process in XP follows a cycle of writing tests, implementing code, and then refactoring the code while keeping all tests passing
- This iterative cycle helps improve code quality, maintainability, and reliability

Test-first Development (Test-driven Development)

- Writing tests before code clarifies the requirements to be implemented
- Tests are written as programs rather than data so that they can be executed automatically
- The test includes a check that it has executed correctly.
 - Usually relies on a testing framework such as Junit.
- All previous and new tests are run automatically when new functionality is added, thus checking that the new functionality has not introduced errors
- Shorthand version:
 - Write a test to verify successful completion of the task before writing the code to perform the task
 - Before you write your next piece of code, think about how you might test it

Steps to Test-first Development

- Write a test case
 - Write the test case that will determine when the subtask is successfully completed
 - Ex: Suppose you decide the next task is to find the largest of three given integers

Test Case

```
if (maxOfThreeInts(1,7,3) == 7 ){  
    System.out.println("Passed Test");  
} else{  
    System.out.println("Failed Test"); }
```

Steps to Test-first Development

- This is a simple test example. There is a disadvantage in that you must look in your output to find “Passed Test”. Better to have tests automated
- In writing this method we decided to use a method that takes three integers as input parameters and produces one integer output

```
public int maxOfThreeInts(int num1, int num2, int num 3)
```

Steps to Test-first Development

- Passing the Test
- After method signature is written now is time to write the method
- The goal here is to write the simplest code possible to pass the test

```
public int maxOfThreeInts(int num1, int num2, int num 3){  
    int max = num1;  
    if (num2 >= num1 && num2 >= num3) max = num2;  
    if (num3 >= num1 && num3 >= num2) max = num3;  
    return max;  
}
```

- There are obviously other ways to do this, but you also don't want to wind up with something slicker that doesn't work
- Important early on to implement something simple and straightforward that passes the test

Steps to Test-first Development

- But it is important to not be too simple
- Ex:

```
public int maxOfThreeInts(int num1, int num2, int num 3){  
    return num2;  
}
```

- This code will pass the test, but it is insufficient
- These types of implementations are useful as a starting point, but it immediately requires another representative test case with the max in a different position

Steps to Test-first Development

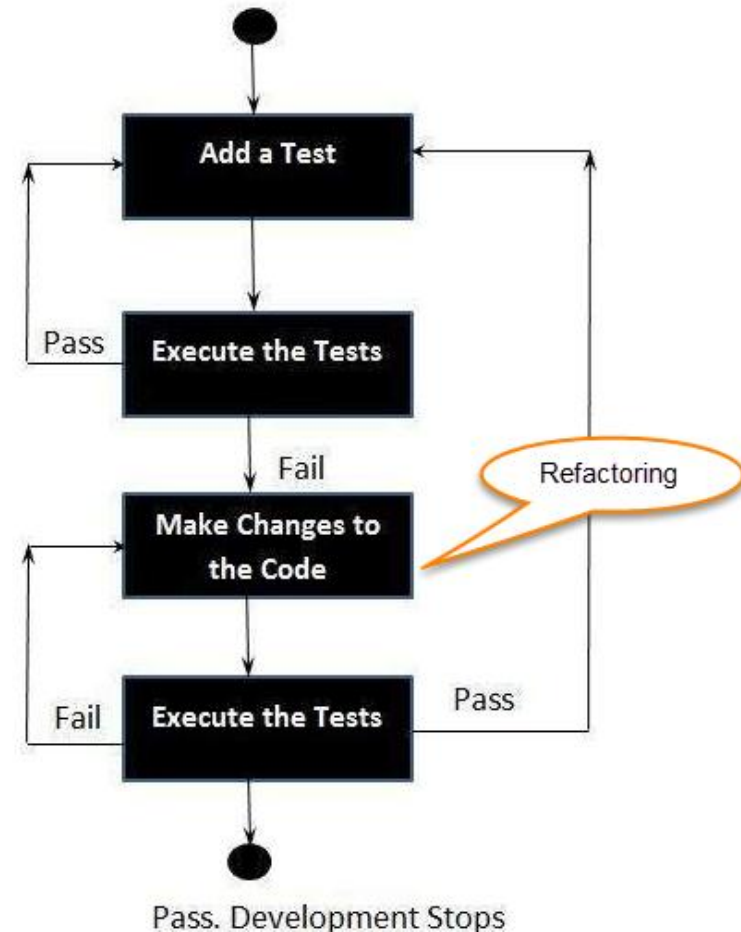
Add a test.

Run all tests and see if any new test fails.

Write some code.

Run tests and Refactor code.

Repeat.

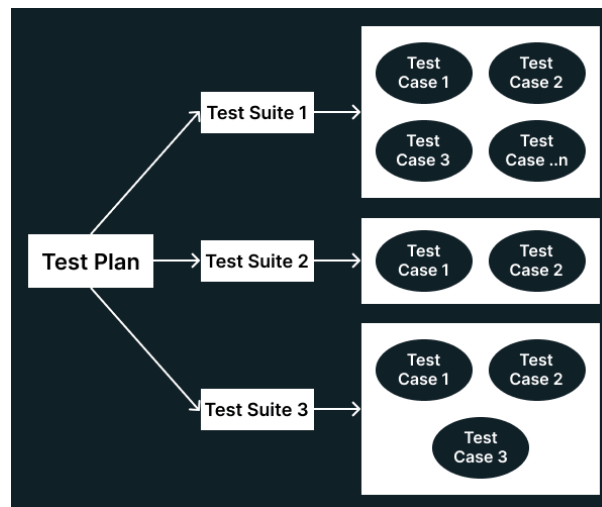


Exhaustive Testing

- Why not just write test cases to handle all the possible scenarios?
- Dijkstra proclaimed in his *Humble Programmer* speech “Program testing can be a very effective way to show the presence of bugs, but it is hopelessly inadequate for showing their absence”
- This claim is another way of saying it is not possible to prove a program is correct through testing
- There is essentially no such thing as defect-free software
- Also, exhaustive testing can be time consuming, difficult and at times over complicated

Test Suite

- Overall goal of a software development project is to create a usable, reliable system that works
- In an XP project, creating tests – collectively known as the test suite – is equally important
- As the development progresses the number of test cases and the breadth of the test suite increases, making it more useful and valuable
- However, to be useful as well as complete, the test suite must be extremely easy to run. It needs to be automated



Automated Testing

- Testing first is only half the story; the other benefit is ongoing testing
- Once a test case has been written and passed, it becomes part of the test suite
- The real advantage to the development team is that they can run tests repeatedly, getting repeated feedback on state of the system
- Whenever any functionality is added to the system, the tests can be run and problems that the new code has introduced can be caught immediately.

Automated Testing

- Effective test suite:
 - Be fully automated (all tests run at the click of a button)
 - Write test cases easily
 - Interpret results
 - Be fast
- Junit is the defacto standard for XP automated testing in Java
 - There are other automated testing systems tailored to different languages
 - CppUnit (C++), PyUnit (Python), many others

Test case description for dose checking

Test 4: Dose checking

Input:

1. A number in mg representing a single dose of the drug.
2. A number representing the number of single doses per day.

Tests:

1. Test for inputs where the single dose is correct but the frequency is too high.
2. Test for inputs where the single dose is too high and too low.
3. Test for inputs where the single dose * frequency is too high and too low.
4. Test for inputs where single dose * frequency is in the permitted range.

Output:

OK or error message indicating that the dose is outside the safe range.

Test First Development - Exercise

- Think of some test cases to design for a register form that includes
 - Username
 - Password
 - Email
 - First and Last Name

Customer Involvement

- The role of the customer in the testing process is to help develop acceptance tests for the stories that are to be implemented in the next release of the system
- The customer who is part of the team writes tests as development proceeds
- All new code is therefore validated to ensure that it is what the customer needs.
- However, people adopting the customer role have limited time available and so cannot work full-time with the development team
- They may feel that providing the requirements was enough of a contribution and so may be reluctant to get involved in the testing process

Problems with test-first development

- Programmers prefer programming to testing and sometimes they take short cuts when writing tests
- For example, they may write incomplete tests that do not check for all possible exceptions that may occur
- Some tests can be very difficult to write incrementally
 - For example, in a complex user interface, it is often difficult to write unit tests for the code that implements the 'display logic' and workflow between screens
- It difficult to judge the completeness of a set of tests
- Although you may have a lot of system tests, your test set may not provide complete coverage

Synergies

- Oftentimes, you will be tempted to pick and choose among XP practices. Don't for now
- There are synergies that will arise when you combine practices
- Use the XP spike to experiment between different XP practices

Upcoming

- Lab Assignment 2 – XP Spike 2 (posted last Friday, due this Friday noon)
- Quiz

References

- Object-Oriented Systems Analysis and Design Using UML by Simon Bennett, Ray Farmer (2010) ISBN: 978-0077125363
- Dennis, Alan, et al. System Analysis and Design. Wiley, 2013.
- Object-Oriented Analysis and Design with Applications. Pearson Education (US), 2011.
- "Object Oriented Analysis and Design Tutorial." www.tutorialspoint.com, Tutorials Point, www.tutorialspoint.com/object_oriented_analysis_design/.
- Craig Larman, Introduction to Object-Oriented Analysis and Design and Iterative Development, Pearson, 2004
- Omeecode. "Object Oriented Analysis and Design." CodeProject - For Those Who Code, CodeProject, www.codeproject.com/Articles/1137299/Object-Oriented-Analysis-and-Design.
- "Reasons for Software Project Failure Globally 2015 | Statistic." Statista, www.statista.com/statistics/627648/worldwide-software-developer-survey-project-failure/.

COSC 205

236 Review

Design Overview



Object-Oriented Paradigm in Analysis

- Object-Oriented Analysis (OOA) is the process of identifying software engineering requirements and developing software specifications in terms of a software system's object model, which comprises of interacting objects.
- In object-oriented approach, requirements are organized around objects, which integrate both data and functions.
- Objects are modelled after real-world objects that the system interacts with. In traditional analysis methodologies, the two aspects - functions and data - are considered separately.

Object-Oriented Paradigm in Analysis

- The primary tasks in object-oriented analysis (OOA) are:
 - Elicit requirements: Define what does the software need to do, and what's the problem the software trying to solve.
 - Specify requirements: Describe the requirements, using use cases (and scenarios) or user stories.
 - Conceptual model: Identify objects, organize the objects by creating object model diagram, define object attributes, define the behavior of the objects, describe how the objects interact

Object-Oriented Paradigm in Design

- In OO design phase, we describe objects (class diagram from conceptual diagram), their attributes, behavior, and interactions.
 - Describe the classes and their relationships using class diagram.
 - Describe the interaction between the objects using sequence diagram.
 - Restructuring the class data
 - Implementation of associations
 - Applying the software design principles
 - Applying design patterns
- The common models used in OOD are class diagrams.
- Other diagrams are used showing interactions between objects, structure of the system, or the behavior of the system and how it responds to events.

Object-Oriented Concepts

- Object – an abstraction containing attributes and behaviors hidden inside of it and representing an entity in a problem domain.
 - Who am I?
 - What can I do?
 - What do I know?
- Class – set of objects specified in the same way.
 - In OOP class is a blue print (template) of an object.
- Abstraction represents the behavior of a real world entity.

Object-Oriented Concepts

- Object is a real world element in OO environment.
 - **identity** that distinguishes it from the other objects in the system
 - **state**: properties and their values
 - **behaviour**: externally visible activities performed by an object effecting its status

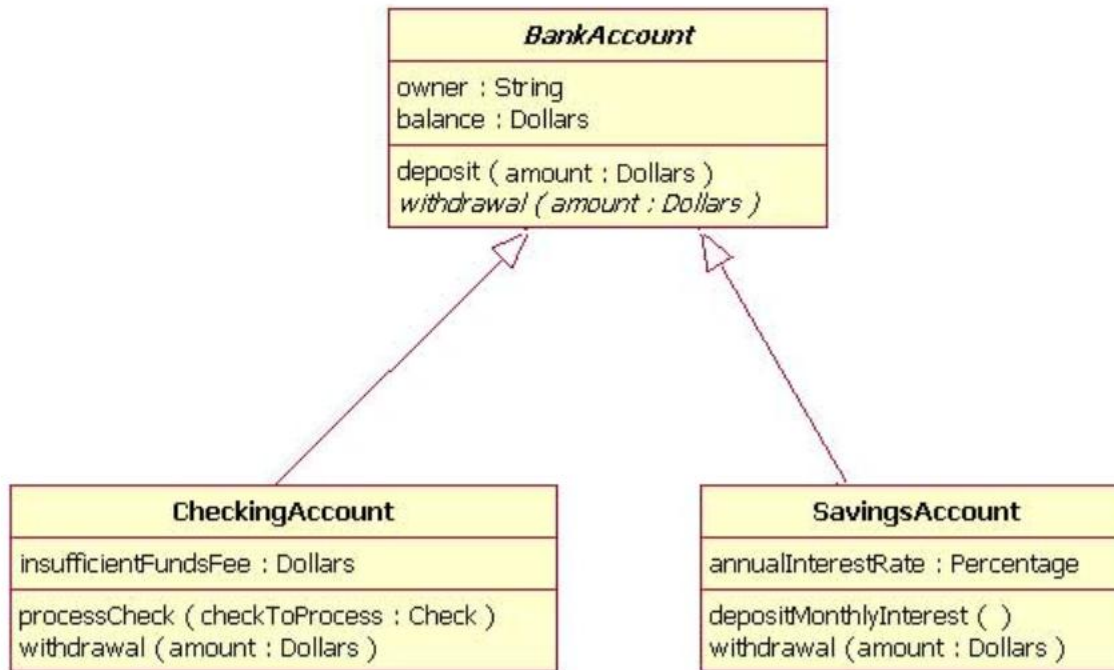
Object-Oriented Concepts

- A class is a specification of a set of objects having the same characteristic properties that exhibit common behaviour.
 - attributes (contains data) for the objects to be instantiated.
 - operations (specifies behaviour).
 - operations that represent the behaviour of the objects of the class.
 - Functions or methods.

Class_Name
- attribute
+ operation()

Object-Oriented Concepts: Inheritance

- Inheritance is the mechanism of making new classes from existing ones.



Is-a relationship

Object Oriented Analysis and Design Concepts

- The most important purpose of OO analysis is to identify objects of a system to be designed.
- The analysis is also done for an existing system.
- An efficient analysis -> how objects can be identified.
- Relationships are identified -> design.

Object Oriented Analysis

- During OO analysis:
 - requirements analysis
 - identify objects and describe them in a proper way (object attributes)
 - identify objects responsibilities (functions performed by the objects)
 - describe how the objects interact

Object Oriented Design

- The second phase is OO design:
 - requirements fulfillment
 - restructuring implementing classes
 - implementation of methods (internal structures and algorithms)
 - implementation of controls
 - implementation of associations

Object Oriented Implementation

- The third phase is OO implementation.
 - In this phase, the design is implemented using OO languages such as Java, C++, etc.

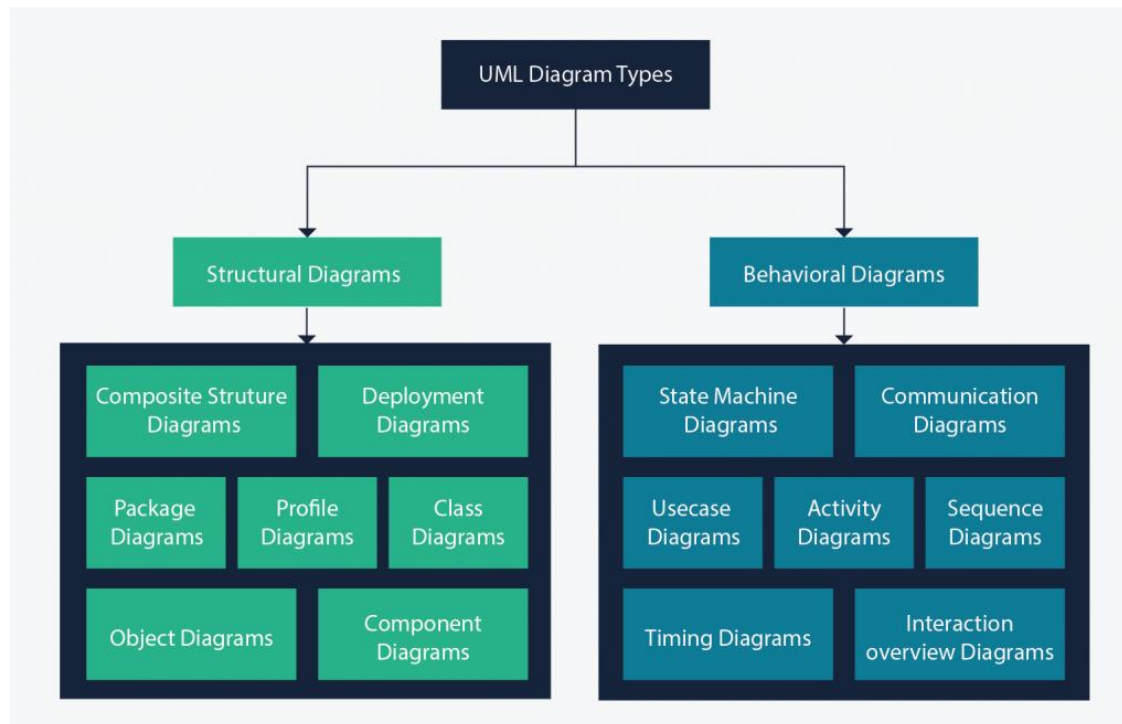
Purpose of Object-Oriented Analysis and Design

- Identifying the objects of a system.
- Identifying their relationships.
- Making a design, which can be converted to executables using OO languages.

OO Analysis → OO Design → OO implementation using OO languages

Object-Oriented Analysis and Design Modelling

- The Unified Modeling Language (UML) is a standard visual modeling language:
 - modeling business and similar processes
 - analysis, design, and implementation of software-based systems



UML

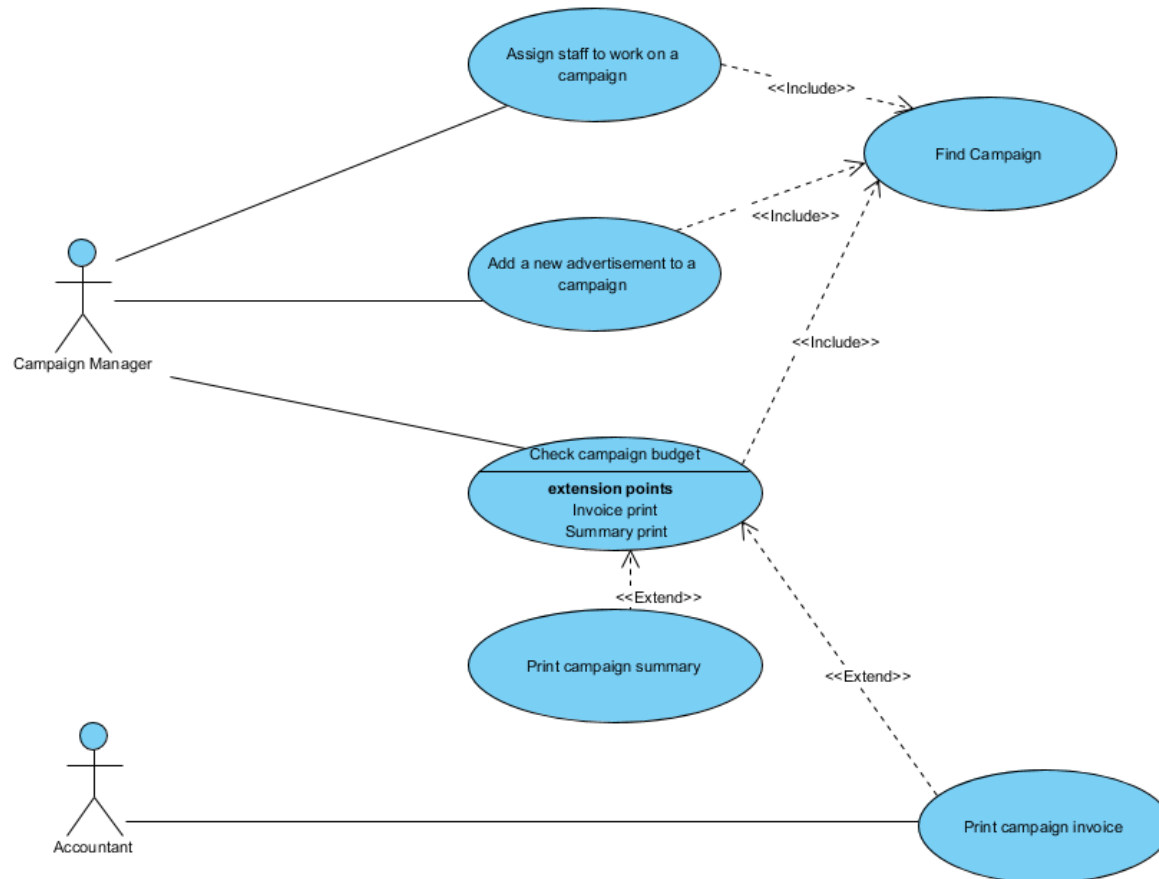
- Conceptual perspective
 - description of the concepts of a domain of study
- Software perspective
 - the elements of the UML directly mapped to elements in a software system (forward engineering). Tools can be used to generate code in various languages using UML diagrams.

Modelling using UML

- Model
 - abstract representation of existing or future system.
- Diagram
 - visual representation of some part of a model.
- UML diagram corresponds to one view of a model of a system.
- Different types of diagrams: different aspects are either emphasized or omitted.
- Diagrams combined result in a good model of a system.

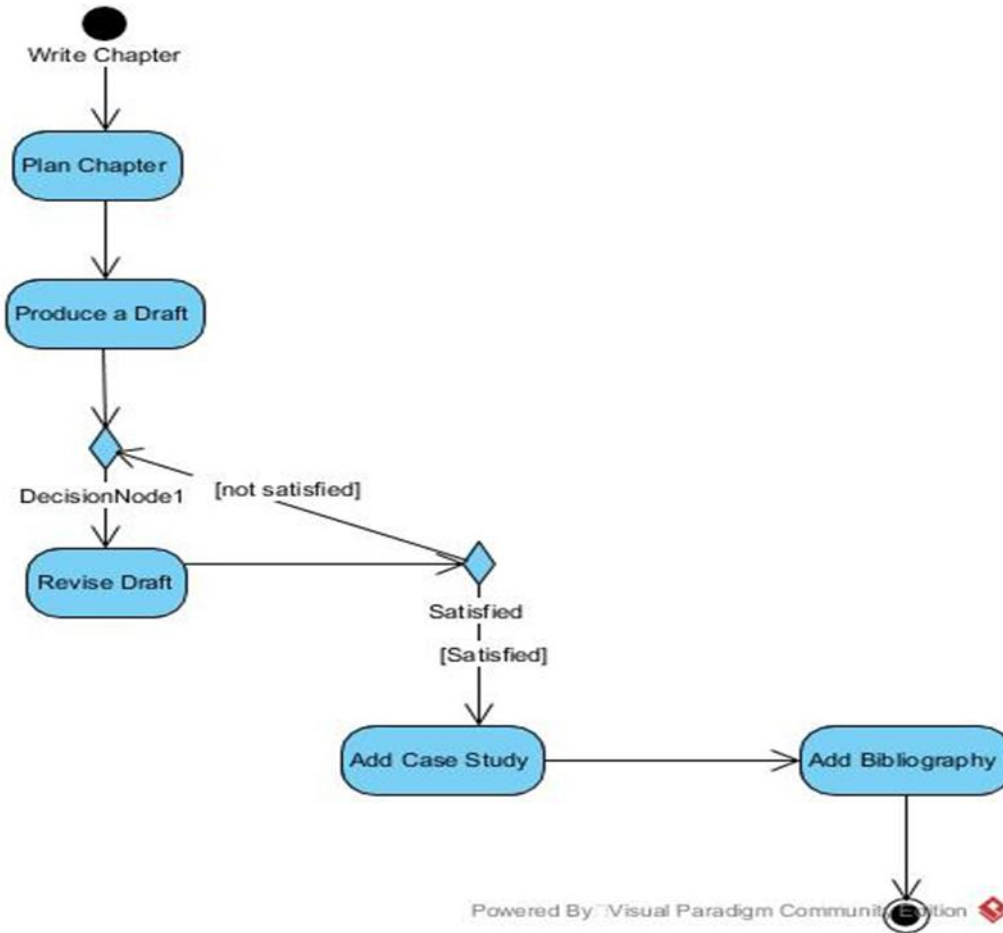
Modelling using UML

- Use case diagram (the campaign manager assigns staff to work on campaign)



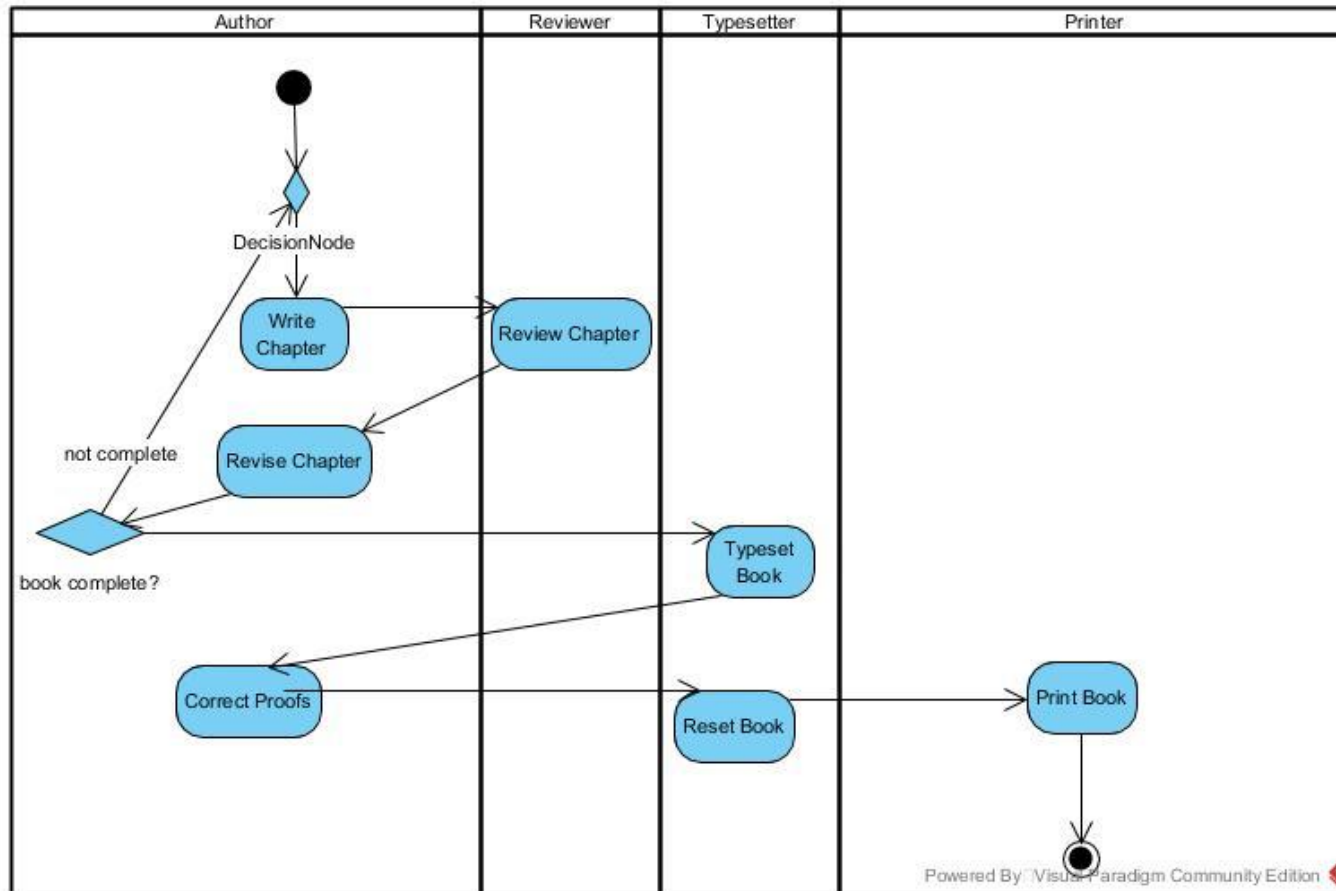
Modelling using UML

- Activity diagram (write a chapter)



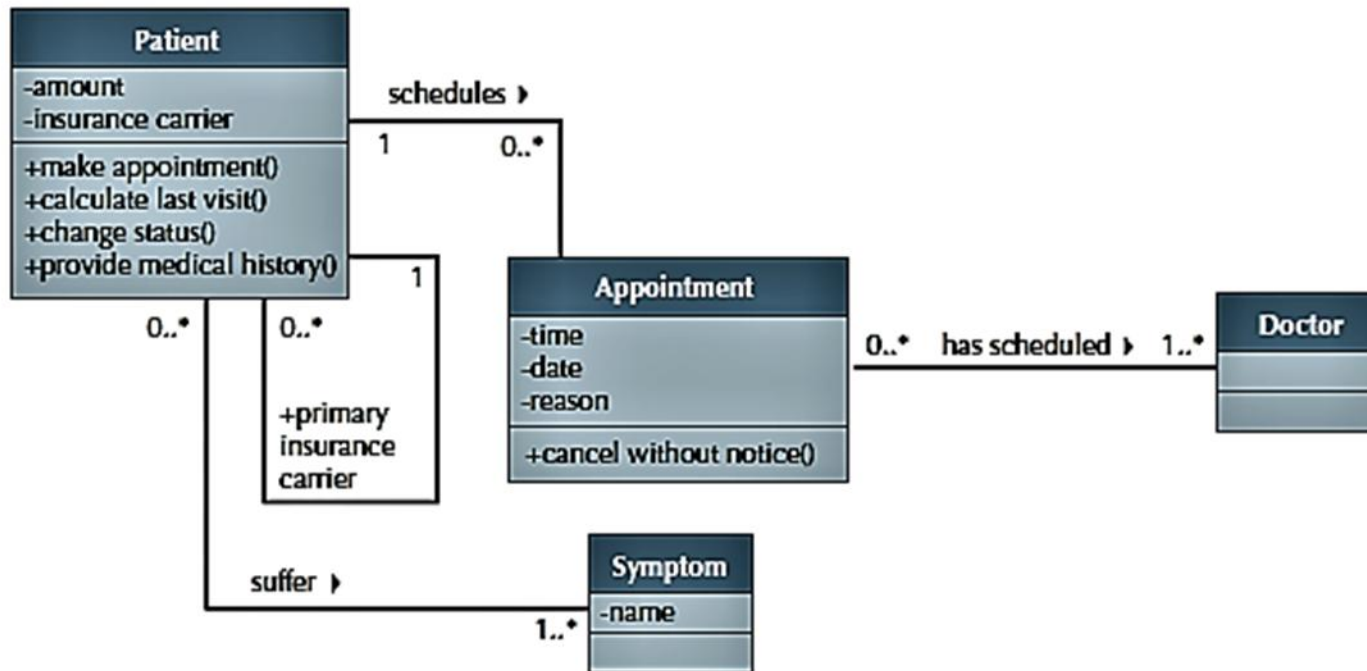
Modelling using UML

- Activity diagram (make a book)



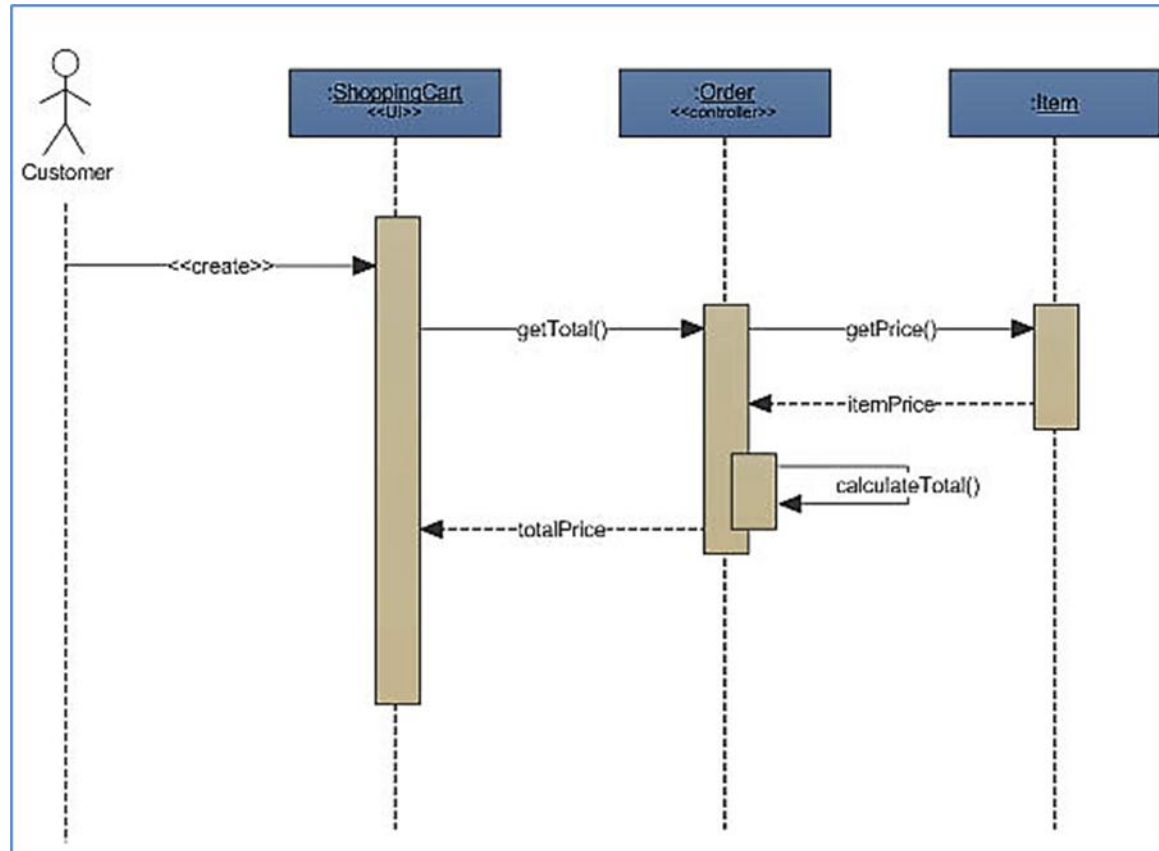
Modelling using UML

- Class diagram (schedule medical appointment)



Modelling using UML

- Sequence diagram (shopping cart)



UML Architecture

- UML building blocks
- Rules to connect the building blocks
- Common mechanisms of UML
- The building blocks of UML :
 - Things
 - Relationships
 - Diagrams

UML Architecture - Things

- Things are the elements of UML.
 - Structural
 - Behavioral
 - Grouping
 - Annotational
 - Diagrams

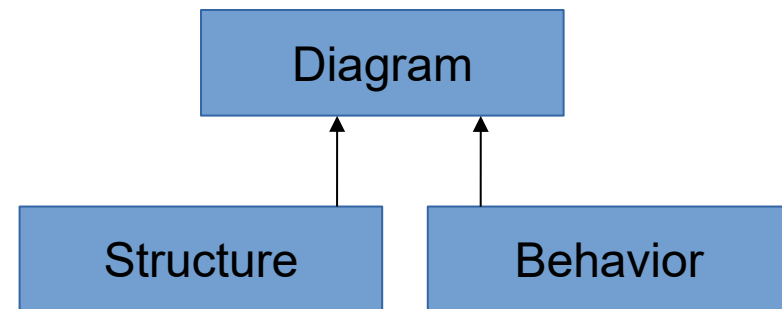
UML Architecture - Relationships

- Shows how the elements are associated with each other
- The association describes the functionality of an application
 - Dependency
 - Association
 - Generalization
 - Realization

UML Diagrams

UML includes

- Structural diagrams
 - represent the static aspect of the system;
 - static parts of the main structure of a diagram
 - static parts are represented by classes, interfaces, objects, components, and nodes
- Behavioral diagrams
 - capture the dynamic aspect of a system
 - changing/moving parts of a system



UML Structural Diagrams

Class diagram

Class, features, and relationships

Object diagram

Example configurations of instances

Package diagram

Compile-time hierarchic structure

Component diagram

Structure and connections of components

Composite structure diagram

Runtime decomposition of a class

Deployment diagram

Deployment of artifacts to nodes

UML Behavioral Diagrams

Activity diagram	Procedural and parallel behavior
Use case diagram	How users interact with a system
State machine diagram	How events change an object over its life
Sequence diagram	Behavior of objects within a use case; emphasis on the time-based ordering of an activity
Communication diagram	Interaction between objects; emphasis on links
Interaction overview diagram	Mix of sequence and activity diagram
Timing diagram	Interaction between objects; emphasis on timing

Requirements Gathering

- Functional
 - What system does or expected to do.
 - Use cases are utilized to document the functionality of the system
- Non-functional
 - How well the system provides the functional requirements.
 - How the system performs or operates
 - E.g. performance, security, etc...
- Usability
 - How well the system allows the user to compete tasks with effectiveness, efficiency, and satisfaction.

Requirements Gathering - Exercise

- Scenario: You have been hired to design a Library Management System for a university. The system should allow students and staff to borrow and return books, search for books in the library's catalog, and receive notifications about overdue items. The university also wants the system to support mobile devices, be secure, and handle up to 1,000 users at the same time
- Identify a few functional requirements for the system and identify a few non-functional requirements for the system

Requirements Gathering – Exercise Solution

- Functional Requirements
 - The system must allow users to search for books by title, author, or ISBN
 - Users must be able to borrow and return books using their university ID
 - The system should notify users of overdue books via email
- Non-Functional Requirements
 - The system must support up to 1,000 concurrent users
 - It must be accessible on mobile devices, including smartphones and tablets
 - Responses to user actions (e.g., search queries) must occur within 2 seconds

Requirements Gathering

- Organizational (business) rules, company reports, charts, policy manuals
- Documentation of existing systems
- Interview sessions with users
- Interviews with stakeholders
- Observations
- Document sampling
- Surveys

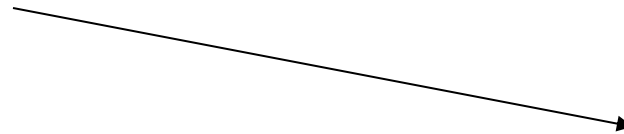
Sampling
Questionnaires
Interview
Reading
Observations

Use Cases

- Use Case represents the functionality of the system.
- Project team gathers requirements from the users.
- Using the gathered requirements, the project team then identifies the business processes and their environment using use cases and use-case diagrams
- A scenario is a sequence of steps describing an interaction between a user and a system.
- A use case is a set of scenarios tied together by a common user goal.
- Actors carry out use cases.

Use Cases

- Requirements capture -> Use Cases
- Use cases are descriptions of functionality of the system from the user's perspective.
- Purpose: document the scope of the systems (users requirements)
- Use cases are supported by behavior specs:
 - use case descriptions, activity diagrams, sequence diagrams, communication diagrams
- Records of interviews and observations
 - Detail of problems
 - Details of requirements
 - Details of users



Use Case
model

Use Cases

	Requirements	Use Cases
1	To maintain staff records of creative and administrative staff	Add new member of staff
2	To maintain details of staff grades and the pay for the grades	Add new staff grade Change the rate for a staff grade
3	To record names, address, and contact details for each client	Add new client
4	To record details of each campaign (title, estimated start and finish dates, estimated cost, budget, actual cost and dates, current state)	Add new campaign
	...	

Creating Use Cases

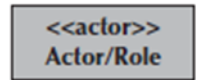
- Choose a **use case**.
- Create **overview description** (name the primary actor, type of the use case, identify stakeholders, level of importance, brief description, triggers, relationships).
- Describe the **normal flow** of events.
- Identify **alternative** or exceptional **flows**.
- Review (simplify by decomposing, add the include, extend, generalization relationships).

Creating Use Cases

- **Actor** is external entity which is involved in the interaction with the system in the use case
- Stakeholders are entities that have an interest in the use case
- **Trigger** is the event that causes the use case to begin
- **include** and **extend** stereotypes
- **Stereotype** is a special use of a model element constrained to behave in a certain way

Use Case Diagram Elements

- **Actor** a person or system that derives benefit from the system
 - is external to the subject.
 - is labelled with its role.
 - can be associated with other actors using a specialization/generalization
 - is placed outside the subject boundary.
- **Use case** represents a major piece of system functionality.
 - can extend another use case.
 - can include another use case.
 - is placed inside the system boundary.
 - is labelled with a descriptive verb–noun phrase.
- **Subject boundary** includes the name of the subject
 - represents the scope of the subject (system or an individual business process)

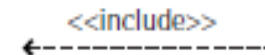


Use Case Diagram Elements

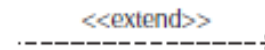
- **Association relationship** links an actor with the use cases with which it interacts.



- **Include relationship** represents the inclusion of the functionality of one use case within another. An arrow drawn **from the base use case to the used use case**.



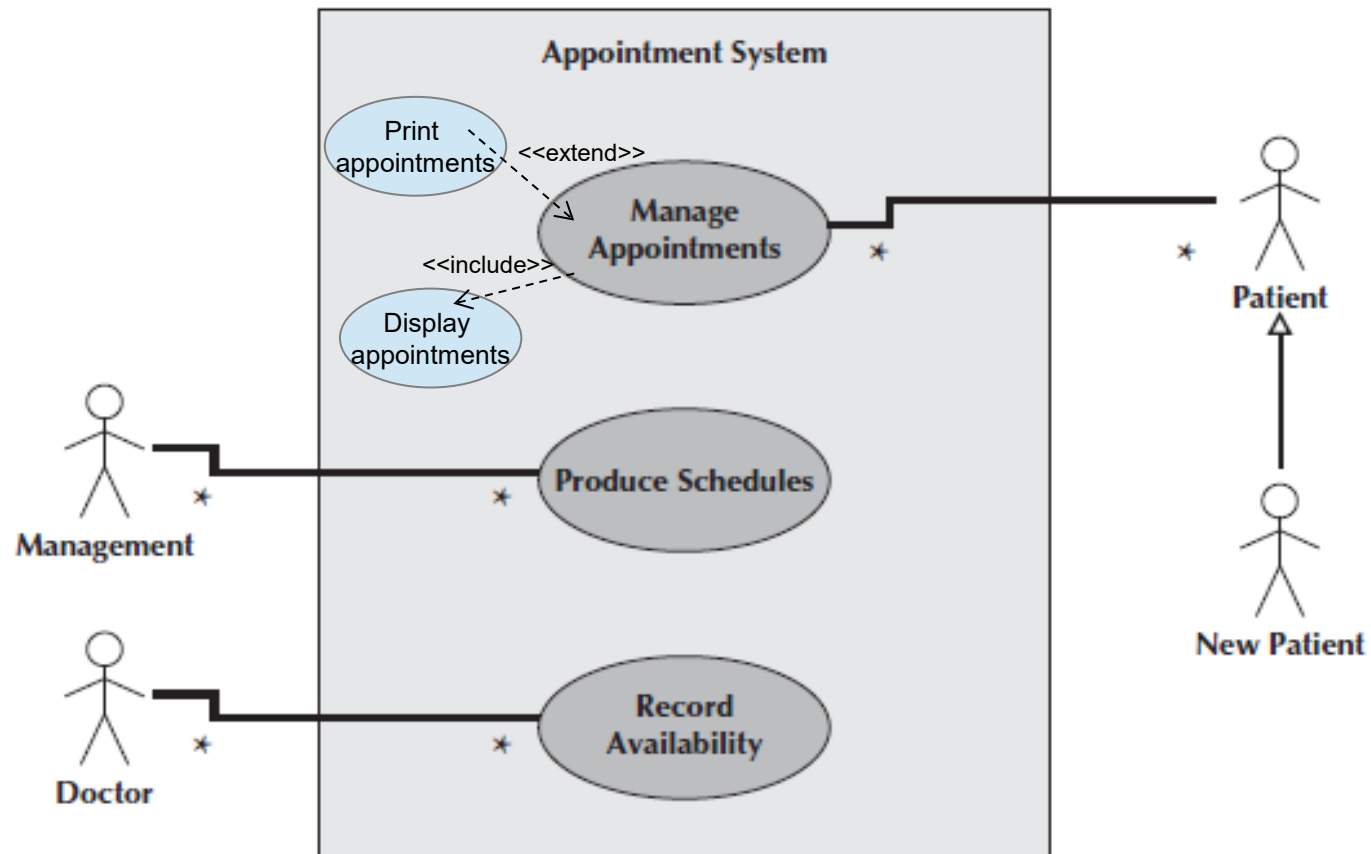
- **Extend relationship** represents the extension of the use case to include optional behavior. An arrow drawn **from the extension use case to the base use case**.



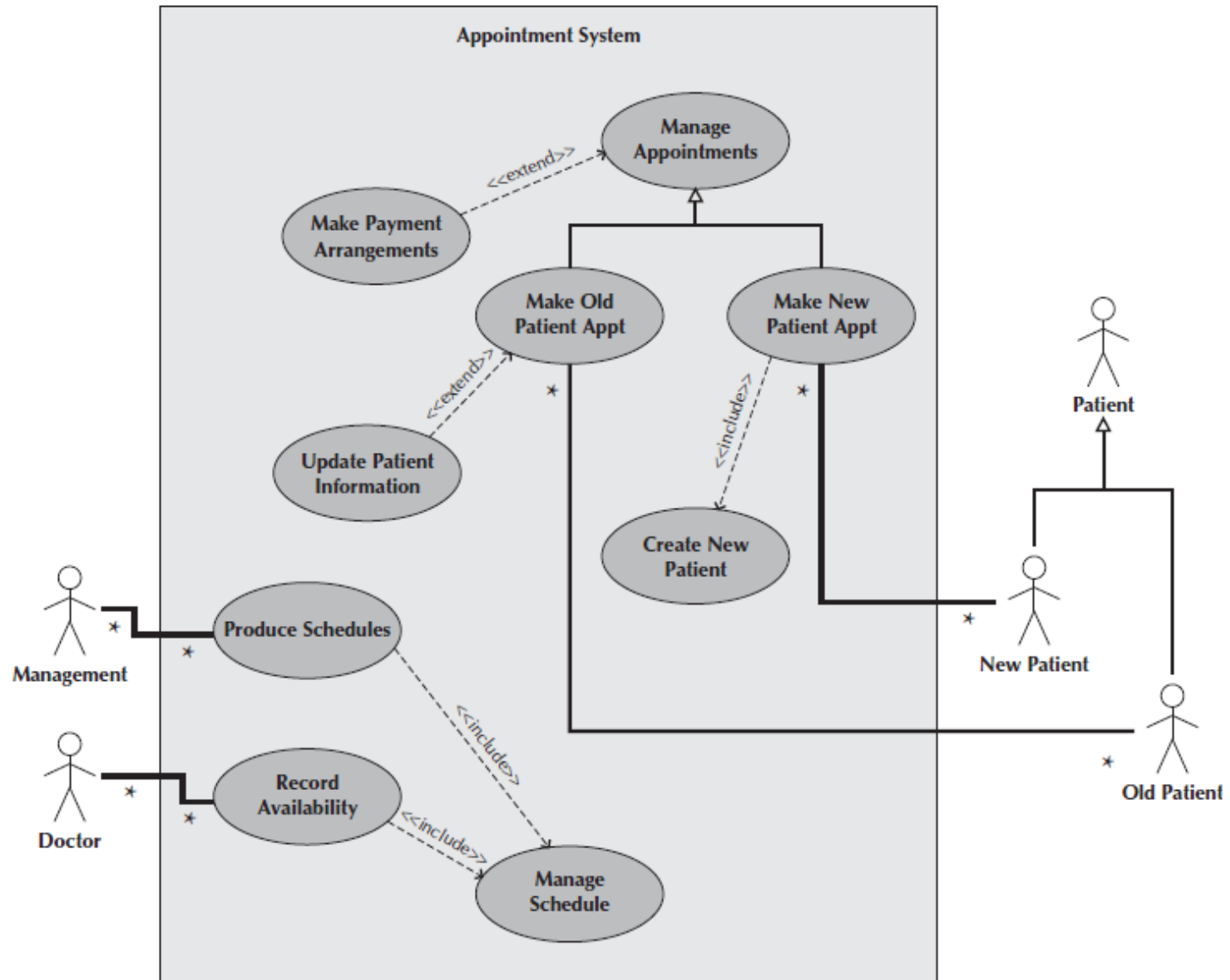
- **Generalization relationship** represents a specialized use case to a more generalized one. An arrow drawn from the specialized use case to the base use case.



Use Case Diagram Example

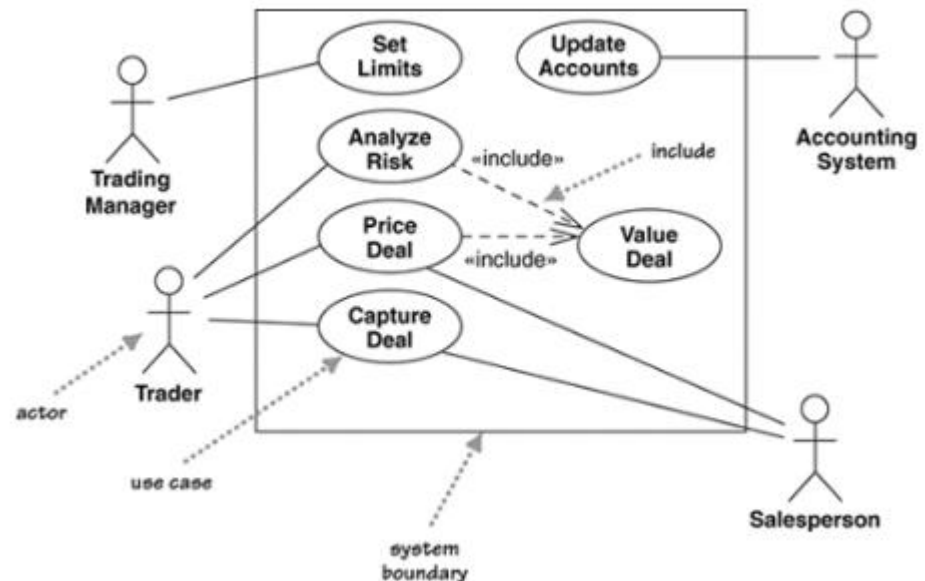


Use Case Diagram Example



Use Case Diagrams

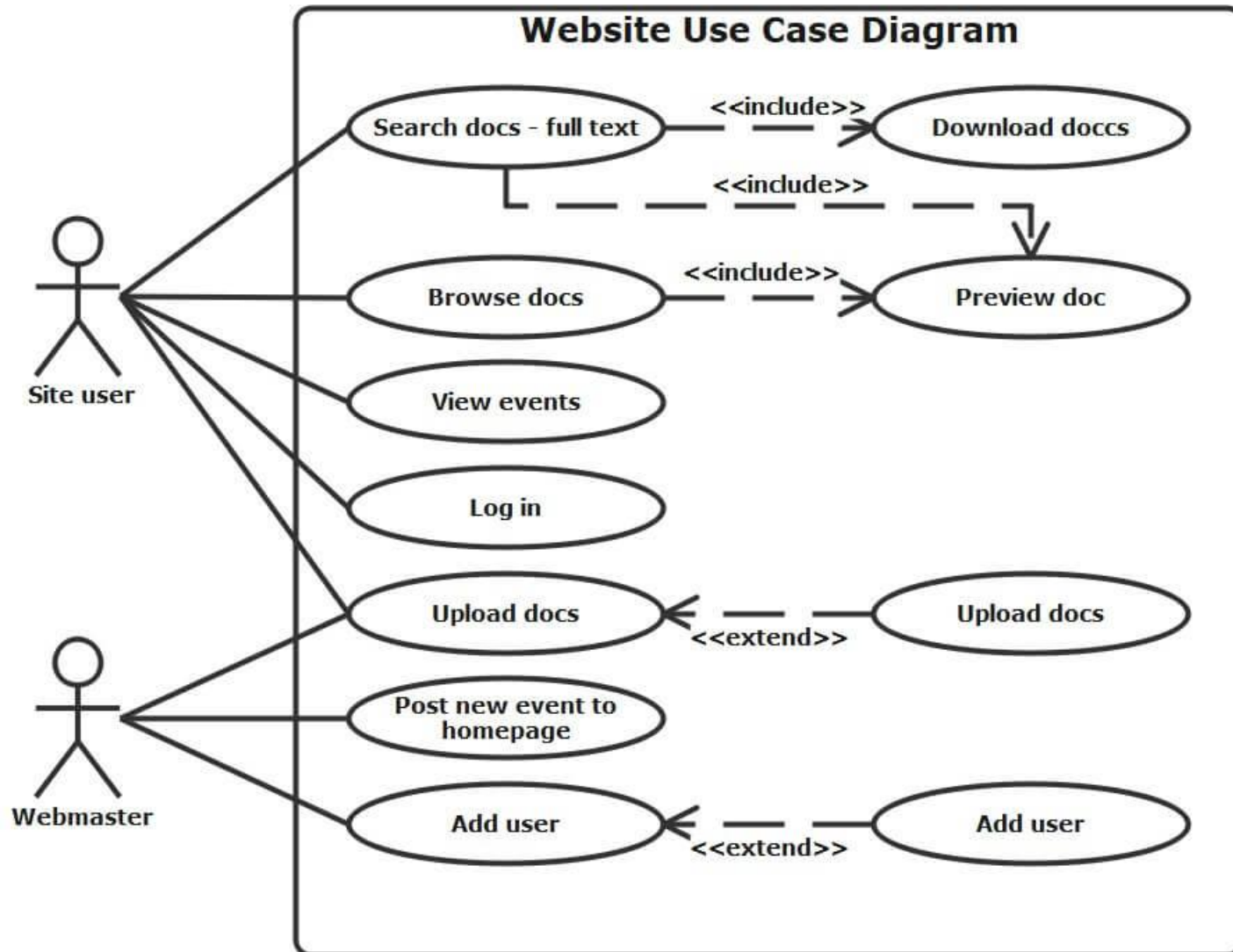
- **Use cases** are a valuable tool to help understand the **functional requirements** of a system.
- Start utilizing use cases early on in the development process.
- Use cases represent an external view of the system.



Use Case Diagrams - Exercise

- With a use case diagram, model the following situation
- Two users (site user and admin)
- The site user should be able to search and browse docs
 - Also have the option to preview and download the doc after searching and browsing for it
- The site user should be able to view events, log in and upload docs
 - The upload doc should have an extension to upload a doc
- The admin should be able to upload docs, post new event, and add new user
 - The add user should have an extension to add a user

Use Case Diagrams – Exercise Solution



Upcoming

- Lab Assignment 3 – XP Spike 3 Due Friday
- More design review (activity, class, sequence, ...)

References

- Object-Oriented Systems Analysis and Design Using UML by Simon Bennett, Ray Farmer (2010) ISBN: 978-0077125363
- Dennis, Alan, et al. System Analysis and Design. Wiley, 2013.
- Object-Oriented Analysis and Design with Applications. Pearson Education (US), 2011.
- "Object Oriented Analysis and Design Tutorial." Www.tutorialspoint.com, Tutorials Point, www.tutorialspoint.com/object_oriented_analysis_design/.
- Omeecode. "Object Oriented Analysis and Design." CodeProject - For Those Who Code, CodeProject, www.codeproject.com/Articles/1137299/Object-Oriented-Analysis-and-Design.
- "Reasons for Software Project Failure Globally 2015 | Statistic." Statista, www.statista.com/statistics/627648/worldwide-software-developer-survey-project-failure/.

COSC 205

236 Review

Design Overview



UML Structural Diagrams

Class diagram

Class, features, and relationships

Object diagram

Example configurations of instances

UML Behavioral Diagrams

Activity diagram

Procedural and parallel behavior

Use case diagram

How users interact with a system

Sequence diagram

Behavior of objects within a use case;
emphasis on the time-based ordering of an activity

Activity Diagram

- Activity diagrams are a technique to describe procedural logic, business process, and work flow.
- The flow of events of a use case describes what needs to be done by the system to provide value to an actor.
- It consists of a sequence of activities that together produce a goal for the actor.
- The flow of events consists of a basic flow, and one or several alternative flows.

- Activity diagrams are used to
 - model a process or task
 - to describe a system function represented in the use case
 - to describe the logic of operation (operation specifications)
 - to model the activities in a lifecycle (USDP)

Activity Diagram

- Activity diagrams portray the primary activities and the relationships among the activities in a process.
- Activity diagram is a set of actions linked together by flows
- Actions and Activities are performed for some specific business reason.

An action is a simple, non-decomposable piece of behavior

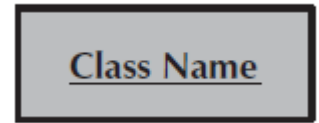
Action

An activity is used to represent a set of actions

Activity

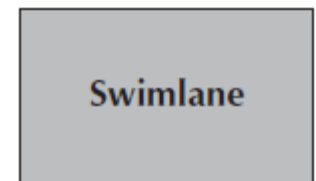
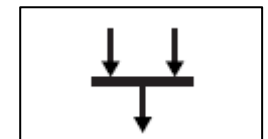
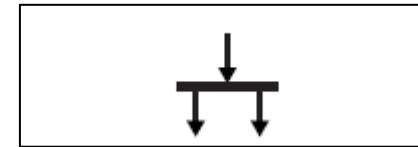
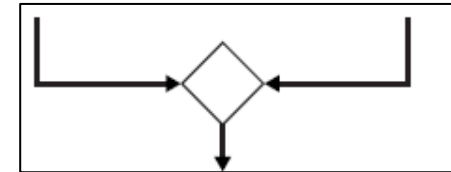
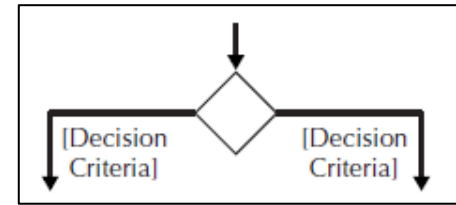
Activity Diagram

- **Object node** is used to represent an object that is connected to a set of object flows.
- **Control flow** shows the sequence of execution
- **Initial node** shows the beginning of a set of actions or activities
- **Final-activity node** is used to stop all control flows and object flows in an activity.
- **Final-flow node** is used to stop a specific control flow or object flow

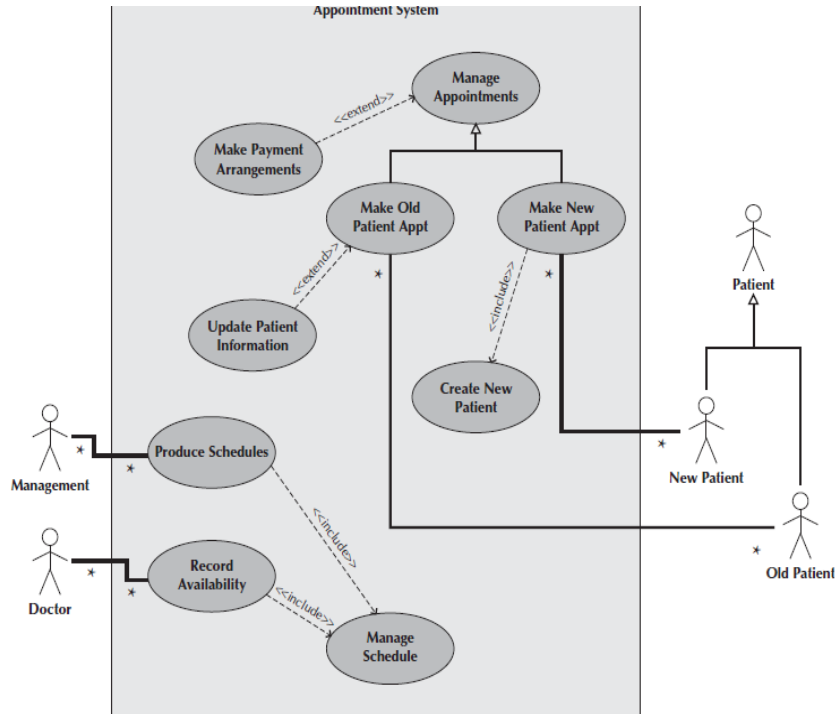


Activity Diagram

- **Decision node** is used to represent a test condition to ensure that the control flow or object flow only goes down one path.
- **Merge node** is used to bring back together different decision paths that were created using a decision node.
- **Fork node** is used to split behavior into a set of parallel or concurrent flows of activities
- **Join node** is used to bring back together a set of parallel or concurrent flows of activities
- **Swimlane** is used to break up an activity diagram into rows and columns to assign the activities to the individuals or objects that are responsible for executing the activity

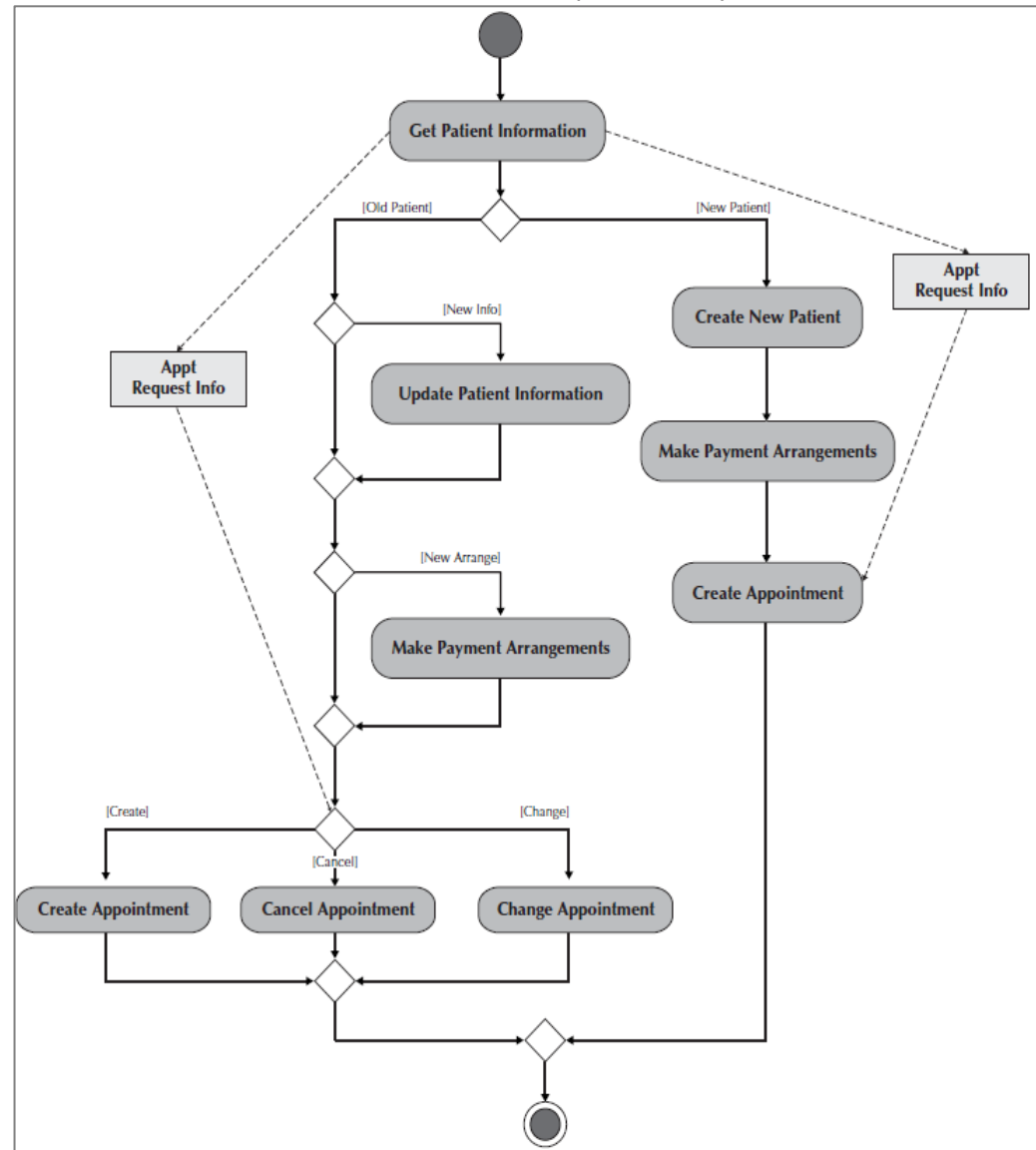


Activity Diagram



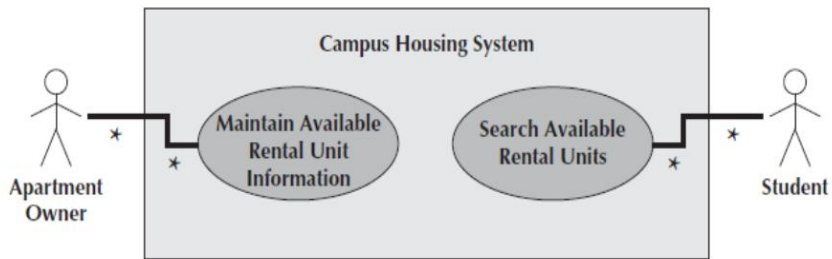
Use case diagram.

Get Patient Information (Use Case)



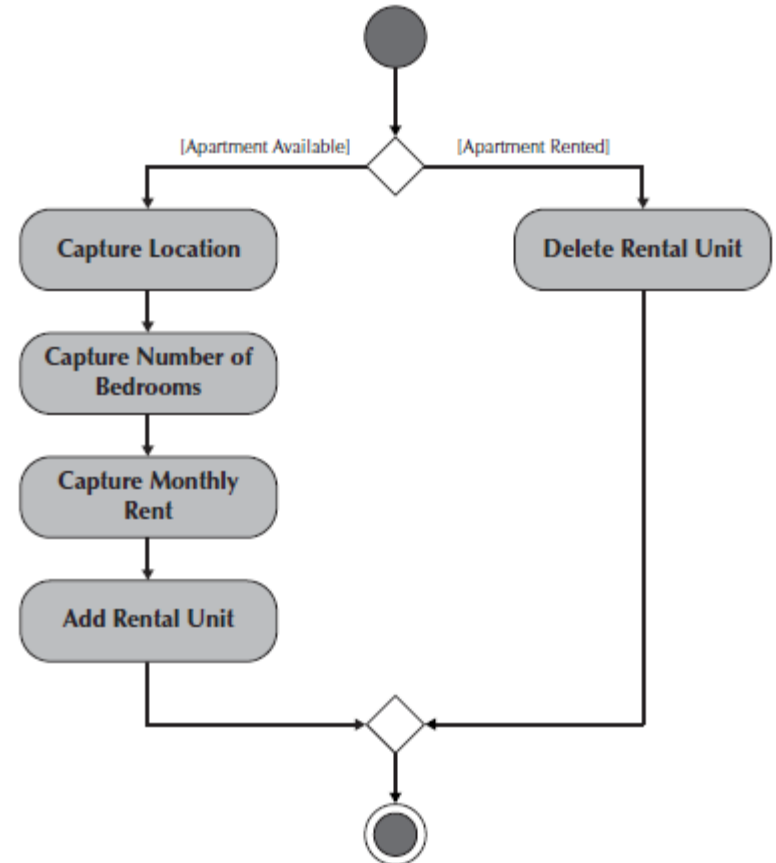
Activity diagram.

Activity Diagram



Use case diagram.

Maintain Available Rental Unit Information



Activity diagram.

Activity Diagram

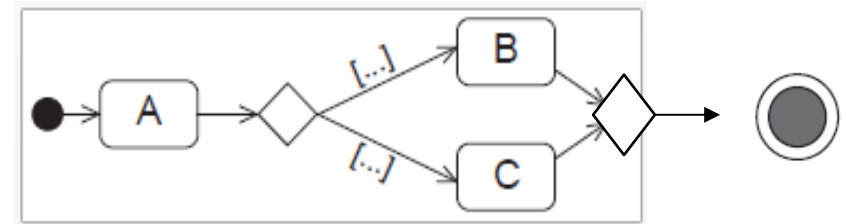
Activity diagram can be used to model any kind of process

- set the context or scope of the activity to be modeled
- give the diagram an appropriate title
- identify the activities, control flows, and object flows that occur between the activities
- identify any decisions that are part of the process being modeled
- identify any prospects for parallelism in the process
- draw the activity diagram

Activity Diagram - Exercise

You are given the following activity diagram. Which of the following action sequences are possible during one execution of the activity diagram?

1. A -> B -> C
2. A -> B
3. A -> C
4. A -> C -> B

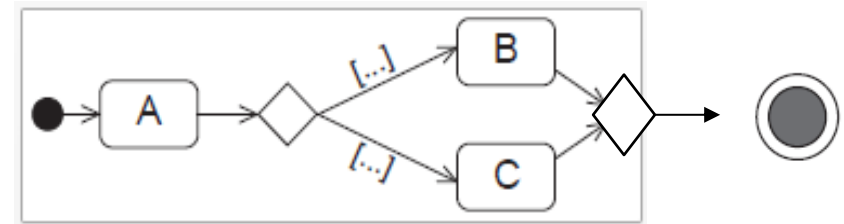


Each token arriving at a decision node can traverse only one outgoing edge.

Activity Diagram - Solution

You are given the following activity diagram. Which of the following action sequences are possible during one execution of the activity diagram?

1. A -> B -> C
2. A -> B
3. A -> C
4. A -> C -> B

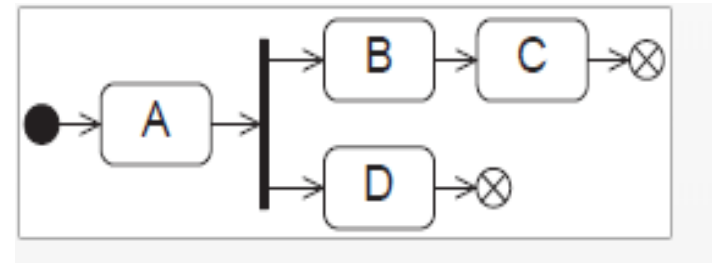


Each token arriving at a decision node can traverse only one outgoing edge.

Activity Diagram - Exercise

You are given the following activity diagram.

Which of the following action sequences are possible during one execution of the activity diagram?



1. A -> B -> C -> D
2. A -> D
3. A -> D -> C
4. A -> D -> B -> C
5. A -> B -> D -> C
6. A -> B -> C

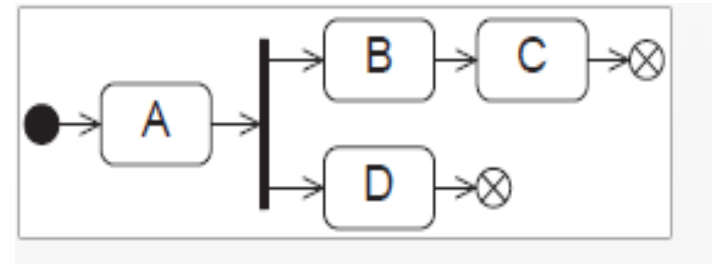
A flow final node  swallows the token that hits it, while all other tokens in the activity continue rolling. This means that only this one flow is terminated.

All behaviors stop when the first token reaches an Activity Final Node. 

Activity Diagram - Solution

You are given the following activity diagram.

Which of the following action sequences are possible during one execution of the activity diagram?

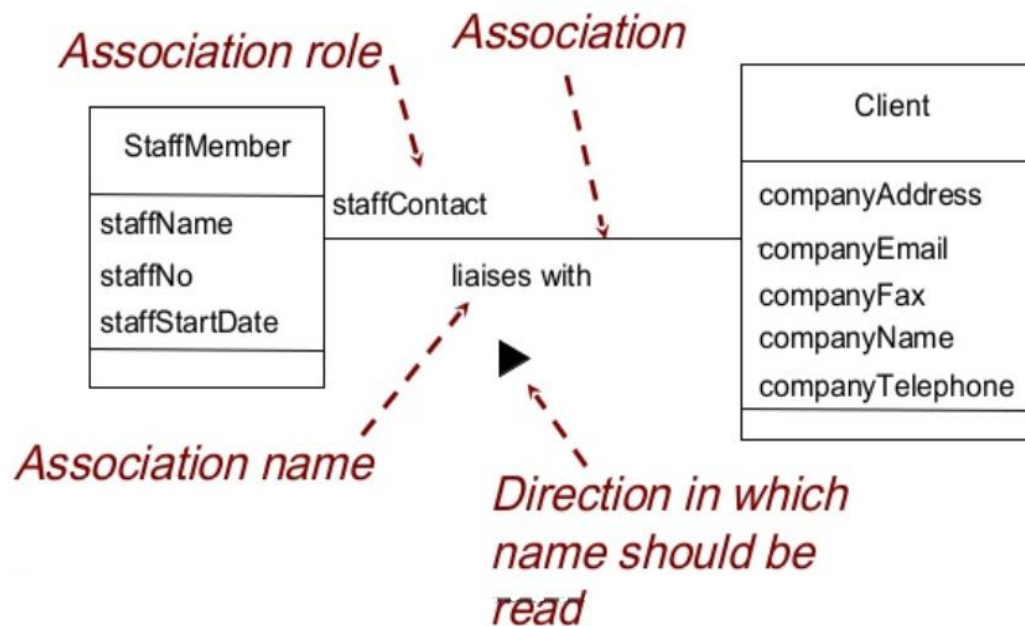
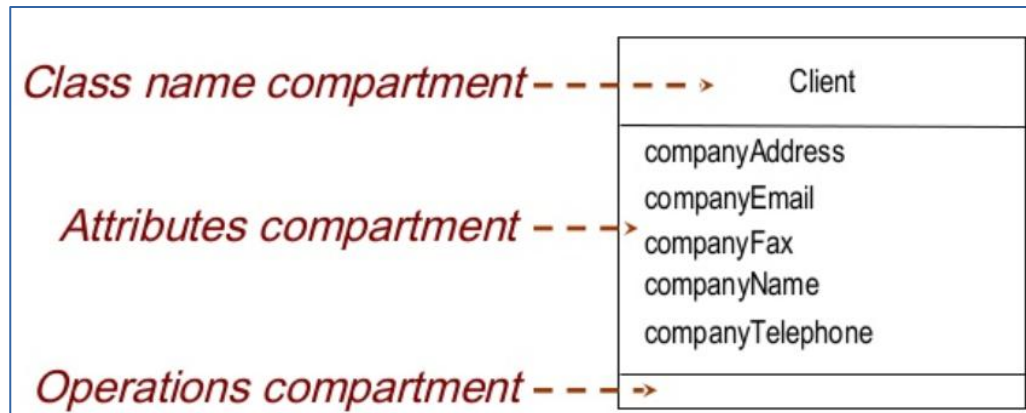


1. A -> B -> C -> D
2. A -> D
3. A -> D -> C
4. A -> D -> B -> C
5. A -> B -> D -> C
6. A -> B -> C

A flow final node  swallows the token that hits it, while all other tokens in the activity continue rolling. This means that only this one flow is terminated.

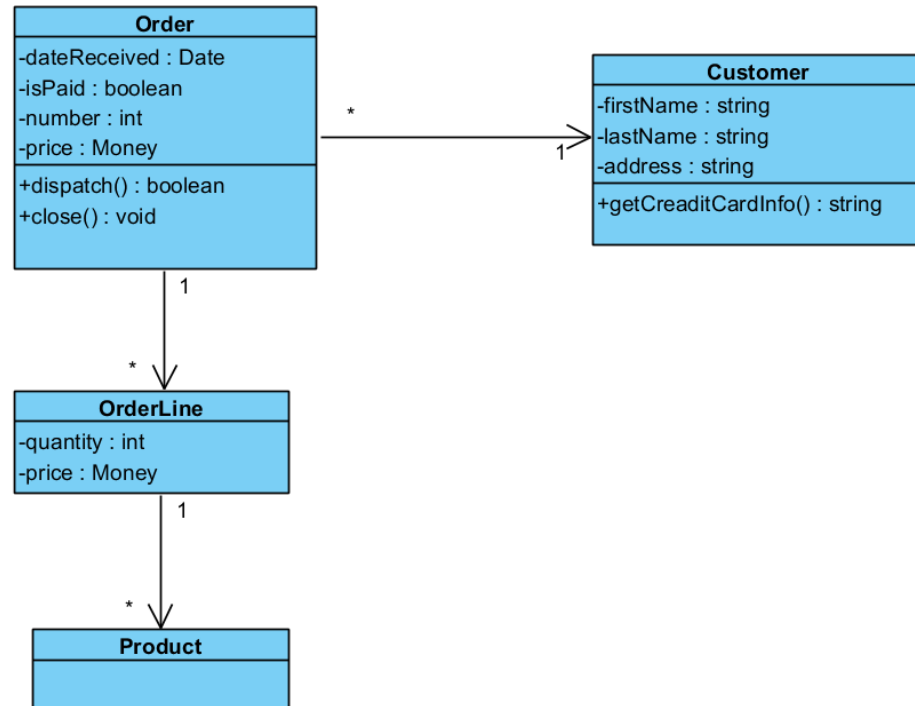
All behaviors stop when the first token reaches an Activity Final Node. 

Class Diagram



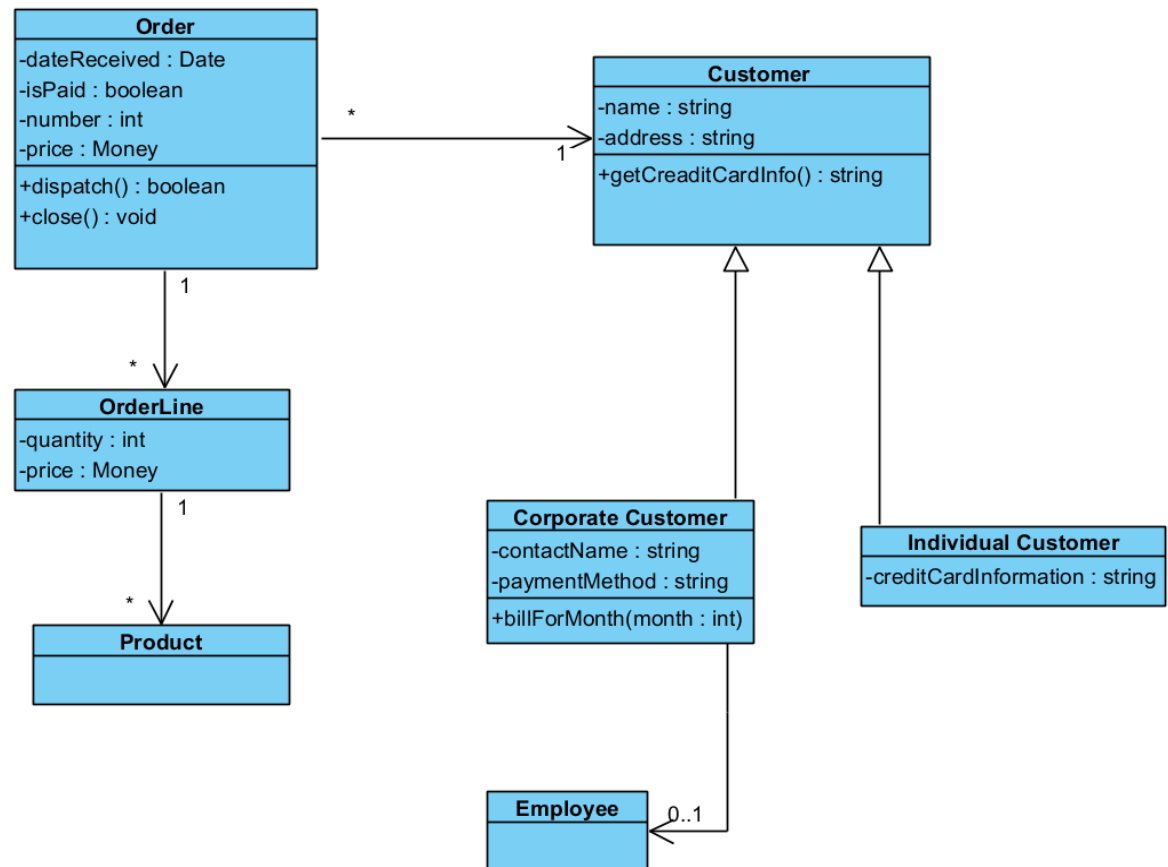
Class Diagram

- Class diagram describes:
 - the types of objects in the system
 - static relationships among types of objects



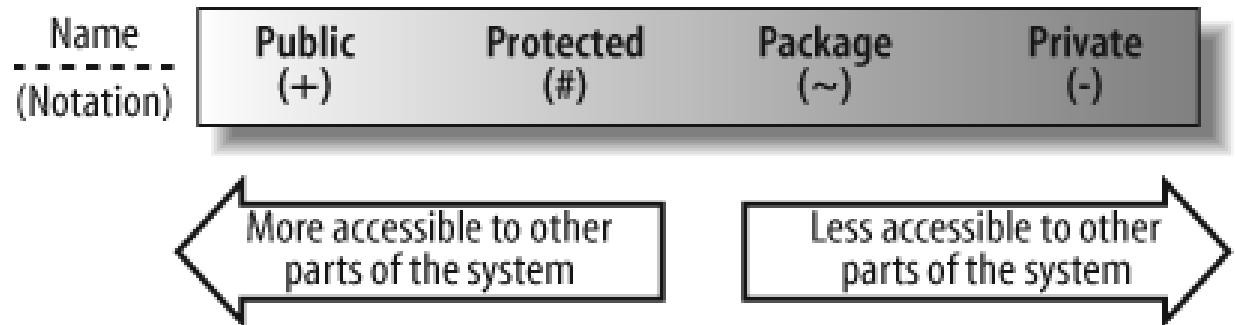
Class Diagram

- Also describes:
 - show the properties of a class
 - show operations of a class
 - associations
 - generalizations



Class Diagram - Visibility

- How does a class selectively reveal its operations and data to other classes?
- Controls access to attributes, operations, and entire classes.
- UML has the following types of visibility:
 - public
 - protected
 - package
 - private



Class Diagram - Visibility

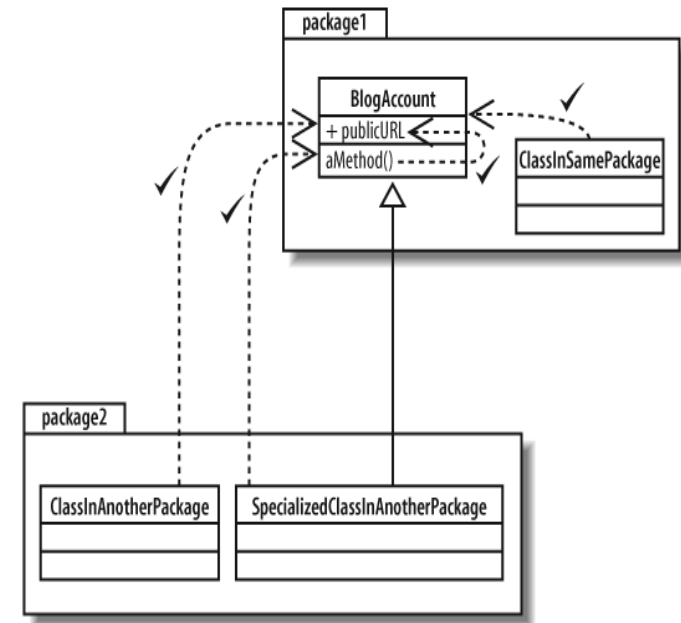
Public visibility

+ symbol before the associated attribute or operation

public attributes and operations can be accessed by other classes.

It is the **part** of your class that other classes will **depend on the most**.

public attributes and operations should be **changed as little** as possible to prevent unnecessary changes wherever your class is used.

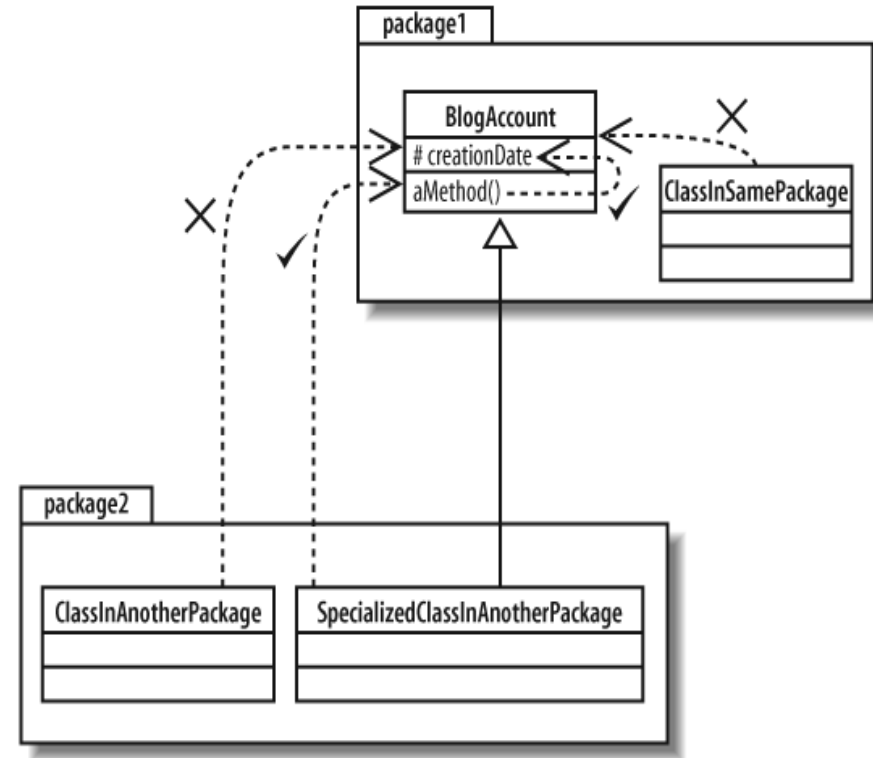


Class Diagram - Visibility

Protected Visibility

symbol before the associated attribute or operation.

Protected elements can be accessed by methods that are part of your class and by methods that are declared on any class that inherits from your class.



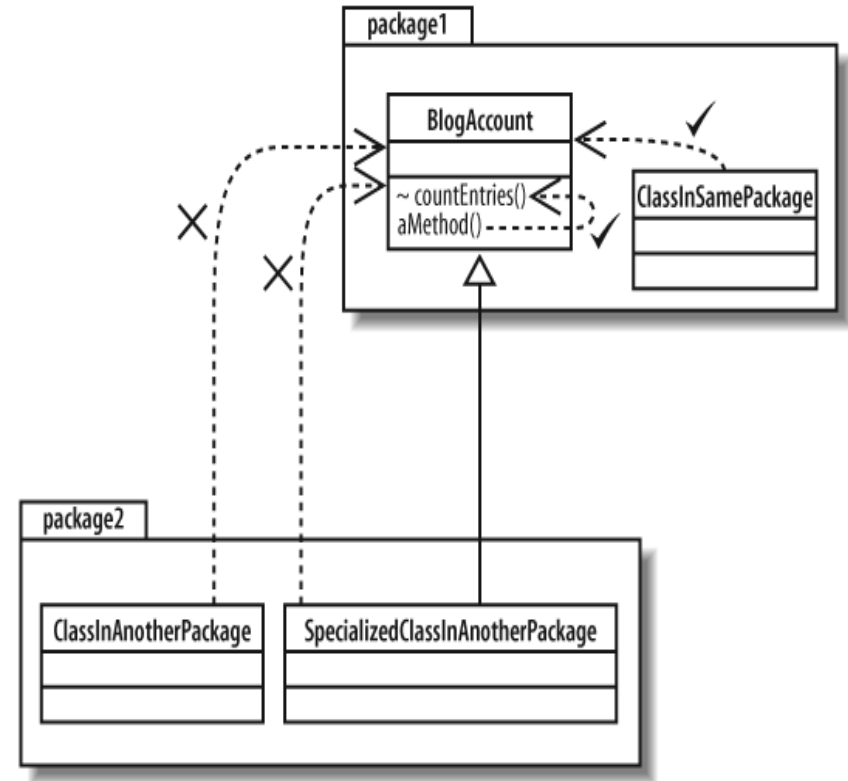
Class Diagram - Visibility

Package Visibility

~ symbol before the associated attribute or operation.

Any class in the **same package** can directly access the **attribute** or **operation**.

Classes **outside** the package **cannot access** protected attributes or operations even if it's an inheriting class.



Class Diagram - Visibility

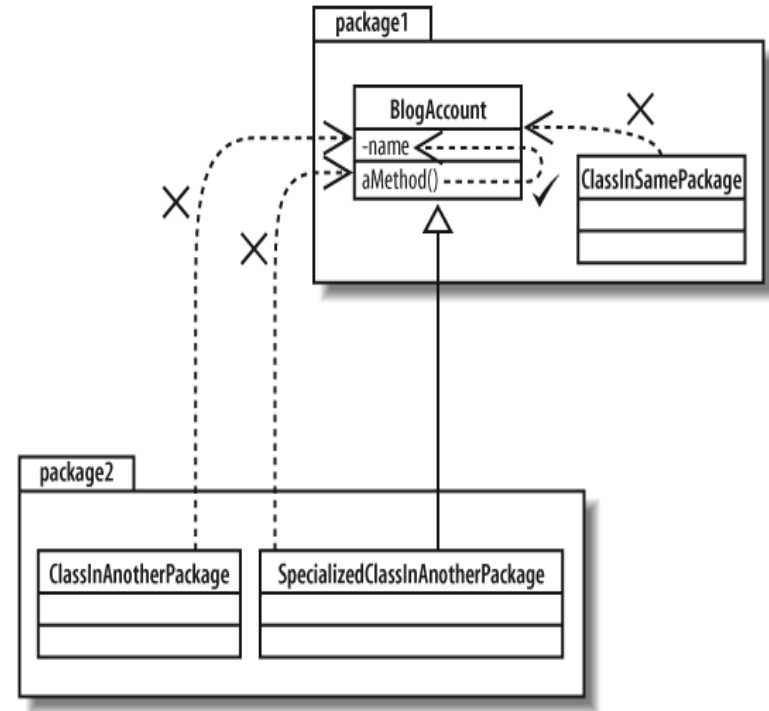
Private Visibility

- symbol before the attribute or operation.

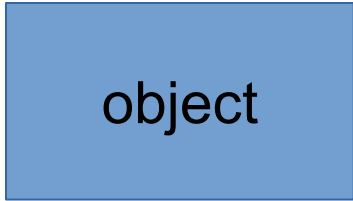
Only the class that contains the private element can work with the data stored in a private attribute or call private operation.

Private visibility is most useful if you have an attribute or operation that you want no other part of the system to depend on.

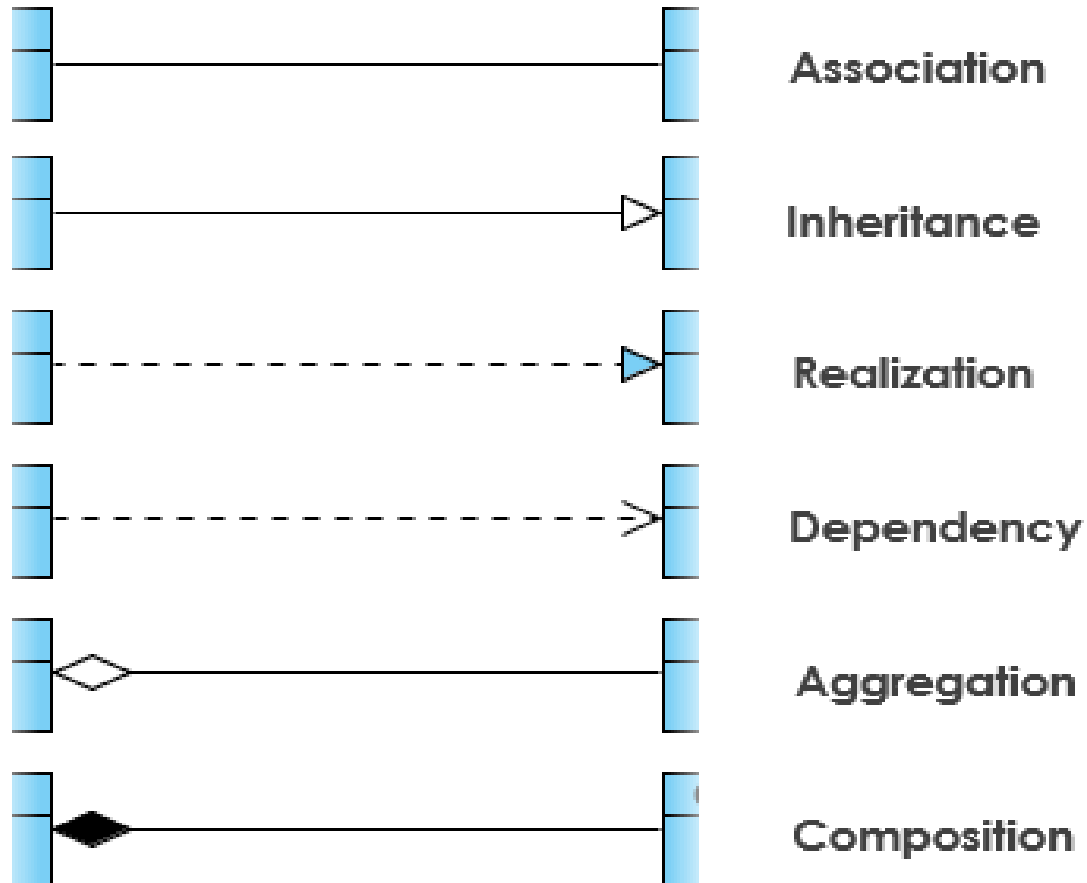
attributes should always be private or **protected** if need to share the class' attribute with classes that inherit from your class



Class Diagram - Operations

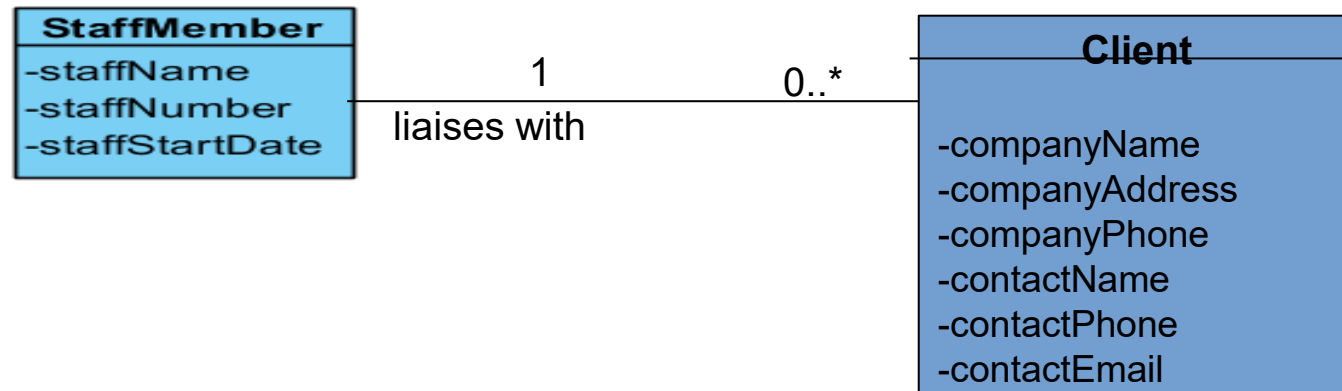
- Operations are the elements of common behavior shared by all instances of a class.
- Carried out by 
- Carried on
- Operations represent services that objects of a class can provide to other objects.
- Operations describe what instances of a class can do:
 - Set or get attributes
 - Perform calculations
 - Send messages to other objects
 - Create or destroy links

Class Diagram – Relationships between classes



Class Diagram - Associations

- **Association** is an abstraction that connects two classes, and represents links between their objects
- Association shows as a solid line between two classes.
- Directed from the source class to the target class.
- Name of the property and multiplicity goes at the target end of the association
- The target end of the association links to the class that is the type of the property.



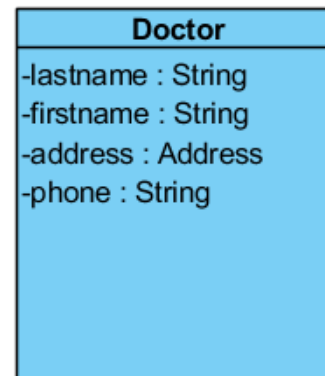
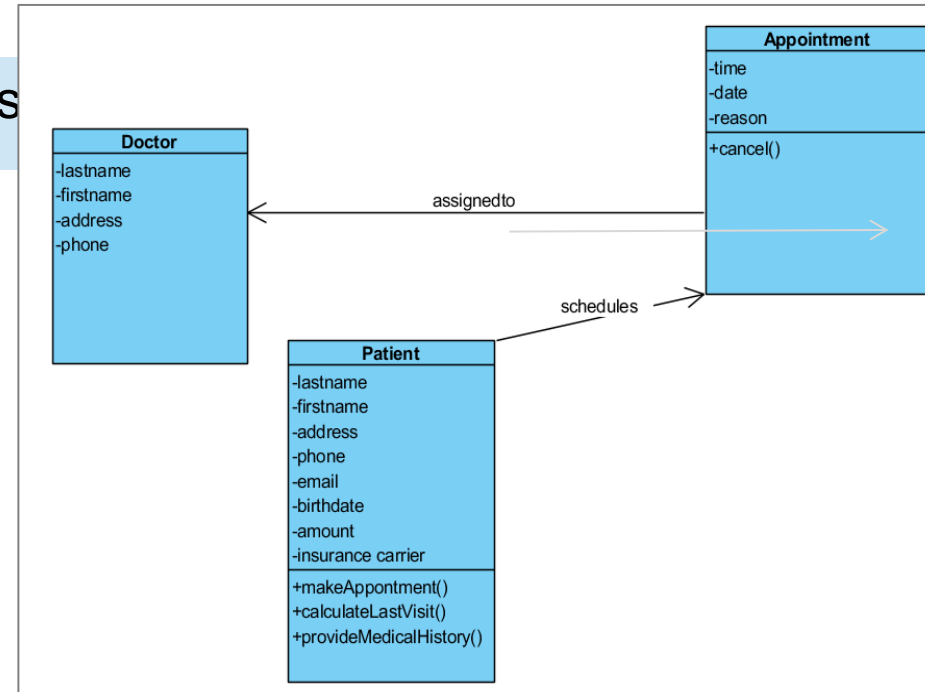
- **Association** describes the relationship between two or more components.
- **Multiplicity** describes the number of elements possible in a relationship between those components.
- **Navigability** describes the directional flow of data in the association.

Class Diagram

Programming interpretation of attributes

```
package booking_appointment;
```

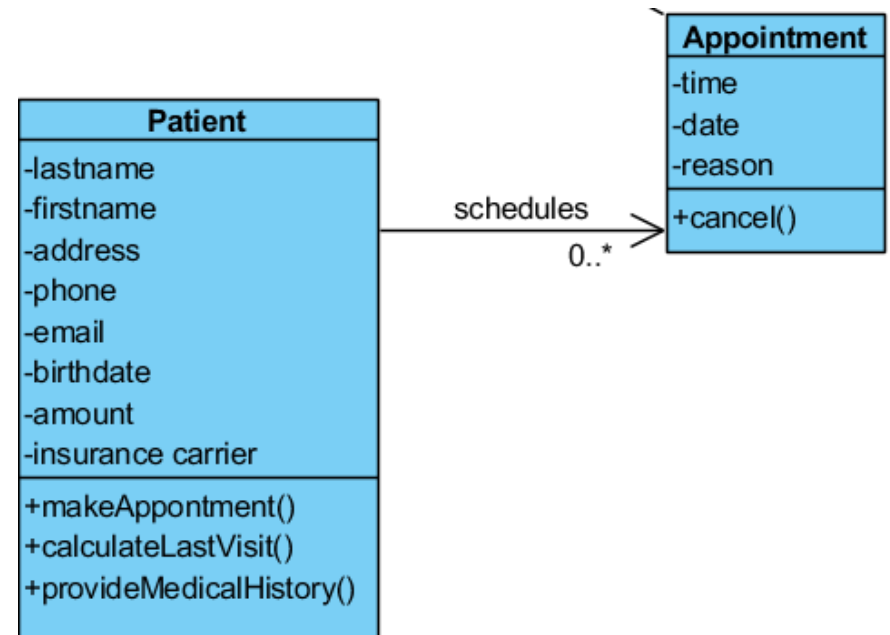
```
public class Doctor {  
    private String lastname;  
    private String firstname;  
    private Address address;  
    private String phone;  
    private Appointment assignedto;  
}
```



Class Diagram – Uni-directional Associations

Uni-directional association

- shows that the source object can invoke methods of the destination class.
- The instance variable of source class referencing the destination class.



```
public class Patient {  
    public ArrayList<Appointment> schedules = new ArrayList<>();  
  
}
```

Class Diagram – Bi-directional Associations

In **bi-directional associations**

each of the classes in the relationship refers to each other by calling each others method.



```
public class Customer {  
    private String name;  
    private String address;  
    private String contactNumber;  
    private Car car;  
}
```

```
public class Car {  
    private String modelNumber;  
    private Customer owner;  
}
```

Class Diagram - Multiplicities

- Multiplicity (cardinality, i.e. number of elements) specifies the number of objects that can participate in the association.

Multiplicity	Option	Cardinality
0..0	0	Collection must be empty
0..1		No instances or one instance
1..1	1	Exactly one instance
0..*	*	Zero or more instances
1..*		At least one instance
5..5	5	Exactly 5 instances
m..n		At least m but no more than n instances

Class Diagram - Generalizations

Generalization property represents the **inheritance** feature of the object oriented concept.

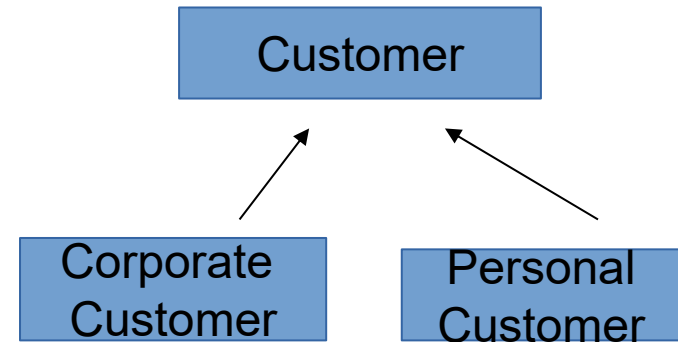
Customer class is a **superclass**,
Personal Customer and Corporate
Customer are subclasses.

Similarities (attributes, operations) can be
placed in the supertype Customer class.

Corporate Customer is a subtype of Customer if all instances of Corporate
Customer are also, by definition, instances of Customer.

Inheritance

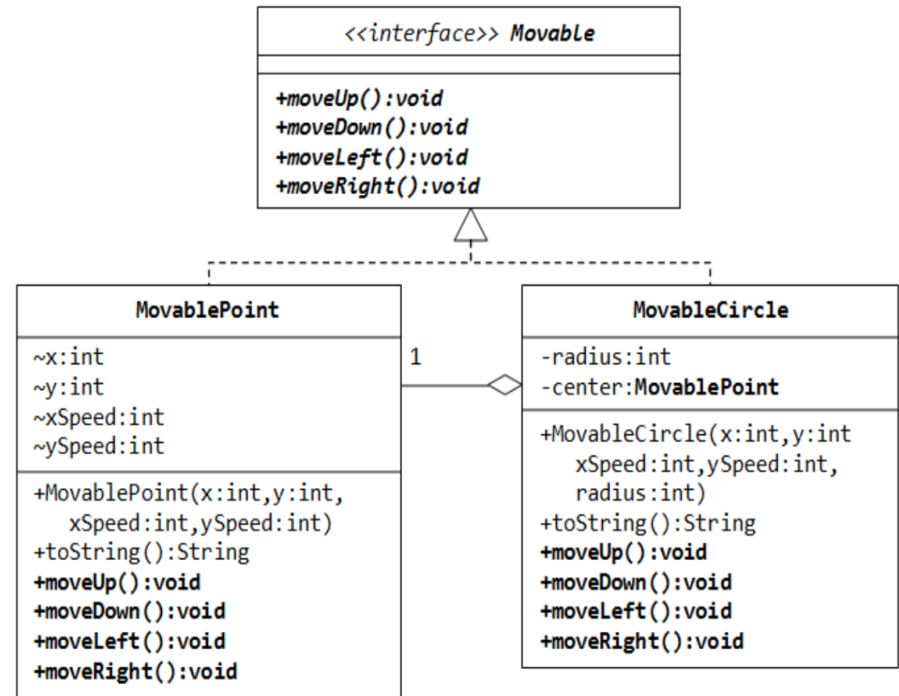
indicates that child (subclass) is considered to be a specialized form of the parent
(super class).



Class Diagram - Realization

Realization/Implementation

a relationship between two model elements, in which one model element implements/executes the behavior that the other model element specifies.

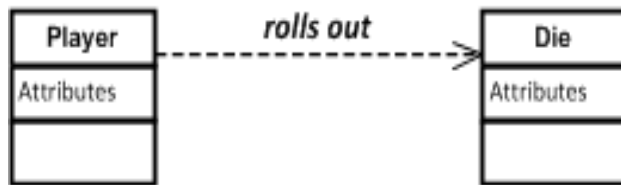


Class Diagram - Dependency

Dependency is normally created when you receive a reference to a class as part of a particular operation/method.

Dependency indicates that you may invoke one of the APIs of the received class reference and any modification to that class may break your class as well.

Dependency is represented by a dashed arrow starting from the dependent class to its dependency.

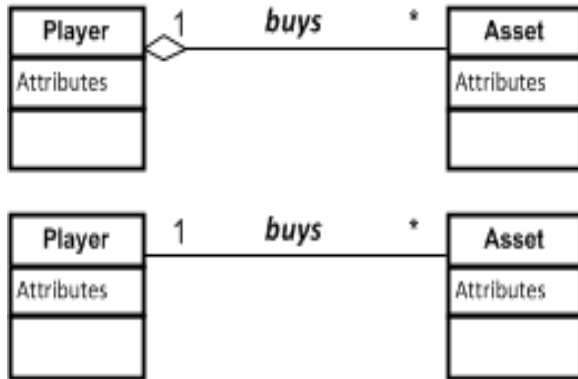


```
class Die { public void Roll() { ... } }

class Player
{
    public void TakeTurn(Die
die)
        { die.Roll(); ... }
}
```

Class Diagram - Aggregation

Aggregation represents one-to-many / many-to-many / part-whole relationships (i.e. higher multiplicity), can be represented by via association.



```
class Asset { ... }
```

```
class Player {
    List<Asset> assets = ArrayList<Asset>();
    public void AddAsset(Asset
newlyPurchasedAsset)    {
        assets.add(newlyPurchasedAssest); ... }
    ...
}
```

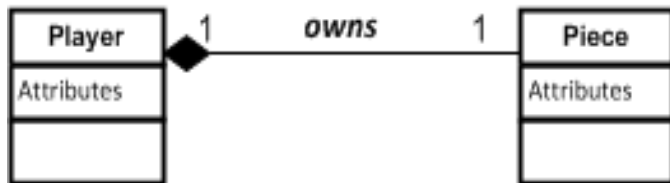
“has a” or “**is part of**” relationship.

Class Diagram - Composition

Composition relates to **instance creational responsibility**.

When class B is composed by class A, class A instance **owns the creation** or controls lifetime of instance of class B.

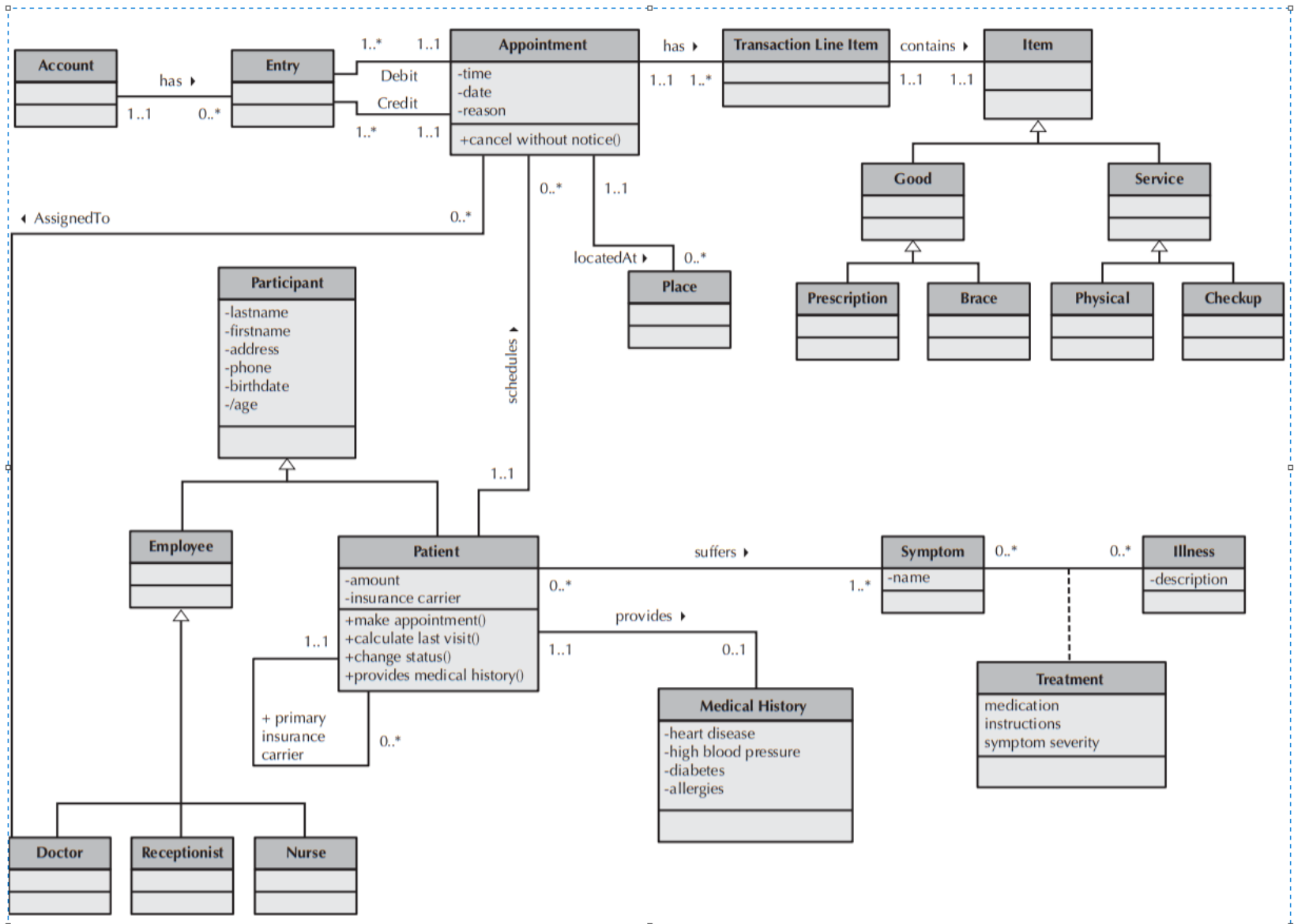
When class instance A instance is destructed (garbage collected), class B instance would get destructed as well. Composition is indicated by line connecting two classes with addition of a solid diamond at end of the class who owns the creational responsibility.



```
public class Piece { ... }

public class Player
{
    final private Piece piece;
    public Player(){
        Piece piece = new
        Piece();
    }
}
```

Class Diagram

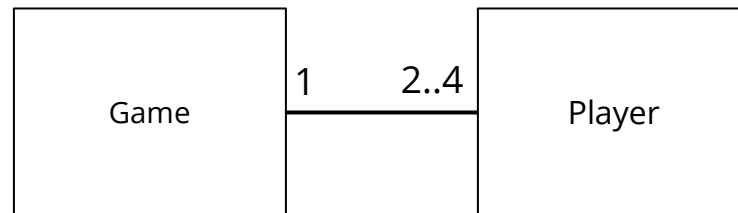
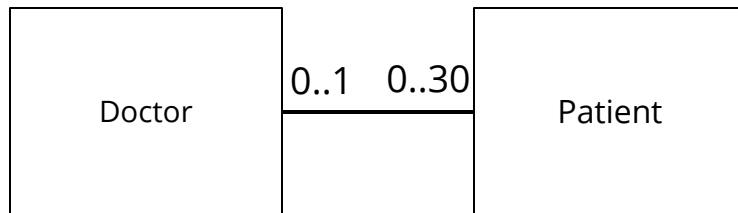
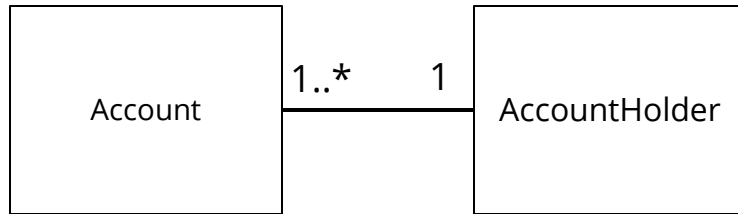


Class Diagram - Exercise

- An account has one and only one accountholder. An accountholder has one or more accounts.
- A doctor can have no more than 30 patients. A patient can have 0 or 1 doctor.
- A game can have 2 or 4 players. A player can have one game.

Class Diagram – Exercise Solution

- An account has one and only one accountholder. An accountholder has one or more accounts.
- A doctor can have no more than 30 patients. A patient can have 0 or 1 doctor.
- A game can have 2 or 4 players. A player can have one game.



Sequence Diagram

Interaction diagrams

- describe how a group of objects collaborate in some behavior – for a single use case;
- visualize the interactive behavior of the system.
- show a number of objects and the messages that are passed between these objects within the use case.

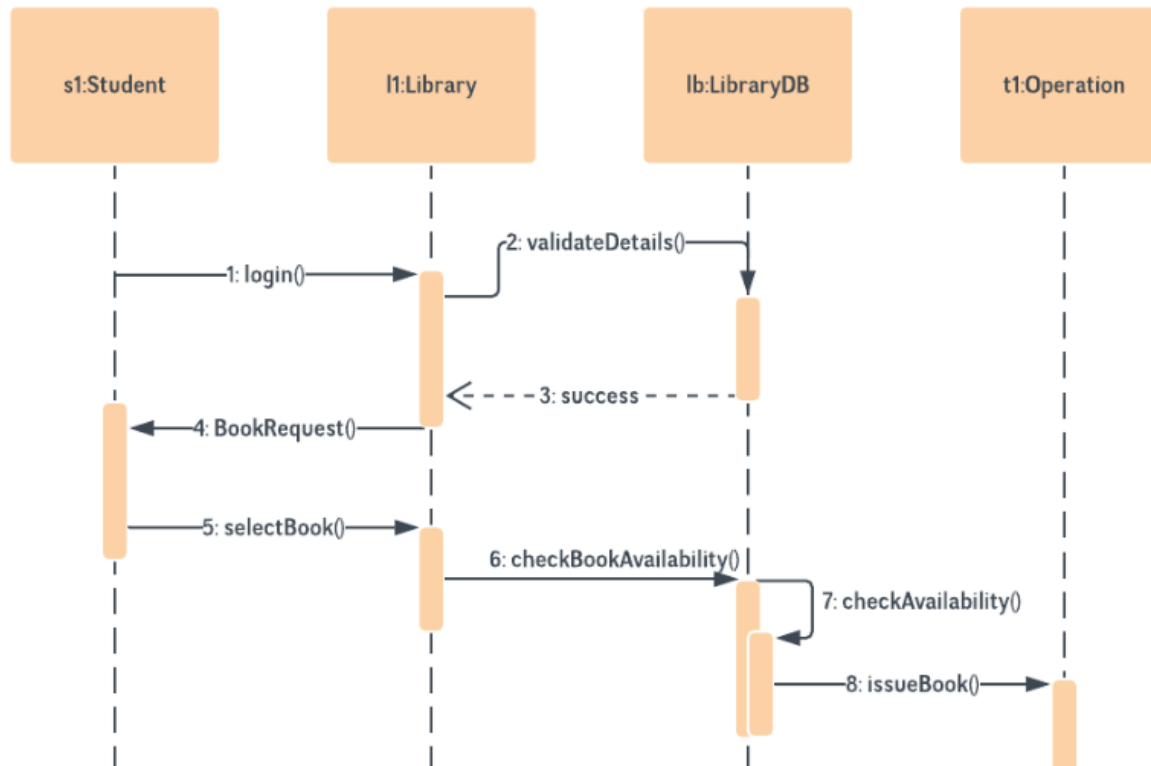
Sequence diagram and communication diagram are interactions diagrams.

Sequence Diagram

- To capture the dynamic behaviour of a system.
- To describe the message flow in the system.
- To describe the structural organization of the objects.
- To describe the interaction among objects.

Sequence Diagram

- A sequence diagram is a dynamic model that shows the explicit sequence of messages that are passed between objects in a defined interaction.



Sequence Diagram

Vertical dimension

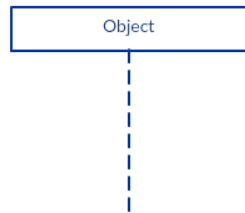
- represents a timeline
- starts at the top
- descends gradually to mark the sequence of interactions

Horizontal dimension

- represent all objects involved in the interaction
- each object is represented by a lifeline
- each object has a column (activation or execution)
- messages exchanged between them are represented by arrows.

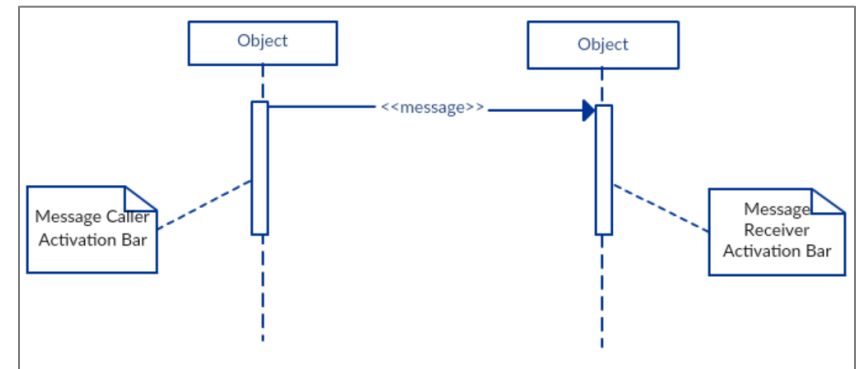
Sequence Diagram

Lifeline



Activation bar

indicates that an object is active (or instantiated) during an interaction between two objects. The length of the rectangle indicates the duration of the objects staying active.



Sequence Diagram

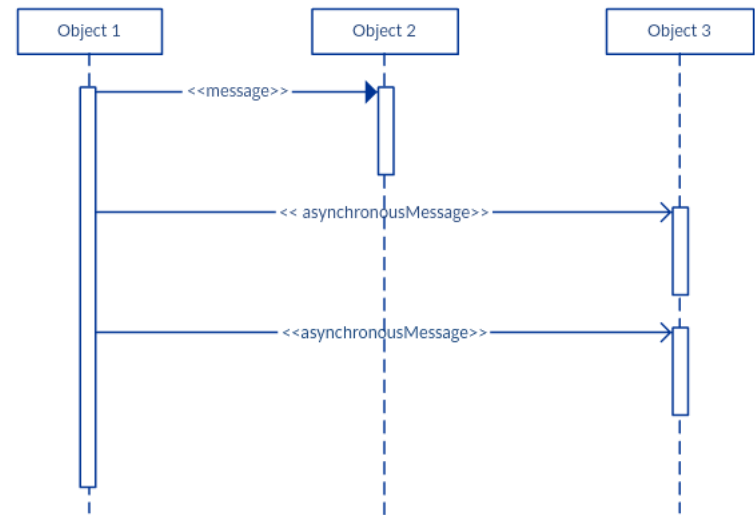
Message arrow

arrow from the Message Caller to the Message Receiver



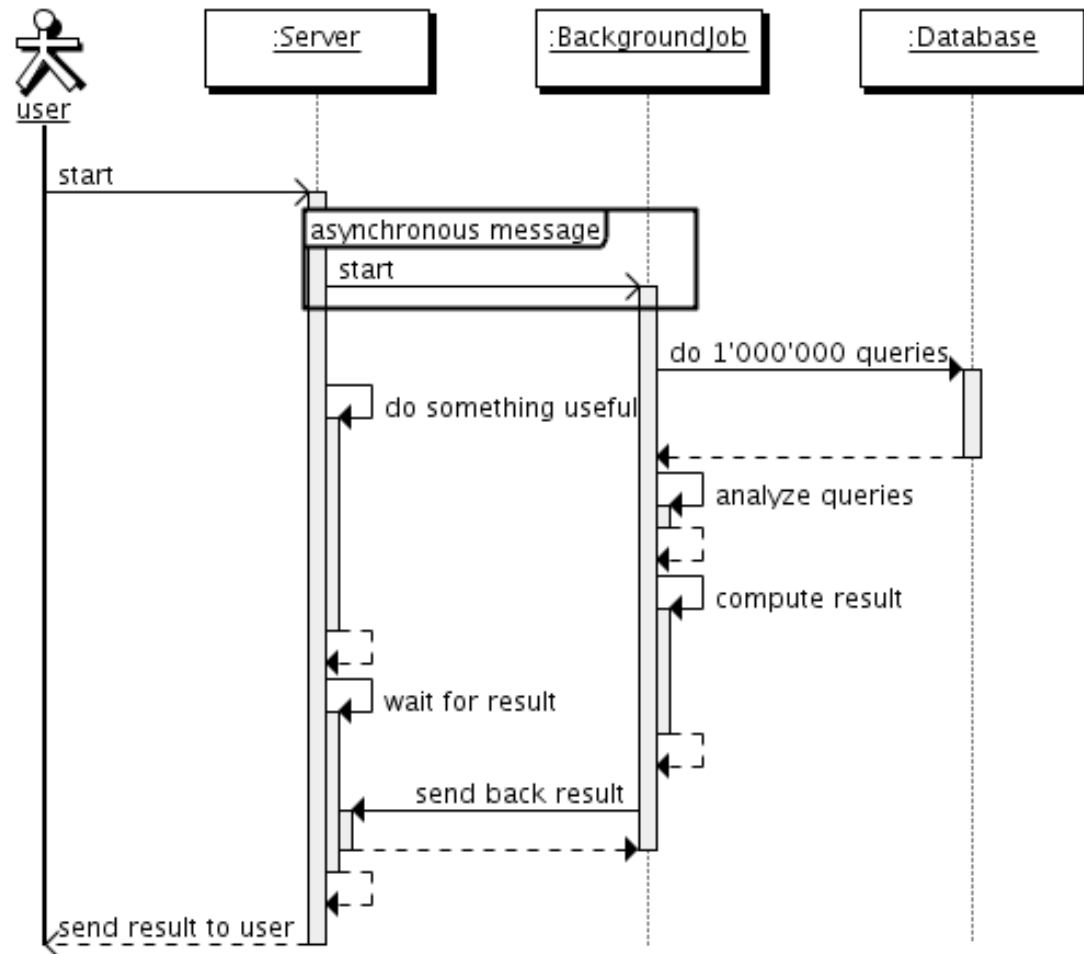
Synchronous message is used when the sender **waits** for the receiver to process the message and return before carrying on with another message.

Asynchronous message is used when the message caller **does not wait** for the receiver to process the message and return before sending other messages to other objects within the system (multithreaded applications, event-driven applications).



Sequence Diagram

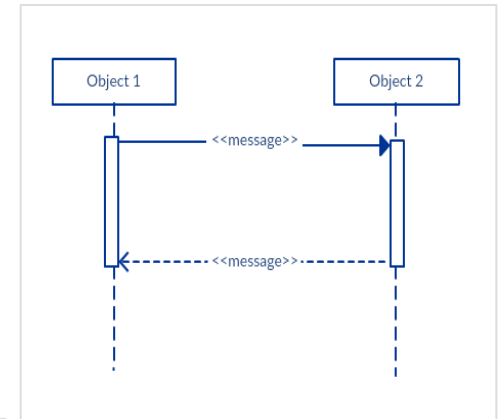
If a caller sends an **asynchronous message**, it can continue processing and doesn't have to wait for a response.



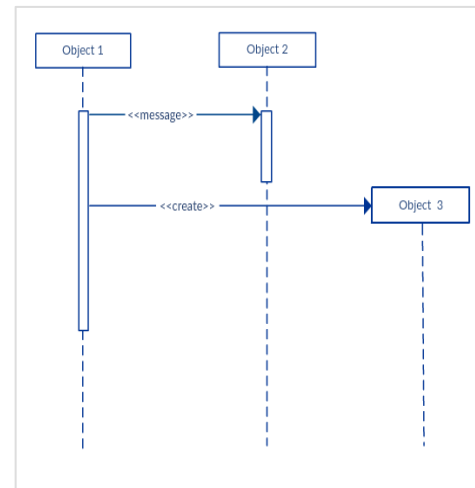
Sequence Diagram

Return message

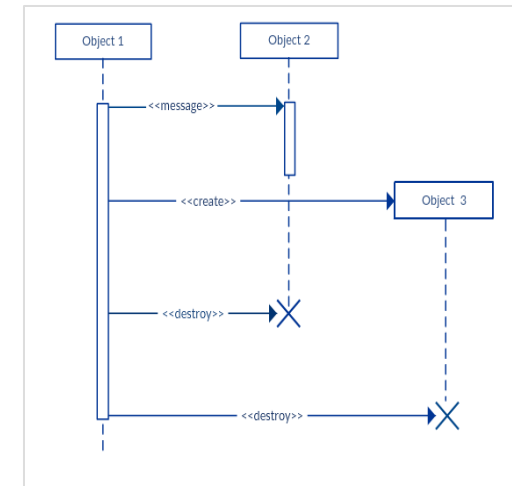
A return message is used to indicate that the message receiver is done processing the message and is returning control over to the message caller.



Participant **creation** message



Participant **destruction** message



Sequence Diagram

Reflexive message

object sends a message to itself

Sequence **fragments**

shows complex interactions such as **alternative flows** and **loops** in a more structured way.

Loop fragment

represents a repetitive sequence with the guard condition.

Alt alternative combination fragment

a choice needs to be made between two or more message sequences.

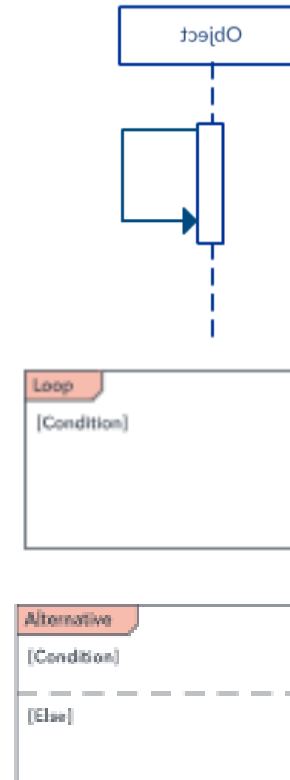
It models the “if then else” logic.

opt

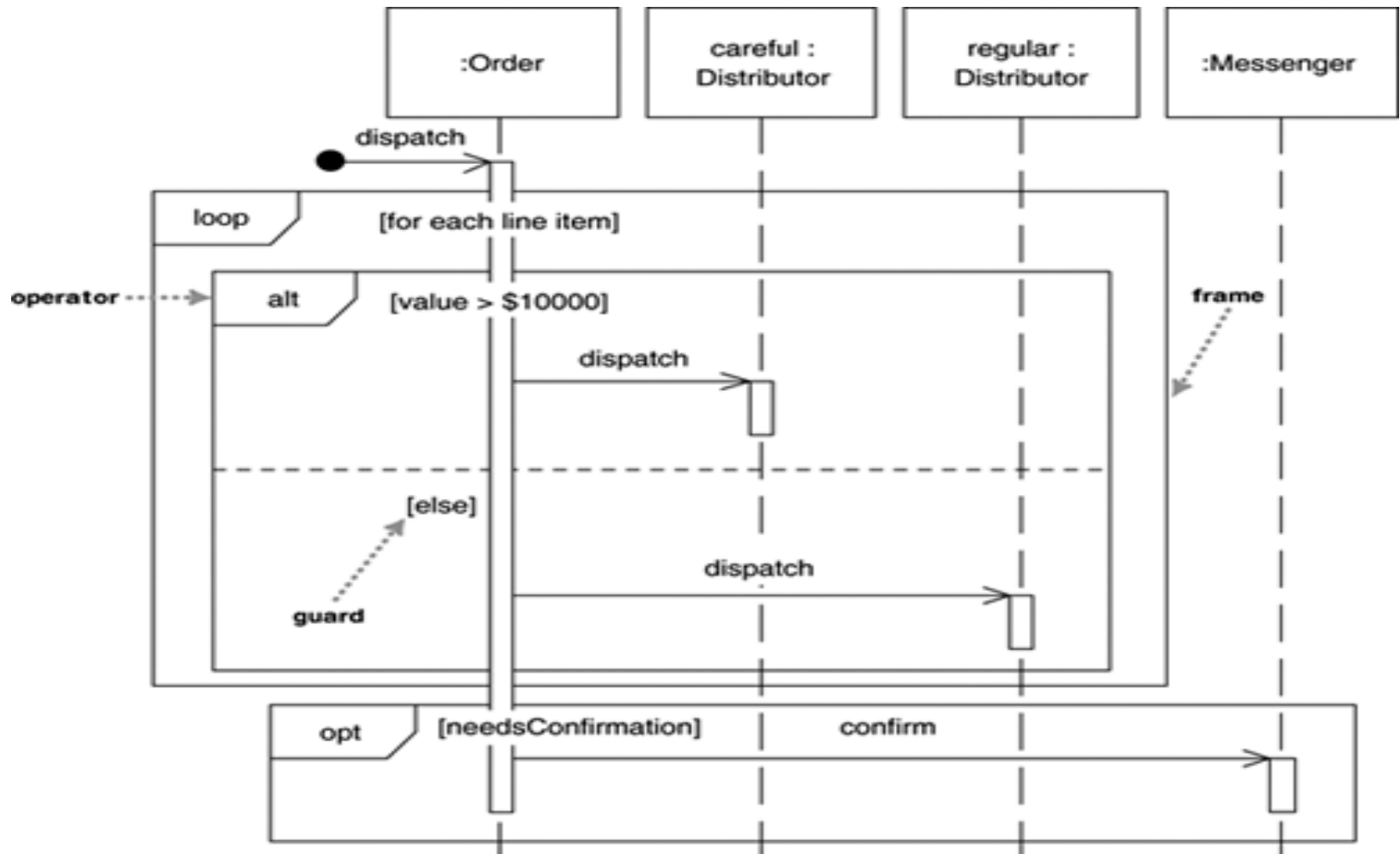
Defines condition to a single call - the call will execute only if the supplied condition is true . Equivalent to an alt with only one trace.

par

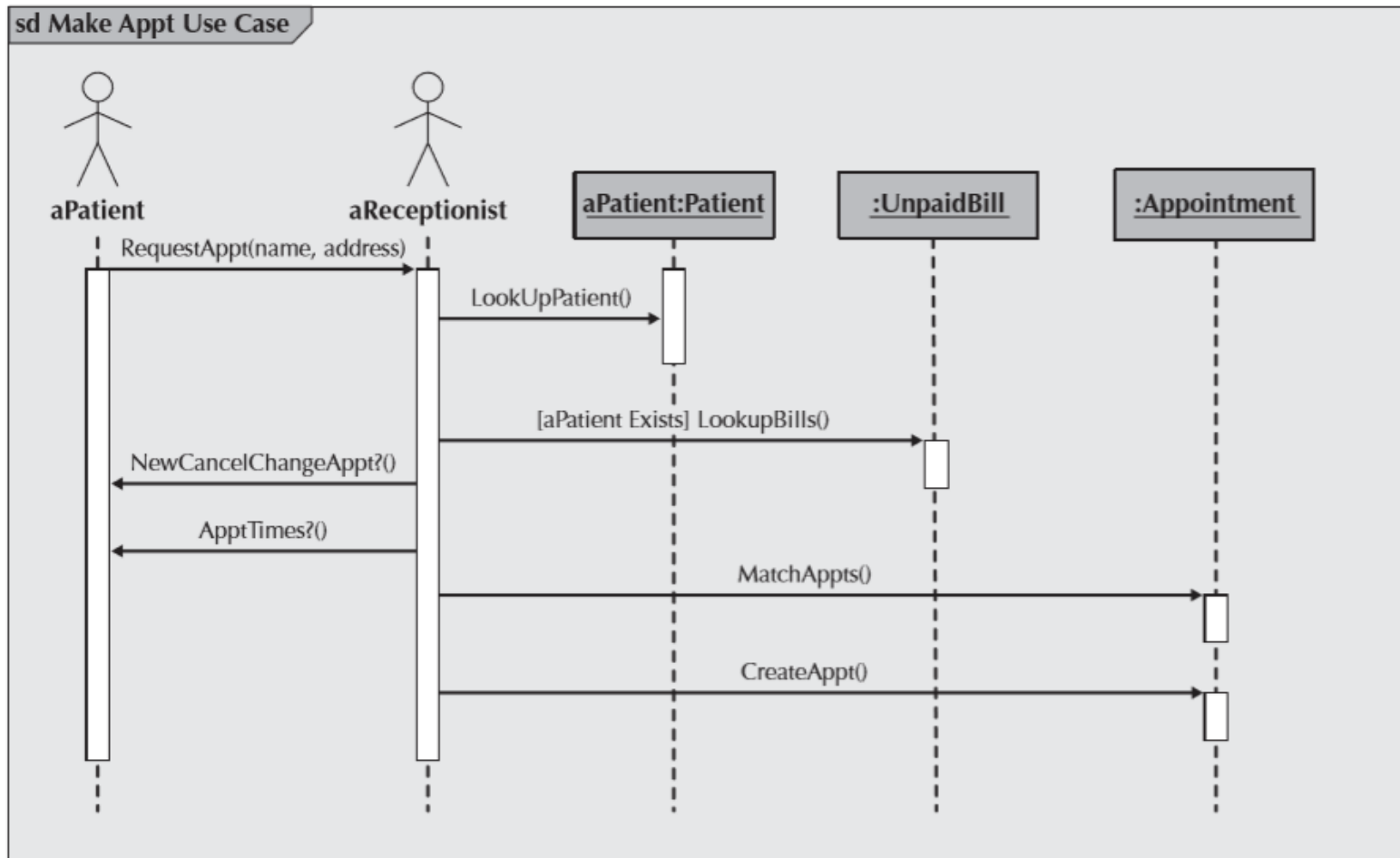
Defines that the calls within the fragment run in parallel.



Sequence Diagram



Sequence Diagrams



Sequence Diagrams

Pros:

- Sequence diagrams make the calls between objects clear, which objects are doing which processing.
- Model the flow of control by time sequence.

Cons:

- Sequence diagrams aren't good at showing details of algorithms (loops and conditional behavior)

Sequence Diagram - Exercise

Use case: Add a new advert to a campaign

The **campaign manager** selects a particular **client**.

The **campaign manager** lists **campaigns** for the **client**.

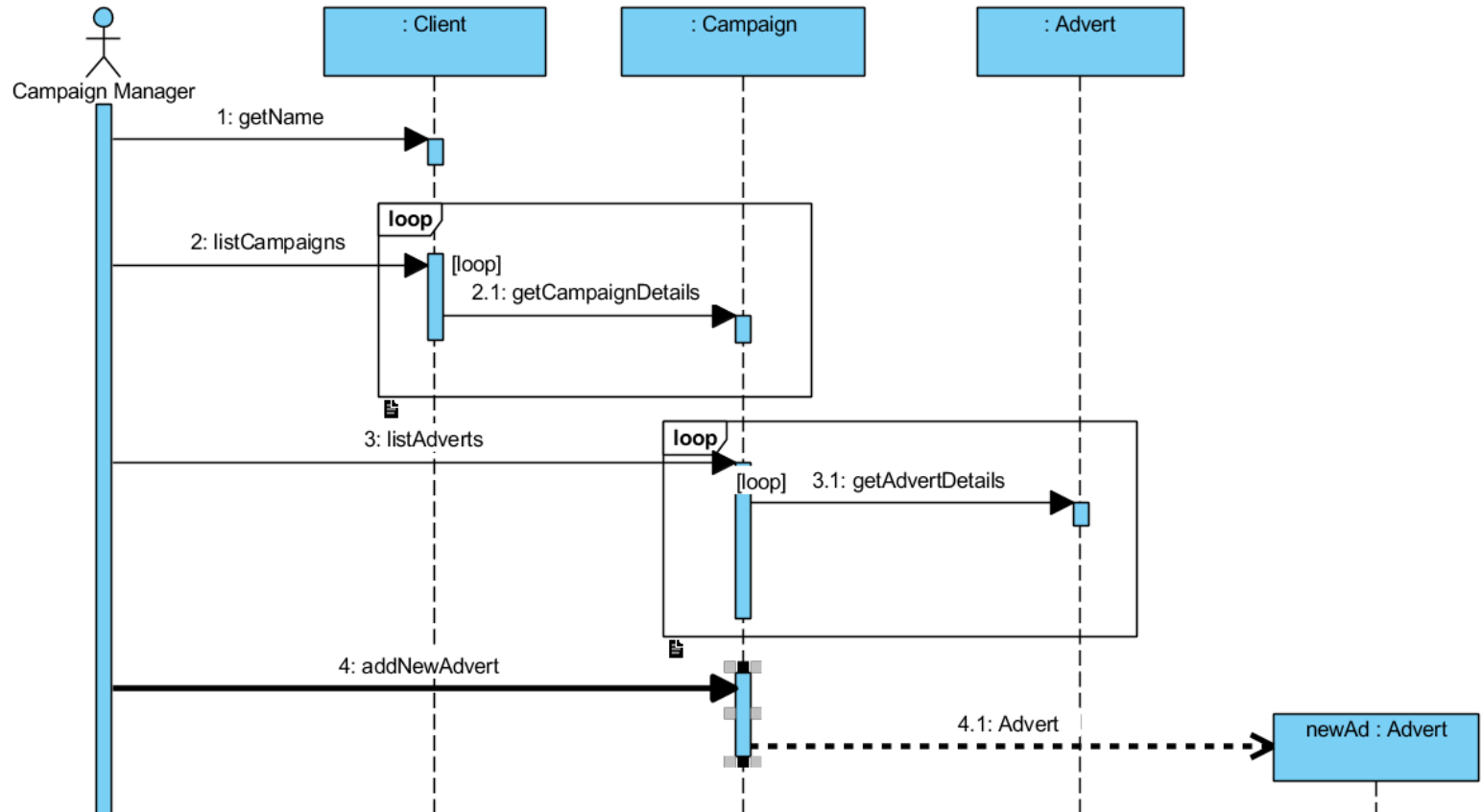
The **campaign manager** selects the required **campaign** for the selected **client**.

The **campaign manager** lists all the **adverts** for the **campaign** with adverts' details.

The **campaign manager** adds new **advert** to the **campaign** with completed details for the **advert**.

Sequence Diagram – Exercise Solution

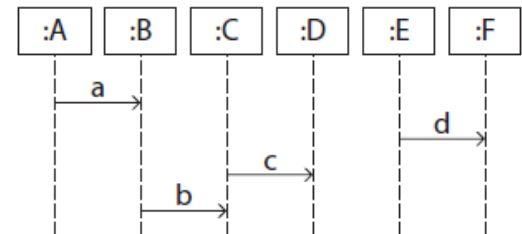
sd add a new advert to a campaign



Sequence Diagram – Exercise

Questions

You are given the following sequence diagram. Which traces are possible?

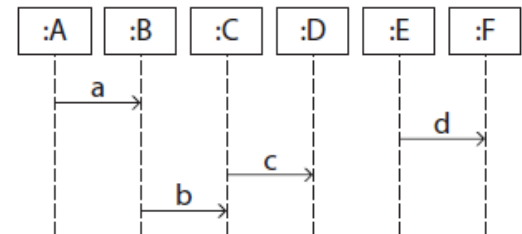


1. $b \rightarrow a \rightarrow d \rightarrow c$
2. $d \rightarrow c \rightarrow b \rightarrow a$
3. $d \rightarrow a \rightarrow c \rightarrow b$
4. $a \rightarrow d \rightarrow c \rightarrow b$
5. $d \rightarrow a \rightarrow b \rightarrow c$

Sequence Diagram – Exercise

Questions

You are given the following sequence diagram. Which trace is possible?



1. $b \rightarrow a \rightarrow d \rightarrow c$

2. $d \rightarrow c \rightarrow b \rightarrow a$

3. $d \rightarrow a \rightarrow c \rightarrow b$

4. $a \rightarrow d \rightarrow c \rightarrow b$

5. $d \rightarrow a \rightarrow b \rightarrow c$

Upcoming

- Lab Assignment 3 – XP Spike 3 Due Friday
- Quiz Due Friday

References

- Object-Oriented Systems Analysis and Design Using UML by Simon Bennett, Ray Farmer (2010) ISBN: 978-0077125363
- Dennis, Alan, et al. System Analysis and Design. Wiley, 2013.
- Object-Oriented Analysis and Design with Applications. Pearson Education (US), 2011.
- "Object Oriented Analysis and Design Tutorial." Www.tutorialspoint.com, Tutorials Point, www.tutorialspoint.com/object_oriented_analysis_design/.
- Omeecode. "Object Oriented Analysis and Design." CodeProject - For Those Who Code, CodeProject, www.codeproject.com/Articles/1137299/Object-Oriented-Analysis-and-Design.
- "Reasons for Software Project Failure Globally 2015 | Statistic." Statista, www.statista.com/statistics/627648/worldwide-software-developer-survey-project-failure/.

COSC 205

Client

Meetings

Estimations and story points



Spike Review

- During the spike, you should have figured out how to
 - Pair program
 - Test first
 - Refactor
- Also developed self-awareness of how long certain tasks take you
- Now it is time to meet with your client and begin the actual project
- We will cover these main ideas
 - Estimating user stories
 - How much work your group can accomplish in a sprint
 - Negotiating with customers
 - User stories → tasks

The Client

- Relationship with the client will help determine the success of the project
 - Hopefully, the client is available as much as possible
 - Comfortable asking client questions
 - Delivering bad or good news as soon as possible
- XP requires the use of social skills to be able to succeed

The Client - Exercise

- How is the client in this course different from “real” clients?

The Client - Exercise

- How is the client in this course different from “real” clients?
 - Instructors/college solicited the client’s involvement
 - Client wasn’t shopping around for best development team
 - Client is performing a service for you
 - Different expectations between you and customer
 - You as students don’t have as much say as to who your client is

The Client's Job

- 4 variables in software development: **cost, time, quality, scope**
- Client in real-world is not the same as a client in this course
 - Client knows the end date of your project, so **time** is taken out of the customer's hands
 - **Cost** is irrelevant
- **Quality**
 - With a fixed-end date, at some point customer will have to decide that something is acceptable
 - E.g. is correct errors more relevant then adding 2 new features with 2 weeks left
- **Scope**
 - At all times, customer decides what is to be done next (scope)
 - So, even as a developer you may want parts of a system to be included, but client gets final say

The Client in this Project

Investment

- Customer is not paying for your work
- Affect on you
 - Group members might make observation that customer isn't paying but also you are paying tuition
 - However, customer is adding a lot of value to the class as they aren't compensated for their time either
- Affect on customer
 - Affect how your efforts and entire project is viewed
 - Oftentimes, having a higher price tag on something will cause people to want to buy it more

The Client in this Project

Commitment

- What has client signed up for?
 - Initial meetings to turn ideas into user stories
 - Negotiate how much done iteration-iteration
 - Available to answer questions
 - Run acceptance tests on each iteration
 - Full time job for client
- Commitment levels will vary from client to client

The Planning Game

- Spending time talking to members of your team and to the client
- User stories
 - Creating
 - Revising
 - Estimating
 - Ranking
 - Coding
 - Accepting

Meeting with the Client

- Goal is for “you” to get the user stories
- Client has the ideas around what their project requires
- Some parts clear, some more detailed
- Your job is to translate these ideas to user stories
- Always write user stories in language client can understand

Meeting with the Client

- Talk in a manner the client will understand as well
- Ex: You ask the client “Is there a persistence requirement?”
 - They many look confused and say “I don’t think there is any need”
 - However, they may not even realize what that means. So, your job is to break it down further for them
- Do you want to work on the same document in different sessions?
- Would you like your work to be saved automatically?
- Always get a full understanding from your client

Meeting with the Client

- When you have enough user stories to get going on the project, stop
- Then have the client rank the stories in order of importance
- Maybe some items fall under the “if there is enough time” category
- This may take two meetings in our project.
 - First meeting to get a sample requirements sheet and more details from the client.
 - Second meeting convert requirements to user stories and give them to the client

Analysis and Estimation

- Stack of user stories containing the client's description of the most immediate features
- Time to gather with your team and ask some questions about user stories:
 - Do you understand what is being described?
 - Can you estimate the story?
 - Is there a way that the client can test that you have finished the story?
 - Is the story too large?

Estimation

- XP Spike provided you with a better idea of how much you can do in an hour and how many hours per week you can commit to working
- Estimating is hard
- These estimates are important as it essentially tells the client how much work can be done
 - Don't want to give too little work and not too much

Story Points

- Story points are units of measure for expressing an estimate of the overall effort required to fully implement a product backlog item or any other piece of work.
- Teams assign story points relative to work complexity, the amount of work, and risk or uncertainty.
- Over time, this helps teams understand how much they can achieve in a period and builds consensus and commitment to the solution.

Story Points and Planning Poker (Scrum Poker)

- Scrum poker, also known as “planning poker” and “pointing poker”, is a gamified technique that development teams use to guess the effort of project management tasks.
- These estimations are based on the entire group’s input and consensus, making them more engaging and accurate than other methods.
- To help gauge the number of story points for the relevant tasks, teams use planning poker cards, which are like poker cards.

How does planning poker work?

1. Distribute an identical deck of cards to everybody. Each one has a number that the team has agreed to use as their estimate. Numbers represent number of story points. (1,2,4,8 etc...)
2. Product owner takes the user story and reads it aloud
3. Discuss the story (ask questions)
 - How should we handle the work?
 - How many people are expected to get involved?
 - What skills will be needed to work on the story?
 - How should we tackle any roadblocks that delay progress?

How does planning poker work?

4. Select and share

- After the discussion, each person will privately select a card from the deck and show them at the same time
- If a player shows a higher card, it conveys that the story will be completed with greater difficulty and take a longer period to complete. Keep in mind that it's common for estimates to vary a lot

5. Reach a consensus

- When team members show the same card, that number turns into a consensus.
- Participants with higher or lower estimates than others will communicate their points of view. Then, they'll attempt to convince their teammates to understand their differing numbers.
- Once this new deliberation ends, everyone will go through their deck and show them again.
- Usually, the estimates start to converge after the second round. If not, then the process repeats itself until the team agrees on a single number.

When to hold planning poker sessions?

- Usually, teams arrange a session after creating the initial backlog. Although sessions can sometimes take more than one day, it leads to the development of initial estimates that are helpful in sizing or scoping the project.
- Items are added to the product backlog incrementally throughout the project's lifespan.
- That's why it's usually more convenient for teams to conduct sessions once per iteration. In most cases, this happens some days after the iteration ends. Similarly, it also occurs right after a daily standup (a type of agile meeting) because the entire team is present.

Planning Poker - Exercise

- Development team tasked with building a social media platform like Instagram or Twitter
- In groups using planning poker estimate the following user stories (say story points can be 1,2,4,8,16)
 - Have everyone think about what they think should be the number and then after a minute or too everyone shares their number with the rest of the group
- As a user, I want to be able to post a photo to my profile.
- As a user, I want to be able to add filters and effects to my uploaded photos.
- As a user, I want to be able to follow other users and see their posts on my feed.
- As a user, I want to receive push notifications for new likes and comments on my posts.

Determining Your Workload

- You can't just go to the customer and say:
"We've decided three 3 user stories will take four story points each and those two will take five story points each"
- This doesn't mean anything to the client
 - Don't understand what a story point is and how many story points you can do in a week
- More so concerned with how much work you can get done in a sprint cycle and how that relates to the full project
- XP Spike should have helped with estimates and over time you will get better at estimating the total work you can do in a sprint

Determining Your Workload

- After estimating out the sprint cycle then you can further split this into total number of hours your group can commit each week
- But because of the way this course is designed you must produce a way to consider story points with hours available in a week
 - Other courses going on so can't devote all your time to the project
 - Extracurriculars...
 - Commonly up to 10 hours per week per student should be devoted toward the project
- So, now you have 5 people in your group which amounts to 50 hours of work, which you as a group say is worth 30 story points

Reevaluating and Prioritizing the User Stories

- You have met with your team, estimated user stories, and produce how much you can accomplish in one sprint
- Client in conjunction with the team then:
 - Chooses that amount of work in estimated from the user story estimates provided
 - Ex: 40 story points worth of work to be allotted, client picks user stories until that allotment is met according to their priority of tasks
 - Again, your team is involved in this process as well it is just client gets the final say

Choosing and Estimating Tasks

- You have the user stories for the sprint
- Now your job as a team is to break user stories into tasks (sub-tasks in Jira)
 - Sub-tasks = steps required to accomplish user story
- Afterwards, team members sign up for tasks
 - Keeping track of how many hours you put into a task as well
- Good time to assign tasks:
 - Meetings with your team, assign week-by-week tasks
 - Then after every week you are aware of how much work you have completed which will then help further down the line in producing better estimates

Dealing with Disappointment

- About a week into your sprint, you see estimates are way off (underestimated)
- Ensure you mention these issues in your daily scrum meetings
 - Allows for reassessment with your team
 - Have the product owner reach out to the client mentioning you may have taken on too much work
- Not a bad thing that you can't accomplish as many tasks
 - You mentioning this will only help the team more in the future
 - More accurate measurements
- XP is built upon conversation with your client
 - Honesty is better than lying
 - If team is struggling client should know that
 - Don't wait until the end

Upcoming

Final Spike Presentation (no other lab starts this week)

References

- <https://www.atlassian.com/agile/project-management/estimation>
- <https://www.atlassian.com/blog/platform/scrum-poker-for-agile-projects>

COSC 205

First Iterations

Acceptance Tests



Acceptance Tests

- Acceptance testing is a type of testing that ensures a system or application meets the specified requirements and is acceptable to end-users or stakeholders.
- The primary goal of acceptance testing is to validate that the software behaves as expected and fulfills the business requirements.
- Hence, it is important to know when you are finished with a story and be able to demonstrate this to the client

Acceptance Tests

- Acceptance test criteria will vary from client-client
 - Some may want to really exercise your application you built
 - Others, may want a proof of concept
- Acceptance tests provide a way for a client to tell you “When I do this and get that result, then this story will be completed to my satisfaction”
- Having this set criteria in place helps developers be more confident that they are done with a story

Writing Acceptance Tests

- Over time the client's definition of what they require for a user story to be considered done may change
- Hence, early on it is not a requirement to have acceptance tests fully fledged out but more so have the client generate "Done" scenarios of when they believe the story is complete
- Afterwards, we can turn these "Done" scenarios into acceptance tests

Writing Acceptance Tests

- It is equally important however, to write user stories that are testable
 - i.e. in the judgement of the customer and the development team, it is possible to test the story
- So, if you write a user story that isn't testable (i.e. not detailed enough requirements from client) then you must recreate the user story

Exercise - Are these user stories testable?

- As a user, I want the user interface to be friendly and easy for the clerk to use
- As a clerk, I want the price to be retrieved quickly once the item has been scanned in or entered
- As a user, once items are entered in by the clerk, their name and price are added to a receipt, and the price is added to the subtotal

Are these user stories testable?

- As a user, I want the user interface to be friendly and easy for the clerk to use

NO

- No direction is given
- Client has a different idea of friendly from the development team
- Can't generate an acceptance test for this

Are these user stories testable?

- As a clerk, I want the price to be retrieved quickly once the item has been scanned in or entered

NO

- What is meant by quickly?
- What is an acceptable response time?

Are these user stories testable?

- As a user, once items are entered in by the clerk, their name and price are added to a receipt and the price is added to the subtotal

YES

- Clear indication of what is required
- Easy way to test it via checking the receipt to see items are being added and checking to see if price of all items matches the subtotal

Clients and Acceptance Tests

- So, when you are gathering information to generate user stories for the client it is important to probe as much information you can, so your development team has a clear understand of what is considered “Done” for the client
- There may be multiple acceptance tests or definitions of done for a single user story
- If you are writing an acceptance test, name it and indicate the user story it is intended to test

Example Acceptance Tests

- Acceptance tests can be written in simple language
- No specific format to follow
- When writing the acceptance test document (end of course) it will be a requirement to go into more detail
- Ex:
 - Feature: backup database
 - Acceptance criteria:
 - The backup file is successfully created
 - Can navigate to the proper page to create the backup file
 - Etc...
 - Feature: log in
 - Acceptance criteria:
 - After logging in, user session should persist throughout each page

Exercise – Definitions of Done

- For the following user stories, write some Definitions of Done for each user story
 1. As a user, I want to reset my password so that I can regain access to my account if I forget it
 2. As an administrator, I want to view a list of all users with their roles so that I can manage permissions effectively
 3. As a user, I want to receive an email confirmation after making a purchase so that I know my order was successful

Exercise – Definitions of Done Possible Solutions

- As a user, I want to reset my password so that I can regain access to my account if I forget it
 - The "Forgot Password" link is accessible on the login page
 - Clicking the link leads to a form where users can input their email
 - An email with a secure, time-sensitive reset link is sent to the user
 - The reset link allows the user to set a new password that meets the security requirements (e.g., at least 8 characters, includes a number and special character).

Exercise – Definitions of Done Possible Solutions

- As an administrator, I want to view a list of all users with their roles so that I can manage permissions effectively
 - The admin dashboard displays a paginated table of users
 - The table includes columns for name, email, and role
 - The admin can sort users by name or role.
- As a user, I want to receive an email confirmation after making a purchase so that I know my order was successful
 - After completing a purchase, an email is automatically sent to the user
 - The email includes order details (e.g., items purchased, total cost, estimated delivery date)
 - A copy of the email is logged for troubleshooting purposes.

Acceptance Tests Suite

- Over time throughout your project, you will start building up your acceptance test suite (like the test suite we discussed before)
- Allows for your group to show running tests at each meeting
- Converted to an official acceptance test document at the end of the project

Beginning the First Iteration/Sprint

- Group roles
- Pick a project
- Organize team meeting times
 - At least 2 daily scrums per week
 - Full group meetings, meetings in pairs, more meetings the better
 - In person, discord, etc...
- Create GitHub repository
- Jira/Confluence will be automatically assigned to groups

Beginning the First Iteration/Sprint

- We talked last lecture about the first meetings with the client
- First meeting
 - Introduction
 - Requirements list
- Second meeting
 - Show client user stories with estimates
 - Get plan for first sprint
- Estimation will be hardest during the first sprint
- Breaking down user stories (for sprint) into sub-tasks

Beginning the First Iteration/Sprint

- During the initial meetings with the client, you should set times that work for you and your client to meet
 - Ensure at least one at the start and end of each sprint (overlap with one another)
 - One per week would be even more beneficial
 - Same time every week, don't change the times around too much unless client insists on this
 - If any extra communication is required with the client the Product Owner should reach out via email
 - If the client is local, you can set up face-to-face meetings

Living the Iteration/Sprint

Sample week overview (~10 to 16 hours per week commitment)

- Monday:
 - Meet with team to conduct daily scrum (to-do, done, blocks)
 - Do some work in pairs/individually/group
- Tuesday:
 - Working day (no meetings)
- Wednesday
 - Bi-weekly meeting with instructor
 - Meeting with team
- Thursday
 - Daily scrum
 - Client meeting (ensure your team knows what to ask/show the client before the meeting to not waste their time)
- Friday
 - Working day
 - Meeting with another group member

Upcoming

- Quiz 4
- Next Class
 - Sprint 1: Inception, End of Sprints, Project Planning

COSC 205

Project Management

Project Planning



Software Project Management

- Concerned with activities involved in ensuring that software is delivered on time and on schedule and in accordance with the requirements of the organizations developing and procuring the software.
- Project management is needed because software development is always subject to budget and schedule constraints that are set by the organization developing the software.

Factors Influencing Project Management

- Company size
- Software customers
- Software size
- Software type
- Organizational culture
- Software development processes
- These factors mean that project managers in different organizations may work in quite different ways.

Universal Management Activities

- Project planning
 - Project managers are responsible for planning, estimating and scheduling project development and assigning people to tasks.
- Risk management
 - Project managers assess the risks that may affect a project, monitor these risks and act when problems arise.
- People management
 - Project managers must choose people for their team and establish ways of working that leads to effective team performance.

Universal Management Activities

- Reporting
 - Project managers are usually responsible for reporting on the progress of a project to customers and to the managers of the company developing the software.
- Proposal writing
 - The first stage in a software project may involve writing a proposal to win a contract to carry out an item of work.
 - The proposal describes the objectives of the project and how it will be carried out.

Project Planning

- Project planning involves breaking down the work into parts and assign these to project team members, anticipate problems that might arise and prepare tentative solutions to those problems.
- The project plan, which is created at the start of a project, is used to communicate how the work will be done to the project team and customers, and to help assess progress on the project.

Planning Stages

- The aim of planning at this stage is to provide information that will be used in setting a price for the system to customers.
- Project pricing involves estimating how much the software will cost to develop, taking factors such as staff costs, hardware costs, software costs, etc. into account
- At the proposal stage, when you are bidding for a contract to develop or provide a software system.
- During the project startup phase, when you have to plan who will work on the project, how the project will be broken down into increments, how resources will be allocated across your company, etc.
- Periodically throughout the project, when you modify your plan in the light of experience gained and information from monitoring the progress of the work.

After winning the contract

- At this stage, you know more about the system requirements but do not have design or implementation information
- Create a plan with enough detail to make decisions about the project budget and staffing.
 - This plan is the basis for project resource allocation
- The startup plan should also define project monitoring mechanisms
- A startup plan is still needed for agile development to allow resources to be allocated to the project

Development Planning

- The project plan should be regularly amended as the project progresses and you know more about the software and its development
- The project schedule, cost-estimate and risks must be regularly revised

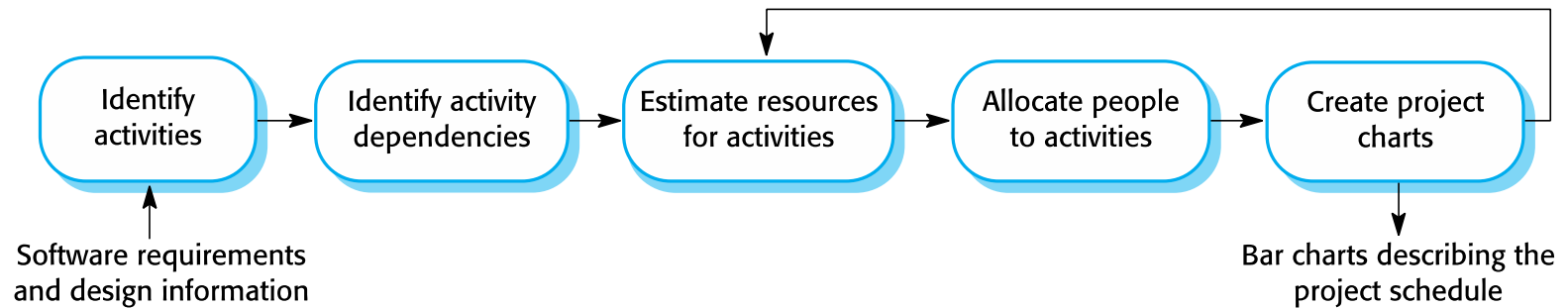
Project Plans

- A project plan sets out the resources available to the project, the work breakdown and a schedule for carrying out the work.
- Consists of:
 - Introduction
 - Project organization
 - Risk analysis
 - Hardware and software resource requirements
 - Work breakdown
 - Project schedule
 - Monitoring and reporting mechanisms

Project Scheduling

- Project scheduling is the process of deciding how the work in a project will be organized as separate tasks, and when and how these tasks will be executed.
- You estimate the calendar time needed to complete each task, the effort required and who will work on the tasks that have been identified.

Project Scheduling Process



Problems with Scheduling

- Estimating the difficulty of problems and hence the cost of developing a solution is hard.
- Productivity is not proportional to the number of people working on a task.
- Adding people to a late project makes it later because of communication overheads.
- The unexpected always happens. Always allow contingency in planning.

Agile Planning vs Plan-Driven

- Unlike plan-driven approaches, the functionality of these increments is not planned in advance but is decided during the development.
 - The decision on what to include in an increment depends on progress and on the customer's priorities.
- The customer's priorities and requirements change so it makes sense to have a flexible plan that can accommodate these changes.

Release and iteration planning

- Release planning involves selecting and refining the stories that will reflect the features to be implemented in a release of a system and the order in which the stories should be implemented.
- Stories to be implemented in each iteration are chosen, with the number of stories reflecting the time to deliver an iteration (usually 2 or 3 weeks).
- The team's velocity is used to guide the choice of stories so that they can be delivered within an iteration.

Agile Planning vs Plan-Driven - Question

- So why might having a planned schedule from the start be more beneficial?

Agile Planning vs Plan-Driven - Question

- Agile planning is reliant on customer involvement and availability.

This can be difficult to arrange, as customer representatives sometimes must prioritize other work and are not available for the planning game.

- Furthermore, some customers may be more familiar with traditional project plans and may find it difficult to engage in an agile planning process.
- Having the set-out schedule can help with price estimates
- Get a quick overview of some basic guidelines for the project to follow
- Most companies use a mix of plan-driven and agile
 - Aspects of each

Creating a Schedule

- Graphical notations are normally used to illustrate the project schedule.
- These show the project breakdown into tasks. Tasks should not be too small. They should take about a week or two.
- Calendar-based
 - Bar charts are the most used representation for project schedules. They show the schedule as activities or resources against time.

Projects Activities (tasks)

- Project activities (tasks) are the basic planning element. Each activity has:
 - a duration in calendar days or months,
 - an effort estimate, which shows the number of person-days or person-months to complete the work,
 - a deadline by which the activity should be complete,
 - a defined end-point, which might be a document, the holding of a review meeting, the successful execution of all tests, etc.

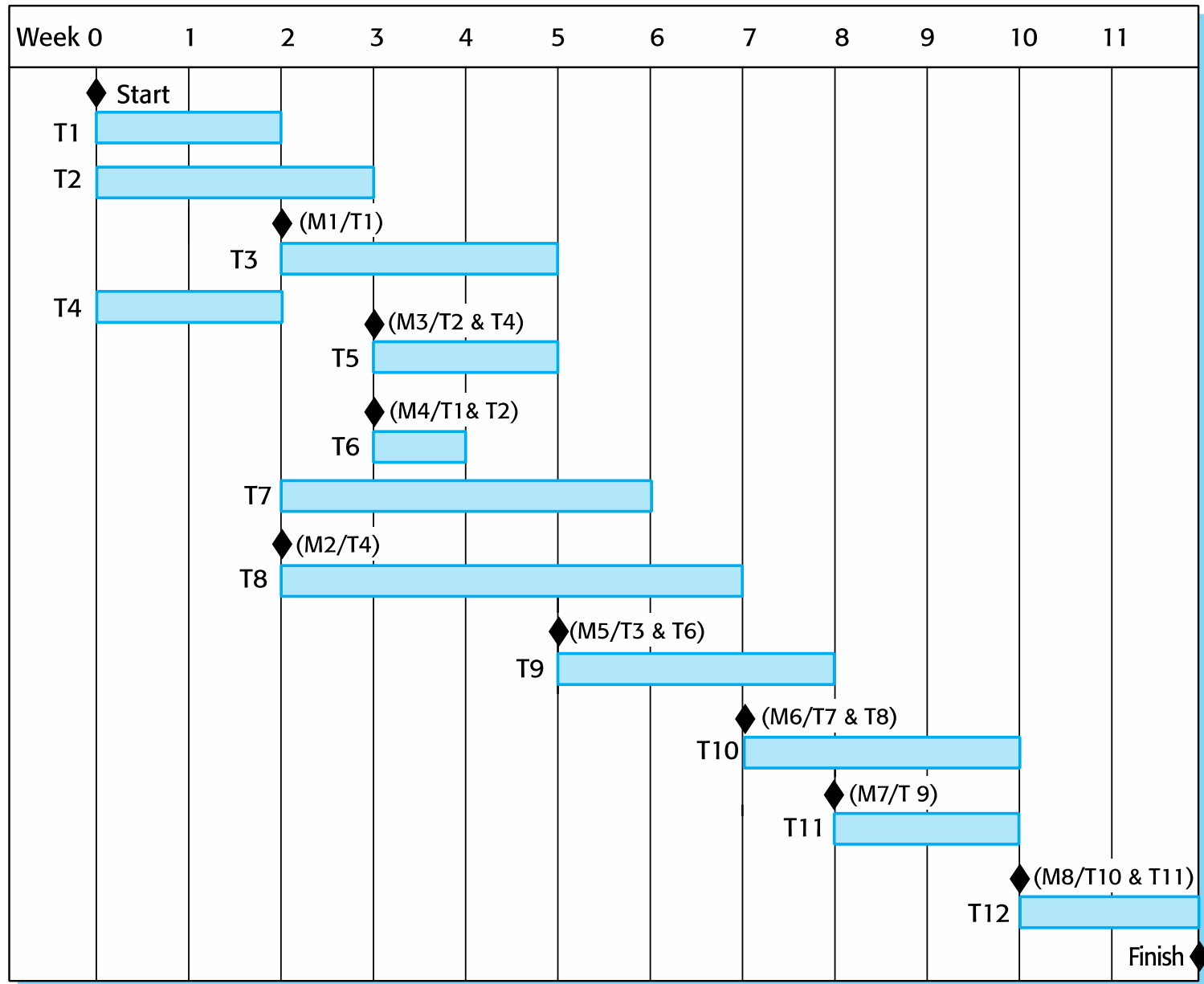
Milestones and Deliverables

- Milestones are points in the schedule against which you can assess progress, for example, the handover of the system for testing.
- Deliverables are work products that are delivered to the customer, e.g. a requirements document for the system, a prototype, etc

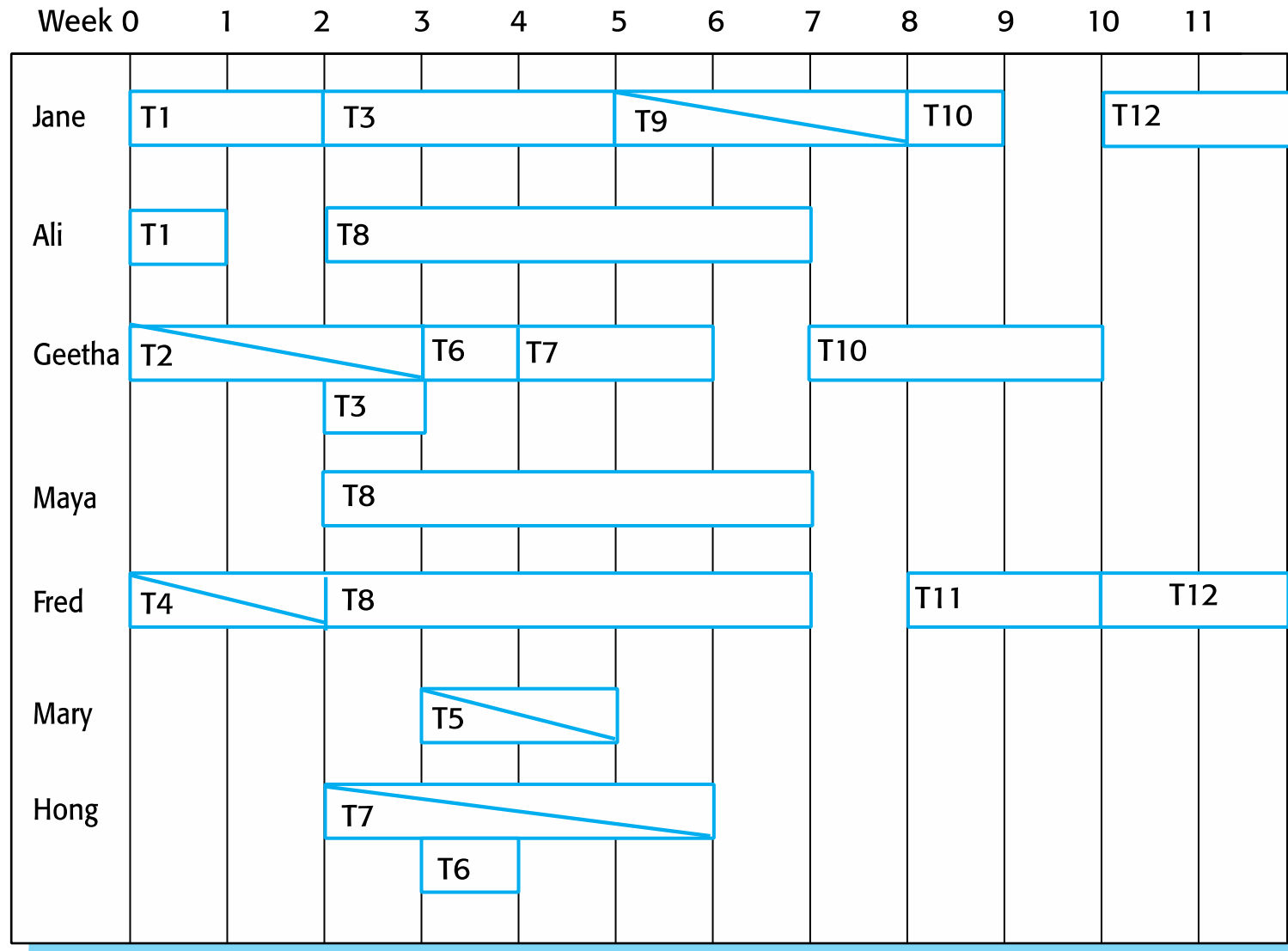
Tasks, Duration, Dependencies

Task	Effort (person-days)	Duration (days)	Dependencies
T1	15	10	
T2	8	15	
T3	20	15	T1 (M1)
T4	5	10	
T5	5	10	T2, T4 (M3)
T6	10	5	T1, T2 (M4)
T7	25	20	T1 (M1)
T8	75	25	T4 (M2)
T9	10	15	T3, T6 (M5)
T10	20	15	T7, T8 (M6)
T11	10	10	T9 (M7)
T12	20	10	T10, T11 (M8)

Activity Bar Chart



Staff Allocation Chart



Example Dependency Tasks

- Authentication System:
 - Task 1: Implement user registration.
 - Task 2: Implement login functionality.
 - Dependencies: User registration (Task 1) must be completed before users can log in (Task 2).
- Database Schema Design and Initialization:
 - Task 1: Design the database schema.
 - Task 2: Initialize the database with sample data.
 - Dependencies: Task 1 needs to be completed before Task 2, as the schema must be in place before data can be added.

Upcoming

- Lab 4
- Midterm next Monday
 - Midterm review lecture next Wed
- Risk management

COSC 205

Risk Management

People Management



Risk Management

- Project managers assess the risks that may affect a project, monitor these risks and act when problems arise.

Risk Management

- Risk management is concerned with identifying risks and drawing up plans to minimize their effect on a project.
- Software risk management is important because of the inherent uncertainties in software development.
 - Loosely defined requirements
 - Changes in customer needs
 - Difficulties estimating
- You must anticipate risks, understand the impact of these risks on the project, the product and the business, and take steps to avoid these risks.

Risk Classification

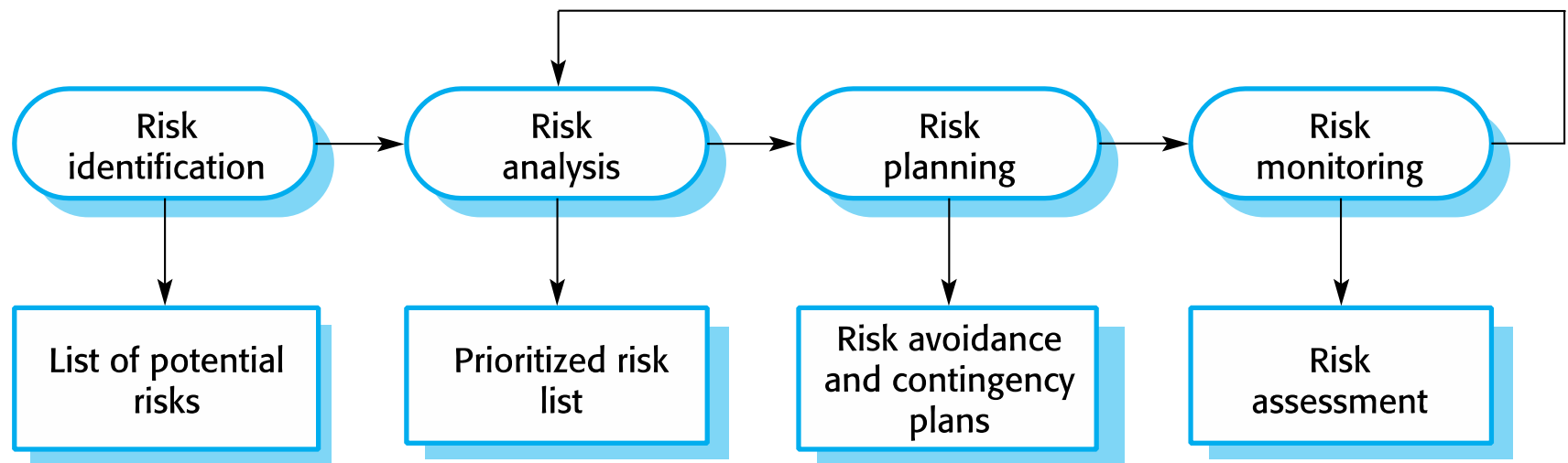
- There are two dimensions of risk classification
 - The type of risk (technical, organizational, ..)
 - what is affected by the risk:
- Project risks affect schedule or resources;
- Product risks affect the quality or performance of the software being developed;
- Business risks affect the organization developing or procuring the software.

Examples of Project, Product, and Business Risks

Risk	Affects	Description
Staff turnover	Project	Experienced staff will leave the project before it is finished.
Management change	Project	There will be a change of organizational management with different priorities.
Hardware unavailability	Project	Hardware that is essential for the project will not be delivered on schedule.
Requirements change	Project and product	There will be a larger number of changes to the requirements than anticipated.
Specification delays	Project and product	Specifications of essential interfaces are not available on schedule.
Size underestimate	Project and product	The size of the system has been underestimated.
Technology change	Business	The underlying technology on which the system is built is superseded by new technology.
Product competition	Business	A competitive product is marketed before the system is completed.

The Risk Management Process

- Risk identification
 - Identify project, product and business risks;
- Risk analysis
 - Assess the likelihood and consequences of these risks;
- Risk planning
 - Draw up plans to avoid or minimize the effects of the risk;
- Risk monitoring
 - Monitor the risks throughout the project;



Risk Identification

- May be a team activities or based on the individual project manager's experience.
- A checklist of common risks may be used to identify risks in a project
 - Technology risks.
 - Organizational risks.
 - People risks.
 - Requirements risks.
 - Estimation risks.

Examples of Different Risk Types

Risk type	Possible risks
Estimation	The time required to develop the software is underestimated. The rate of defect repair is underestimated. The size of the software is underestimated.
Organizational	The organization is restructured so that different management are responsible for the project. Organizational financial problems force reductions in the project budget.
People	It is impossible to recruit staff with the skills required. Key staff are ill and unavailable at critical times. Required training for staff is not available.
Requirements	Changes to requirements that require major design rework are proposed. Customers fail to understand the impact of requirements changes.
Technology	The database used in the system cannot process as many transactions per second as expected. Reusable software components contain defects that mean they cannot be reused as planned.
Tools	The code generated by software code generation tools is inefficient. Software tools cannot work together in an integrated way.

Risk Analysis

- Assess probability and seriousness of each risk.
- Probability may be very low, low, moderate, high or very high.
- Risk consequences might be catastrophic, serious, tolerable or insignificant.

Risk Types and Examples

Risk	Probability	Effects
Organizational financial problems force reductions in the project budget	Low	Catastrophic
Faults in reusable software components must be repaired before these components are reused.	Moderate	Serious
The database used in the system cannot process as many transactions per second as expected	Moderate	Serious
Software tools cannot be integrated	High	Tolerable
Customers fail to understand the impact of requirements changes	Moderate	Tolerable
Required training for staff is not available	Moderate	Tolerable

Risk Types, Probability, and Effects - Exercise

In your groups discuss how probable and the effect that each of the following risks could have on a project

Classify the probabilities as: very low, low, moderate, high or very high.

Risk consequences as: catastrophic, serious, tolerable or insignificant.

1. Code generated by code generation tools is inefficient
2. It is impossible to recruit staff with the skills required for the project
3. Key staff are ill at critical times in the project
4. Changes to requirements that require major design rework are proposed
5. The time required to develop the software is underestimated
6. The size of the software is underestimated

Risk Types, Probability, and Effects - Exercise

Risk	Probability	Effects
Code generated by code generation tools is inefficient	Moderate	Insignificant
It is impossible to recruit staff with the skills required for the project	High	Catastrophic
Key staff are ill at critical times in the project	Moderate	Serious
Changes to requirements that require major design rework are proposed	Moderate	Serious
The time required to develop the software is underestimated	High	Serious
The size of the software is underestimated	High	Tolerable

Risk Planning

- Consider each risk and develop a strategy to manage that risk.
- Avoidance strategies
 - The probability that the risk will arise is reduced;
- Minimization strategies
 - The impact of the risk on the project or product will be reduced;
- Contingency plans
 - If the risk arises, contingency plans are plans to deal with that risk;

What-if Questions

- What if several engineers are ill at the same time?
- What if the performance of open-source software is inadequate and the only expert on that open-source software leaves?
- What if the company that supplies and maintains software components goes out of business?
- What if the customer fails to deliver the revised requirements as predicted?

Strategies to Help Manage Risk

Risk	Strategy
Organizational financial problems	Prepare a briefing document for senior management showing how the project is making a very important contribution to the goals of the business and presenting reasons why cuts to the project budget would not be cost-effective.
Recruitment problems	Alert customer to potential difficulties and the possibility of delays; investigate buying-in components.
Staff illness	Reorganize team so that there is more overlap of work and people therefore understand each other's jobs.
Defective components	Replace potentially defective components with bought-in components of known reliability.
Requirements changes	Derive traceability information to assess requirements change impact; maximize information hiding in the design.

Strategies to Help Manage Risk

Risk	Strategy
Organizational restructuring	Prepare a briefing document for senior management showing how the project is making a very important contribution to the goals of the business.
Database performance	Investigate the possibility of buying a higher-performance database.
Underestimated development time	Investigate buying-in components; investigate use of a program generator.

Risk Monitoring

- Assess each identified risks regularly to decide whether it is becoming less or more probable.
- Also assess whether the effects of the risk have changed.
- Each key risk should be discussed at management progress meetings.

Risk Indicators

Risk type	Potential indicators
Estimation	Failure to meet agreed schedule; failure to clear reported defects.
Organizational	Organizational gossip; lack of action by senior management.
People	Poor staff morale; poor relationships amongst team members; high staff turnover.
Requirements	Many requirements change requests; customer complaints.
Technology	Late delivery of hardware or support software; many reported technology problems.
Tools	Reluctance by team members to use tools; complaints about CASE tools; demands for higher-powered workstations.

People Management

- Project managers must choose people for their team and establish ways of working that leads to effective team performance.

Managing People

- People are an organization's most important assets.
- The tasks of a manager are essentially people-oriented. Unless there is some understanding of people, management will be unsuccessful.
- Poor people management is an important contributor to project failure.

People Management Factors

- Consistency
 - Team members should all be treated in a comparable way without favorites or discrimination.
- Respect
 - Different team members have different skills, and these differences should be respected.
- Inclusion
 - Involve all team members and make sure that people's views are considered.
- Honesty
 - You should always be honest about what is going well and what is going badly in a project.

Motivating People

- An important role of a manager is to motivate the people working on a project.
- Motivation means organizing the work and the working environment to encourage people to work effectively.
 - No interest
 - Work slowly
 - More mistakes
- Motivation is a complex issue, but it appears that there are different types of motivation based on:
 - Basic needs (e.g. food, sleep, etc.);
 - Personal needs (e.g. respect, self-esteem);
 - Social needs (e.g. to be accepted as part of a group).

Case Study Individual Motivation - Exercise

Alice is a software project manager working in a company that develops alarm systems. This company wishes to enter the growing market of assistive technology to help elderly and disabled people live independently. Alice has been asked to lead a team of 6 developers than can develop new products based around the company's alarm technology.

Alice's assistive technology project starts well. Good working relationships develop within the team and creative new ideas are developed. The team decides to develop a peer-to-peer messaging system using digital televisions linked to the alarm network for communications. However, some months into the project, Alice notices that Dorothy, a hardware design expert, starts coming into work late, the quality of her work deteriorates and, increasingly, that she does not appear to be communicating with other members of the team.

Alice talks about the problem informally with other team members to try to find out if Dorothy's personal circumstances have changed, and if this might be affecting her work. They don't know of anything, so Alice decides to talk with Dorothy to try to understand the problem.

After some initial denials that there is a problem, Dorothy admits that she has lost interest in the job. She expected that she would be able to develop and use her hardware interfacing skills. However, because of the product direction that has been chosen, she has little opportunity for this. Basically, she is working as a C programmer with other team members.

Although she admits that the work is challenging, she is concerned that she is not developing her interfacing skills. She is worried that finding a job that involves hardware interfacing will be difficult after this project. Because she does not want to upset the team by revealing that she is thinking about the next project, she has decided that it is best to minimize conversation with them.

Case Study Individual Motivation - Exercise

Comments

- If you don't sort out the problem of unacceptable work, the other group members will become dissatisfied and feel that they are doing an unfair share of the work.
- Personal difficulties affect motivation because people can't concentrate on their work.
- They need time and support to resolve these issues, although you have to make clear that they still have a responsibility to their employer.
- Alice gives Dorothy more design autonomy and organizes training courses in software engineering that will give her more opportunities after her current project has finished.

Personality Types

- Motivation should also consider different personality types:
- Task-oriented.
 - The motivation for doing the work is the work itself;
- Self-oriented.
 - The work is a means to an end which is the achievement of individual goals - e.g. to get rich, to play tennis, to travel etc.;
- Interaction-oriented
 - The principal motivation is the presence and actions of co-workers. People go to work because they like to go to work.

Teamwork

- A good group is cohesive and has a team spirit.
 - The people involved are motivated by the success of the group as well as by their own personal goals.
- Group interaction is a key determinant of group performance.
- Flexibility in group composition is limited
- Managers must do the best they can with available people.

Group Cohesiveness

- In a cohesive group, members consider the group to be more important than any individual in it.
- The advantages of a cohesive group are:
 - Group quality standards can be developed by the group members.
 - Team members learn from each other and get to know each other's work; Inhibitions caused by ignorance are reduced.
 - Knowledge is shared. Continuity can be maintained if a group member leaves.
 - Refactoring and continual improvement is encouraged. Group members work collectively to deliver high quality results and fix problems, irrespective of the individuals who originally created the design or program.

The Effectiveness of a Team

- The people in the group
 - You need a mix of people in a project group as software development involves diverse activities such as negotiating with clients, programming, testing and documentation.
- The group organization
 - A group should be organized so that individuals can contribute to the best of their abilities and tasks can be completed as expected.
- Technical and managerial communications
 - Good communications between group members, and between the software engineering team and other project stakeholders, is essential.

Selecting Group Members

- A manager or team leader's job is to create a cohesive group and organize their group so that they can work together effectively.
- This involves creating a group with the right balance of technical skills and personalities, and organizing that group so that the members work together effectively.

Assembling a Team

- May not be possible to appoint the ideal people to work on a project
 - Project budget may not allow for the use of highly-paid staff;
 - Staff with the appropriate experience may not be available;
 - An organization may wish to develop employee skills on a software project.
- Managers must work within these constraints especially when there are shortages of trained staff.

Group Composition

- Group composed of members who share the same motivation can be problematic
 - Task-oriented - everyone wants to do their own thing;
 - Self-oriented - everyone wants to be the boss;
 - Interaction-oriented - too much chatting, not enough work.
- An effective group has a balance of all types.
- This can be difficult to achieve software developers are often task-oriented.
- Interaction-oriented people are very important as they can detect and defuse tensions that arise.

Group Communications

- Good communications are essential for effective group working.
- Information must be exchanged on the status of work, design decisions and changes to previous decisions.
- Good communications also strengthens group cohesion as it promotes understanding.

Group Communications

- Group size
 - The larger the group, the harder it is for people to communicate with other group members.
- Group structure
 - Communication is better in informally structured groups than in hierarchically structured groups.
- Group composition
 - Communication is better when there are different personality types in a group
- The physical work environment
 - Good workplace organization can help encourage communications.

Upcoming

- Midterm next Monday
- Quiz 5 today

COSC 205

Testing



Program Testing

- Testing is intended to show that a program does what it is intended to do and to discover program defects before it is put into use.
- When you test software, you execute a program using artificial data.
- Testing can only show the presence of errors in a program
 - It cannot demonstrate that there are no remaining faults.

Program Testing Goals

- To demonstrate to the developer and the customer that the software meets its requirements.
- To discover situations in which the behavior of the software is incorrect, undesirable or does not conform to its specification.

Unit Testing

- Unit testing is the process of testing individual components of a software application in isolation.
- In unit testing, each unit of code is tested independently of other units
- Units may be:
 - Individual functions or methods within an object
 - Object classes with several attributes and methods
 - Composite components with defined interfaces used to access their functionality.

Development Testing

Development testing includes all testing activities that are carried out by the team developing the system.

- Unit testing
 - where individual program units or object classes are tested.
- Component testing
 - where several individual units are integrated to create composite components.
- System testing
 - where some or all of the components in a system are integrated and the system is tested as a whole
- Acceptance Testing

Automated Testing

- Whenever possible, unit testing should be automated so that tests are run and checked without manual intervention
- In automated unit testing, you make use of a test automation framework (such as JUnit) to write and run your program tests
- Unit testing frameworks provide generic test classes that you extend to create specific test cases
 - They can then run all the tests that you have implemented and report, often through some GUI, on the success or otherwise of the tests

Unit Test Cases

- A unit test case is a specific test scenario designed to verify the behavior of a single unit of code, typically at the function or method level.
- The purpose of a unit test case is to validate that the unit of code behaves as expected under various conditions and inputs.

Automated Test Components

- A setup part, where you initialize the system with the test case, namely the inputs and expected outputs.
- A call/execution part, where you call the object or method to be tested.
- An assertion part where you compare the result of the call with the expected result
 - If the assertion evaluates to true, the test has been successful
 - If false, then it has failed.

Unit Test Cases - Exercise

- For each of the following show an example of how you would initiate the setup, call and assertion phase
 1. Testing a Calculator function, `add()`, that adds two numbers
 2. Testing a method, `get_full_name`, that returns the full name of a person
 3. Testing a function that checks whether a number is even or odd

Unit Test Cases - Exercise

1. Testing a Calculator function, `add()`, that adds two numbers
 - Setup: initialize input values (`x=2`, `y =4`)
 - Execution: call the `add()` function to add `x` and `y`
 - Assertion: verify the result is equal to 6
2. Testing a method, `get_full_name`, that returns the full name of a person
 - Setup: initialize `first_name` and `last_name` variables ("John" and "Doe")
 - Execution: call `get_full_name` with both variables
 - Assertion: verify that return value is equal to "John Doe"
3. Testing a function (`is_even()`) that checks whether a number is even
 - Setup: test for an even number using 4, and test for odd using 5
 - Execution: call `is_even` function with both numbers
 - Assertion: verify that return value on 4 is true and 5 is false

Choosing Unit Test Cases

- The test cases should show that, when used as expected, the component that you are testing does what it is supposed to do.
- If there are defects in the component, these should be revealed by test cases.
- This leads to 2 types of unit test case:
 - The first of these should reflect normal operation of a program and should show that the component works as expected.
 - The other kind of test case should be based on testing experience of where common problems arise.

Testing Strategies

- Partition testing, where you identify groups of inputs that have common characteristics and should be processed in the same way.
 - You should choose tests from within each of these groups.
- Guideline-based testing, where you use testing guidelines to choose test cases.
 - These guidelines reflect previous experience of the kinds of errors that programmers often make when developing components.
 - What are some common errors that are often made?

Partition Testing

- Partition testing (equivalence class partitioning) is a software testing technique used to divide the input data of a software unit into partitions or groups.
- The goal is to reduce the number of test cases needed to cover all possible scenarios while ensuring adequate test coverage.
- The principle behind partition testing is to identify representative values or ranges from the input domain that have similar behavior or characteristics.
- Test cases are then designed to cover each partition, ensuring that at least one test case from each partition is executed during testing.

Steps in Partition Testing

- Identify Input Domain
 - Begin by identifying the input domain of the software unit under test. This includes all possible input values or ranges that the unit can accept.
- Partition Input Domain:
 - Divide the input domain into partitions or equivalence classes based on similar behavior or characteristics.
- Select Test Cases:
 - Choose representative values or ranges from each partition to create test cases. It's important to ensure that the selected test cases cover the entire input domain and are sufficient to detect potential defects.
- Execute Test Cases:
 - Execute the selected test cases against the software unit and observe the results. Verify whether the actual behavior matches the expected behavior for each partition.

Partition Testing - Example

- Function that calculates the grade based on a student's score in an exam
- Input Domain and Partitions:
 - The input domain for the score includes all possible numerical values representing the student's score in the exam.
 - Partition 1: Scores in the range [0, 59] representing failing grades.
 - Partition 2: Scores in the range [60, 69] representing passing grades with a grade of "D."
 - Partition 3: Scores in the range [70, 79] representing passing grades with a grade of "C."
 - Partition 4: Scores in the range [80, 89] representing passing grades with a grade of "B."
 - Partition 5: Scores in the range [90, 100] representing passing grades with a grade of "A."

Partition Testing - Exercise

- Function that validates user input for a sign-up form on a website. The input field we'll focus on is the "Country" field, which accepts country names.
- Identify the input domain, partitions, test cases, and expected results of the test cases

Partition Testing - Exercise

- The input domain for the "Country" field includes all possible country names.
 - Partition 1: Valid country names (e.g., United States, United Kingdom, Canada).
 - Partition 2: Invalid country names (e.g., numbers, special characters).
 - Partition 3: Empty input or null value.
- Test Case 1: Valid Country Name
 - Input: "United States"
 - Expected Outcome: The input is valid, and the sign-up form accepts it without displaying an error message.
- Test Case 2: Invalid Country Name
 - Input: "12345"
 - Expected Outcome: The input is invalid, and the sign-up form displays an error message indicating that only alphabetical characters are allowed for the country name.
- Test Case 3: Empty Input
 - Input: (Empty field)
 - Expected Outcome: The input is invalid, and the sign-up form displays an error message indicating that the country name is required.

Testing Guidelines

- Test software with sequences which have only a single value.
- Use sequences of different sizes in different tests.
- Derive tests so that the first, middle and last elements of the sequence are accessed.
- Test with sequences of zero length.
- Force computation results to be too large or too small.
- Force invalid outputs to be generated

Upcoming

- Lab 6 (released last week but will be discussing JUnit Wednesday)
- Next lecture
 - JUnit
- Quiz Wednesday

COSC 205

JUnit



Unit Tests

- Automatic
- Self-checking
- Run often (almost constantly)

JUnit

- JUnit is a popular unit testing framework for Java, widely used by developers to perform unit testing on their Java code.
- It provides a simple and efficient way to write and execute tests, helping developers ensure the correctness and reliability of their software components.
- Complete user guide <https://junit.org/junit5/docs/current/user-guide/>

Popular Testing Frameworks

- JUnit (Java):
 - JUnit is one of the most widely used testing frameworks for Java.
- NUnit (C#/ .NET):
 - NUnit is a unit-testing framework for .NET languages like C#.
- pytest (Python):
 - pytest is a testing framework for Python that offers simplicity and flexibility.
- RSpec (Ruby):
 - RSpec is a behavior-driven development (BDD) framework for Ruby.
- Mocha (JavaScript):
 - Mocha is a feature-rich testing framework for JavaScript, commonly used for both browser and Node.js environments.
- JUnit (Kotlin):
 - Kotlin developers can also use JUnit for unit testing, similar to Java.
- PHPUnit (PHP):
 - PHPUnit is a unit testing framework for PHP, inspired by JUnit.

JUnit

- Integrated Development Environments such as Eclipse, Visual Studio, NetBeans, IntelliJ have integrated JUnit into their environment and offer first class support for it
- Makes it simple run your test suite automatically
- Every time new code is added, all the test cases have to be re-executed

JUnit Setup

- Start by installing your preferred IDE and ensure you have the Java SE Development Kit installed as well (JDK)
- Download and install JAR files available via JUnit website:
<https://junit.org/junit4/>

JUnit Setup

Click download and install

JUnit 4 ▾ Project Documentation ▾



JUnit 4 / About

Version: 4.13.2 | Last Published: 2021-02-13

JUnit is a simple framework to write repeatable tests. It is an instance of the xUnit architecture for unit testing frameworks.

```
@Test
public void newArrayListsHaveNoElements() {
    assertThat(new ArrayList<Integer>().size(), is(0));
}

@Test
public void sizeReturnsNumberOfElements() {
    List<Object> instance = new ArrayList<Object>();
    instance.add(new Object());
    instance.add(new Object());
    assertEquals(instance.size(), 2);
}
```

Annotations
Start by marking your tests with `@Test`

Let's take a tour »

Welcome

- Download and install
- Getting started
- Release Notes
 - 4.13.2
 - 4.13.1
 - 4.13
 - 4.12
 - 4.11
 - 4.10
 - 4.9.1
 - 4.9
- Maintainer Documentation

Usage and Idioms

- Assertions
- Test Runners
- Aggregating tests in Suites
- Test Execution Order
- Exception Testing
- Matchers and `assertThat`
- Ignoring Tests
- Timeout for Tests
- Parameterized Tests
- Assumptions with `Assume`
- Rules
- Theories

Third-party extensions

- Custom Runners
- `net.trajano.commons.common-testing` for `UtilityClassTestUtil` per #646
- System Rules – A collection of JUnit rules for testing code that uses `java.lang.System`.
- JUnit Toolbox – Provides runners for parallel testing, a `PoolingRule` class to ease asynchronous testing, and a `WildcardPatternSuite` which allow you to specify wildcard patterns instead of explicitly listing all classes when you create a suite class.

../
[junit-4.13.2-javadoc.jar](#)
[junit-4.13.2-javadoc.jar.asc](#)
[junit-4.13.2-javadoc.jar.asc.md5](#)
[junit-4.13.2-javadoc.jar.asc.sha1](#)
[junit-4.13.2-javadoc.jar.md5](#)
[junit-4.13.2-javadoc.jar.sha1](#)
[junit-4.13.2-sources.jar](#)
[junit-4.13.2-sources.jar.asc](#)
[junit-4.13.2-sources.jar.asc.md5](#)
[junit-4.13.2-sources.jar.asc.sha1](#)
[junit-4.13.2-sources.jar.md5](#)
[junit-4.13.2-sources.jar.sha1](#)
[junit-4.13.2.jar](#)
[junit-4.13.2.jar.asc](#)
[junit-4.13.2.jar.asc.md5](#)
[junit-4.13.2.jar.asc.sha1](#)
[junit-4.13.2.jar.md5](#)
[junit-4.13.2.jar.sha1](#)
[junit-4.13.2.pom](#)
[junit-4.13.2.pom.asc](#)
[junit-4.13.2.pom.asc.md5](#)
[junit-4.13.2.pom.asc.sha1](#)
[junit-4.13.2.pom.md5](#)
[junit-4.13.2.pom.sha1](#)

Then click on the junit-4.13.2.jar and hamcrest-core-1.3.jar to download

../
[hamcrest-core-1.3-javadoc.jar](#)
[hamcrest-core-1.3-javadoc.jar.asc](#)
[hamcrest-core-1.3-javadoc.jar.asc.md5](#)
[hamcrest-core-1.3-javadoc.jar.asc.sha1](#)
[hamcrest-core-1.3-javadoc.jar.md5](#)
[hamcrest-core-1.3-javadoc.jar.sha1](#)
[hamcrest-core-1.3-sources.jar](#)
[hamcrest-core-1.3-sources.jar.asc](#)
[hamcrest-core-1.3-sources.jar.asc.md5](#)
[hamcrest-core-1.3-sources.jar.asc.sha1](#)
[hamcrest-core-1.3-sources.jar.md5](#)
[hamcrest-core-1.3-sources.jar.sha1](#)
[hamcrest-core-1.3.jar](#)
[hamcrest-core-1.3.jar.asc](#)
[hamcrest-core-1.3.jar.asc.md5](#)
[hamcrest-core-1.3.jar.asc.sha1](#)
[hamcrest-core-1.3.jar.md5](#)
[hamcrest-core-1.3.jar.sha1](#)
[hamcrest-core-1.3.pom](#)
[hamcrest-core-1.3.pom.asc](#)
[hamcrest-core-1.3.pom.asc.md5](#)
[hamcrest-core-1.3.pom.asc.sha1](#)
[hamcrest-core-1.3.pom.md5](#)
[hamcrest-core-1.3.pom.sha1](#)

On the GitHub page click on both links

Download and Install

Marc Philipp edited this page on Sep 29, 2023 · 35 revisions

To download and install JUnit you currently have the following options.

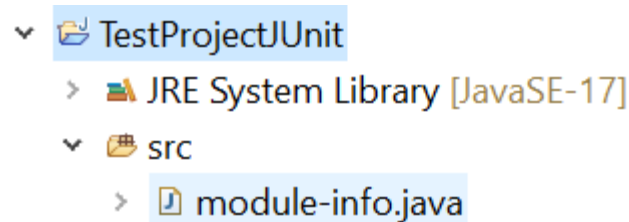
Plain-old JAR

Download the following JARs and add them to your test classpath:

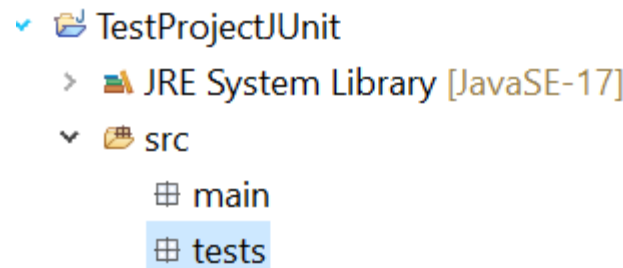
- `junit.jar`
- `hamcrest-core.jar`

JUnit Setup - Example

- Navigate to your IDE and create a new Java project, provide name and create the project
- After creating check in your src folder if there is a module-info.java file. If so, delete this file as it will cause us issues

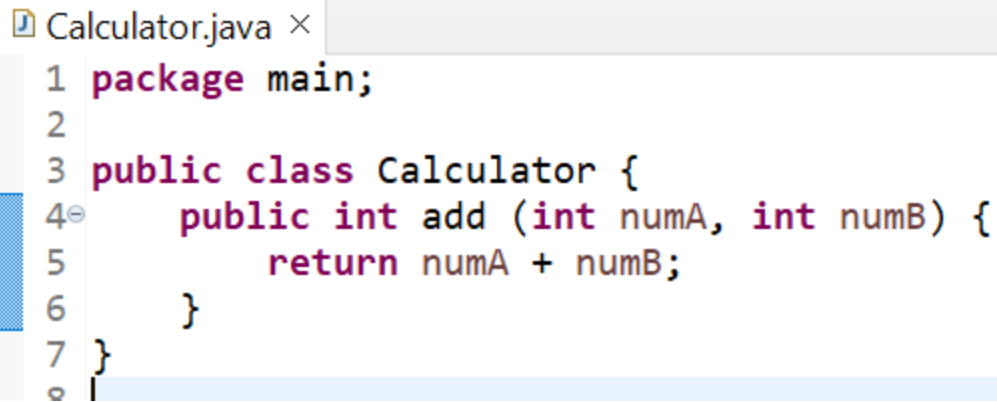


- Create 2 packages
 - One called 'main' to hold all our functions, classes etc..
 - One called 'tests' to hold all our test classes



JUnit Setup - Example

- Inside your 'main' package create a new java class and name it calculator with the following function inside it:

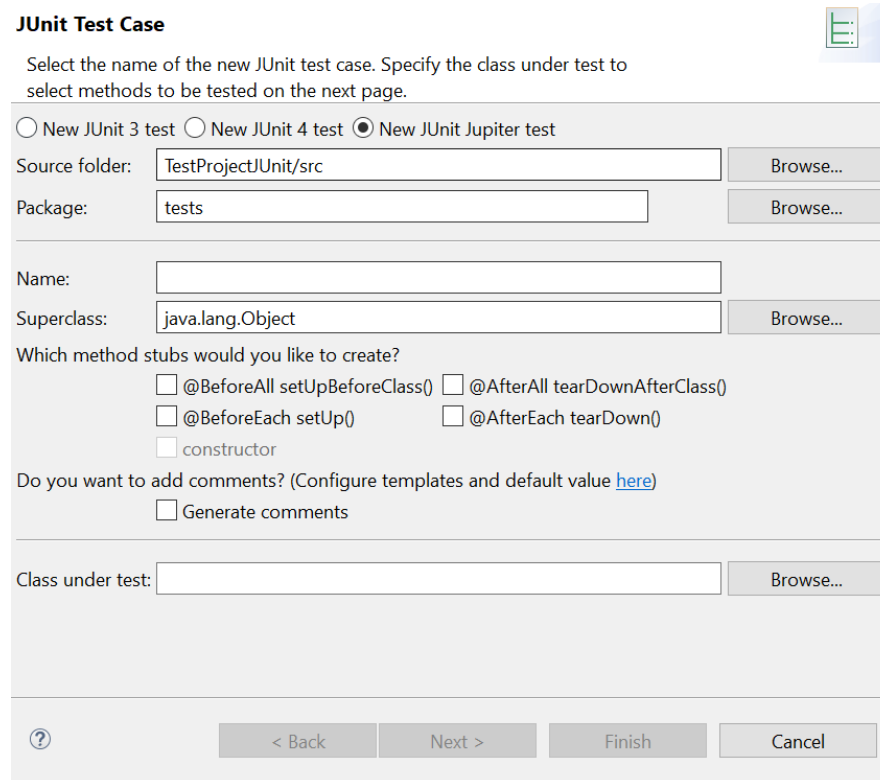


```
Calculator.java ×
1 package main;
2
3 public class Calculator {
4     public int add (int numA, int numB) {
5         return numA + numB;
6     }
7 }
8 |
```

- This will be the function/method we are aiming to test

JUnit Setup - Example

- Now, right click on your 'tests' package and click new → JUnit Test Case. You should now see this popup.



The image shows a 'JUnit Test Case' dialog box. At the top, it says 'Select the name of the new JUnit test case. Specify the class under test to select methods to be tested on the next page.' Below this, there are three radio buttons: 'New JUnit 3 test', 'New JUnit 4 test', and 'New JUnit Jupiter test'. The 'New JUnit Jupiter test' option is selected. There are four 'Browse...' buttons: one for 'Source folder' (containing 'TestProjectJUnit/src'), one for 'Package' (containing 'tests'), one for 'Superclass' (containing 'java.lang.Object'), and one for 'Class under test'. The 'Name' field is empty. Under 'Which method stubs would you like to create?', there are checkboxes for '@BeforeAll setUpBeforeClass()', '@AfterAll tearDownAfterClass()', '@BeforeEach setUp()', '@AfterEach tearDown()', and 'constructor'. The 'Do you want to add comments?' section has a checkbox for 'Generate comments'. At the bottom, there is a '?' icon, '< Back' button, 'Next >' button, 'Finish' button, and 'Cancel' button.

JUnit Test Case

Select the name of the new JUnit test case. Specify the class under test to select methods to be tested on the next page.

☐ New JUnit 3 test ☐ New JUnit 4 test ☒ New JUnit Jupiter test

Source folder:

Package:

Name:

Superclass:

Which method stubs would you like to create?

☐ @BeforeAll setUpBeforeClass() ☐ @AfterAll tearDownAfterClass()
☐ @BeforeEach setUp() ☐ @AfterEach tearDown()
☐ constructor

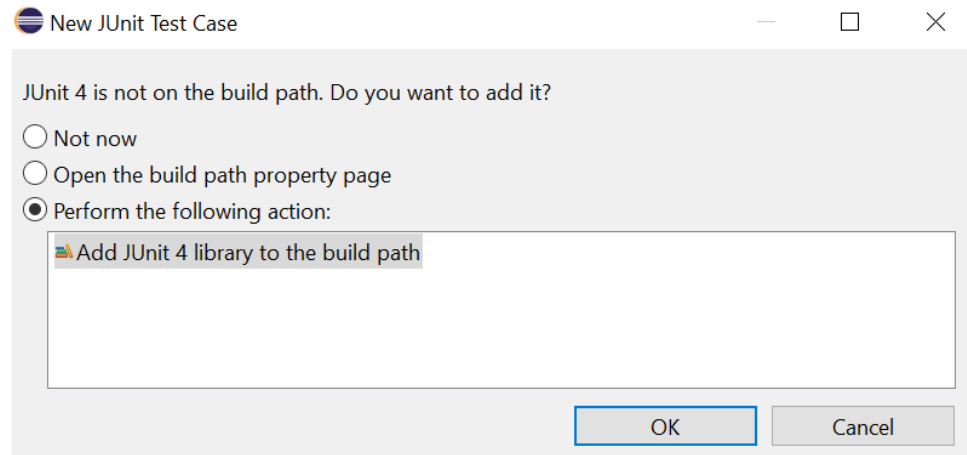
Do you want to add comments? (Configure templates and default value [here](#))
☐ Generate comments

Class under test:

- At the top click 'New Junit 4 Test' instead of 'Jupiter Test'
- Provide your test case a name (CalculatorTest) and you can leave the rest as is

JUnit Setup - Example

- Click 'Finish' and you will get another popup asking if you want to include JUnit4 in the build path. Click Ok



- Now you should have a 'CalculatorTest' class that looks as such:

```
package tests;

import static org.junit.Assert.*;

public class CalculatorTest {

    @Test
    public void test() {
        fail("Not yet implemented");
    }

}
```


JUnit Assertions

- Recall to create an automated test, we need to have three components:
 - A setup part, where you initialize the system with the test case, namely the inputs and expected outputs.
 - A call/execution part, where you call the object or method to be tested.
 - An assertion part where you compare the result of the call with the expected result. If the assertion evaluates to true, the test has been successful if false, then it has failed.

JUnit Assertions

- JUnit provides a set of assertion methods to verify expected outcomes in test methods. Some common assertion methods include:
- `assertEquals(expected, actual)`: Asserts that two objects are equal.
- `assertNotEquals(expected, actual)`: Asserts that two objects are not equal.
- `assertTrue(condition)`: Asserts that a condition is true.
- `assertFalse(condition)`: Asserts that a condition is false.
- `assertNotNull(object)`: Asserts that an object is not null.
- `assertNull(object)`: Asserts that an object is null.
- And many more...

JUnit - Example

- Let's continue to build on our test class from before
- When creating the name of our test method we want to be very descriptive with the exact scenario being tested
- Let's test if two plus two equals four for other method
 - Name the method: `public void twoPlusTwoEqualsFour()`
- Inside the method let's create a Calculator object to be able to call our method on: `Calculator calc = new Calculator();`
- Now, call our add method on this calculator object with the appropriate parameters: `int val = calc.add(2, 2);`
- Now, we want to verify that the result returned is 4
- So, we can use our assertEquals: `assertEquals(4, val);`

expected   actual

JUnit - Example

Full code block:

```
public class CalculatorTest {
```

```
@Test
```

```
public void twoPlusTwoEqualsFour() {
```

```
Calculator calc = new Calculator();
```

```
int val = calc.add(2, 2);
```

```
assertEquals(4, val);
```

```
}
```

```
}
```

```
public class CalculatorTest {
```

```
@Test
```

```
public void twoPlusTwoEqualsFour() {
```

```
Calculator calc = new Calculator();
```

```
int val = calc.add(numA: 2, numB: 2);
```

```
assertEquals(4, val);
```

```
}
```

JUnit Running Tests

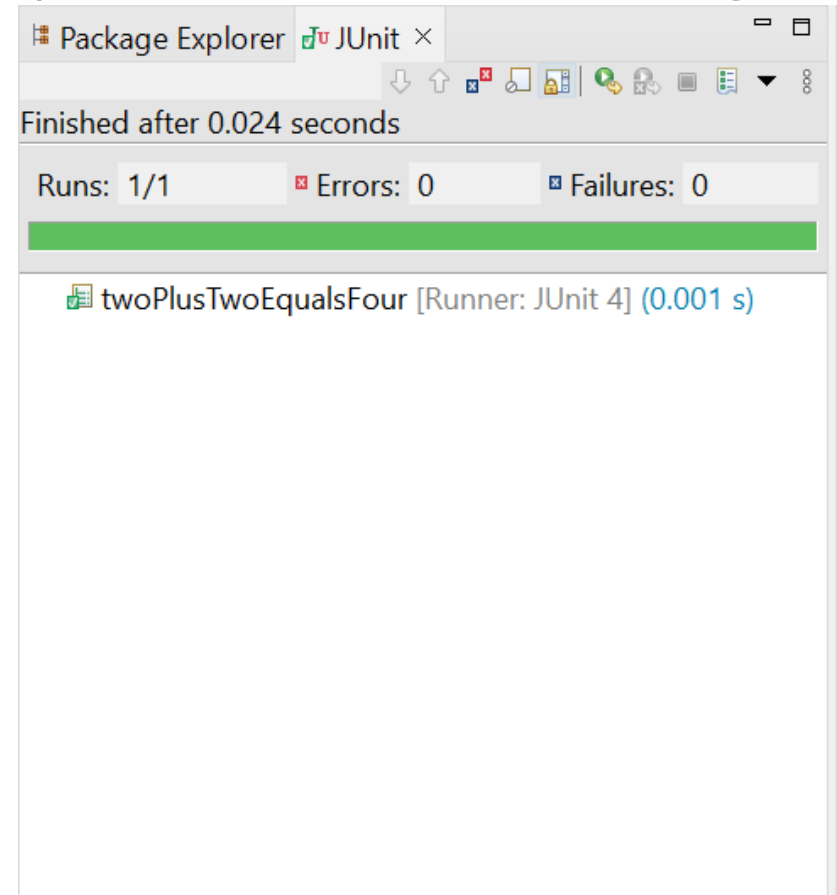
- IDEs like Eclipse, IntelliJ IDEA, and NetBeans provide built-in support for running JUnit tests.
- **Running tests** typically involves executing the test methods defined within your test classes to ensure that your code behaves as expected.
- **Running tests with coverage** provides insights into how much of your code is exercised by your tests.
 - Coverage tools track which parts of your code are executed during the test run, helping identify areas with low test coverage that may require additional testing

JUnit - Example

- Couple of different ways to run tests:
 1. Hover over the test method, right click and click “Run as JUnit Test”
 - If there are multiple test methods within the class, this will only run the one test method
 2. Click the “Run” button in the top header which will run all tests within the class

JUnit - Example

- After running a test, we will be prompted with a nice GUI showing us if the test has passed or failed

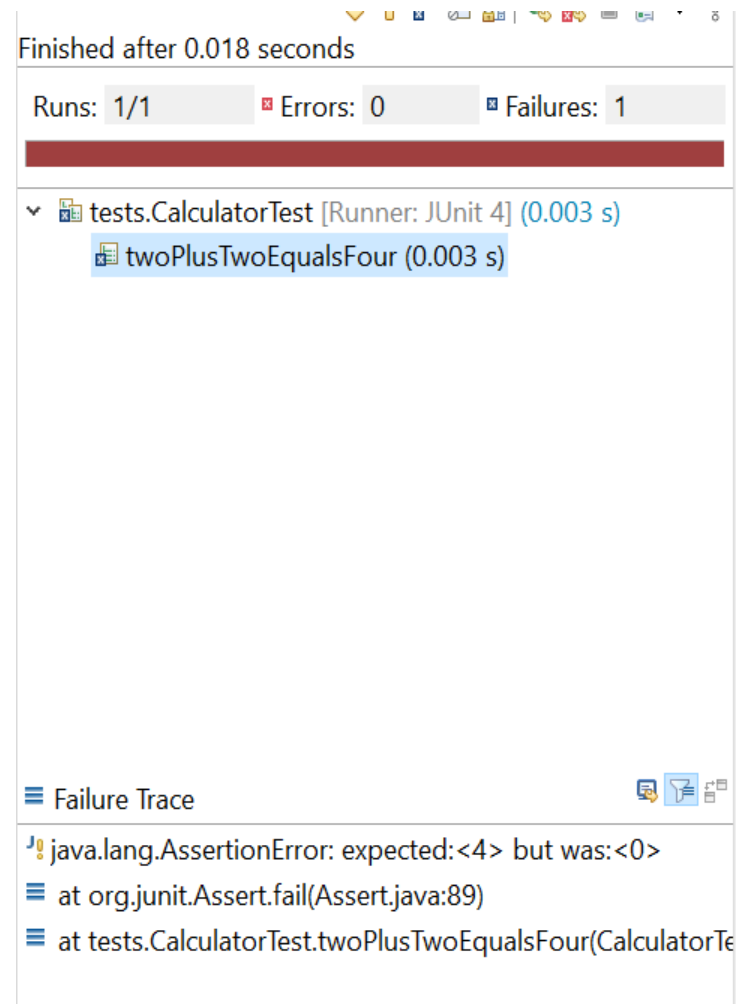


- Passes denoted with a checkmark and fails denoted with a x

JUnit - Example

- If a test fails, JUnit will tell us why it failed
- Ex: say we changed our code to subtract

The test failed because the expected value was 4 but we got 0 as the result



- Hence, our test is doing the right thing because it should be failing if our code is doing the wrong thing

JUnit - Example

- It is crucial though that we build multiple tests out for our methods
- Let's say we change our function to multiply instead of add
- $2*2 = 2+2$ so our test will pass
- However, our code is doing the wrong thing
- One of the goals of unit tests is that whenever the code is not doing the right thing, at least one of our unit tests should fail
- So, it doesn't mean our initial unit test is bad, but we should add more

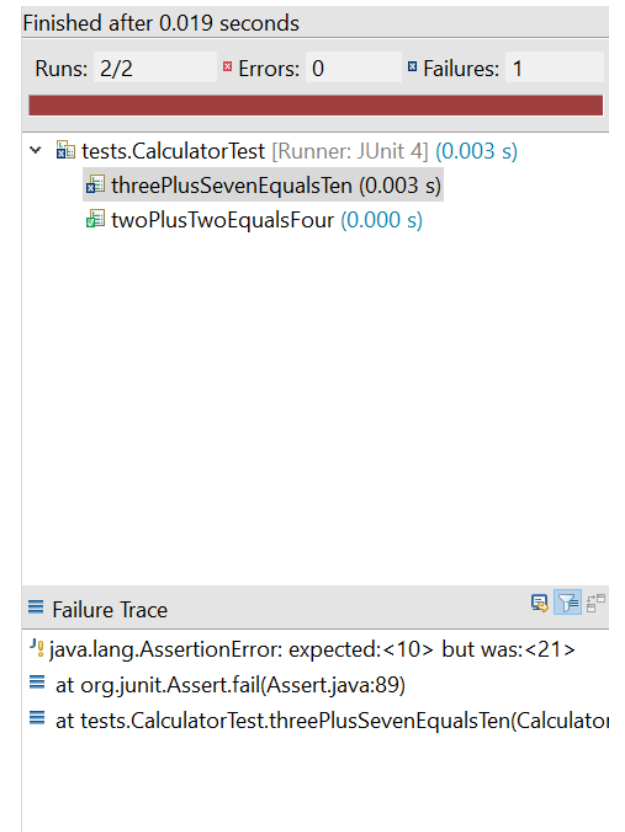
JUnit - Example

- Let's create another unit test:

@Test

```
public void threePlusSevenEqualsTen() {  
    Calculator calc = new Calculator();  
    int val = calc.add(3, 7);  
    assertEquals(10, val);}
```

- Rerun all unit tests:
- We can see that one passed and one failed indicating to us that something is wrong within our code



JUnit Exercise

- In groups or individually build out the following:
- A max method that takes in three numbers and returns the max of the three numbers (Don't use the Math.max function)
- Create at least 3 unique test cases to test your method

JUnit Exercise

```
public class Max {  
    public int max(int num1, int num2,  
        int num3) {  
        int max = num1;  
        if(max < num2) {  
            max = num2;  
        }  
        if(max < num3 ) {  
            max = num3;  
        }  
        return max;  
    }  
}
```

```
public class MaxTest {
```

```
    @Test
```

```
    public void maxThreeFourFiveIsFive() {  
        Max max = new Max();  
        assertEquals(5,max.max(3, 4, 5));  
    }
```

```
    @Test
```

```
    public void maxFiveFourThreeIsFive() {  
        Max max = new Max();  
        assertEquals(5,max.max(5, 4, 3));  
    }
```

```
    @Test
```

```
    public void
```

```
    maxNegativesAndPositivesIsPositive() {  
        Max max = new Max();  
        assertEquals(3,max.max(-5, -4, 3));  
    }}
```

JUnit Code Coverage

- **Running tests with coverage** provides insights into how much of your code is exercised by your tests.
 - Coverage tools track which parts of your code are executed during the test run, helping identify areas with low test coverage that may require additional testing

JUnit Example

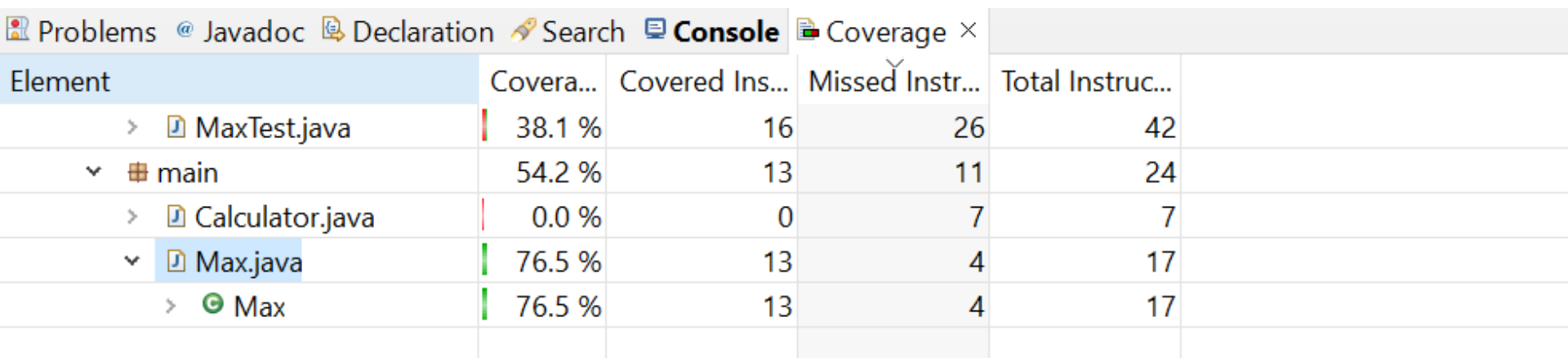
- Say for our max function that we created we gave the inputs:
num1 = 5, num2 = 4, num3 = 3, in our test

```
public int max(int num1, int num2, int num3) {  
    int max = num1;  
    if(max < num2) {  
        max = num2;  
    }  
    if(max < num3 ) {  
        max = num3;  
    }  
    return max;  
}
```

- Which if statements would be used?

JUnit Example Code Coverage

- We can check this by running our tests with 'coverage'
- Right click the test, click 'Coverage As' then 'JUnit Test'
- Eclipse will show us exactly which lines of code were executed when our tests were run



Element	Covera...	Covered Ins...	Missed Instr...	Total Instruc...
> [J] MaxTest.java	38.1 %	16	26	42
▼ [J] main	54.2 %	13	11	24
> [J] Calculator.java	0.0 %	0	7	7
▼ [J] Max.java	76.5 %	13	4	17
> [G] Max	76.5 %	13	4	17

- Here we can see a report showing us that in our Max class we had 13 covered instructions with that one test and 4 missing instructions

JUnit Example Code Coverage

- We can also navigate to the Max class and Eclipse will automatically highlight in Red, Yellow, Green depending on whether the unit test we ran, hit the line or not

```
package main;

public class Max {
    public int max(int num1, int num2, int num3) {
        int max = num1;
        if(max < num2) {
            max = num2;
        }
        if(max < num3 ) {
            max = num3;
        }
        return max;
    }
}
```

- One of the goals of generating unit tests should be to reach 100% of the lines inside of the method

JUnit Testing for Exceptions Thrown

- Say that we want an exception to be thrown if the user provides a negative number

- Ex:

```
if(num1 < 0) {  
    throw new IllegalArgumentException("Number grade cannot be  
    negative");  
}
```

- Let's create a test case for this

Junit Testing for Exceptions Thrown

```
@Test
```

```
public void negativeOneShouldReturnException() {  
    Max max = new Max();  
    max.max(-1, 3, 5);  
}
```

- If we run this as is, the test will fail because if the code you hit throws an exception the test will automatically fail
- We can use an assert statement that will assert if an exception is thrown: `assertThrows` (takes 2 parameters)

```
assertThrows(IllegalArgumentException.class,  
    () -> {  
        max.max(-1, 3, 5);  
    });  
}
```

Parameter 2: The
code that executes
the test

Parameter 1:
Name of the class
of the exception
you expect to be
thrown

JUnit Annotations

- JUnit utilizes annotations to define and configure test methods and test classes.

Some common annotations include:

- '@Test': Marks a method as a test method.
- '@Before'
- '@After'
- '@BeforeClass'
- '@AfterClass'

JUnit Annotations

- @Before:
 - Methods annotated with @Before are executed before each test method in the test class.
 - Use @Before to set up preconditions or initialize resources required for the test methods.
 - Example:

```
public class MyTest {  
    private MyObject obj;  
    @Before  
    public void setUp() {  
        obj = new MyObject();  
    }  
}
```

JUnit Annotations

- @After:
- Methods annotated with @After are executed after each test method in the test class.
- Use @After to perform cleanup actions or release resources acquired during test execution.
- Example:

```
public class MyTest {  
    private MyObject obj;  
    @Before  
    public void setUp() {  
        obj = new MyObject();  
    }  
    @After  
    public void tearDown() {  
        obj = null;}}
```

JUnit Annotations

- @BeforeClass:
- Methods annotated with @BeforeClass are executed once before any test method in the test class.
- Use @BeforeClass for one-time setup tasks that are shared across all test methods in the class.
- Example:

```
public class MyTest {  
    @BeforeClass  
    public static void setUpClass() {  
        // One-time setup logic here  
    }  
}
```

JUnit Annotations

- @AfterClass:
- Methods annotated with @AfterClass are executed once after all test methods in the test class.
- Use @AfterClass for one-time cleanup tasks or releasing resources shared across all test methods.
- Example:

```
public class MyTest {  
    @AfterClass  
    public static void tearDownClass() {  
        // One-time cleanup logic here  
    }  
}
```

Good Unit Test Cases

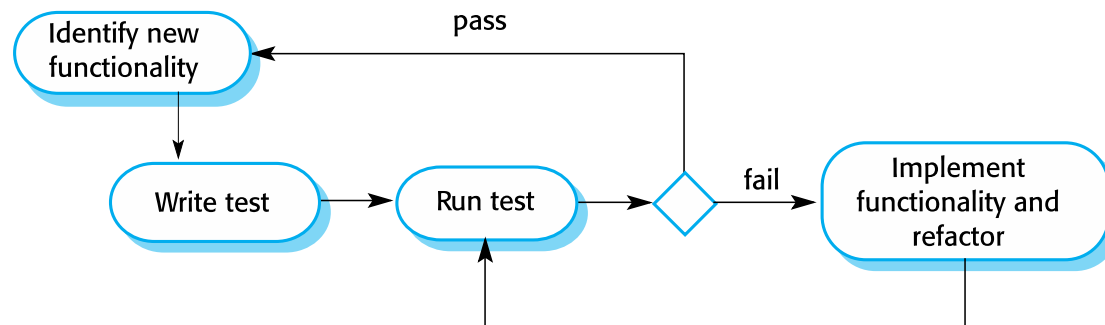
- Independence: Each unit test case should be independent of other test cases, meaning that the outcome of one test case should not affect the outcome of another.
- Readability: Unit test cases should be clear, concise, and easy to understand. Use descriptive names for test methods and meaningful assertions to clearly communicate the expected behavior.
- Coverage: Aim to achieve sufficient test coverage by writing test cases that exercise all relevant paths through the code, including normal exceptional, and edge cases.
 - Unit Tests should aim to cover, an edge case, normal input, guaranteed failure
- Maintainability: Unit tests should be easy to maintain and update as the codebase evolves.

Benefits of Unit Testing

- Early Bug Detection
 - Unit testing helps identify defects in the codebase early in the development process.
- Faster Debugging
 - When a unit test fails, it provides immediate feedback about the specific unit of code that is not functioning correctly.
- Facilitates Refactoring
 - Unit tests and test suite serve as a safety net when refactoring code.
- Documentation and Specification
 - Unit tests serve as executable documentation for the codebase, providing clear examples of how individual units of code are intended to behave.
- Can be sure code is correct instead of believing it is correct

Test-driven Development Process Activities

- Start by identifying the increment of functionality that is required. This should normally be small and implementable in a few lines of code.
- Write a test for this functionality and implement this as an automated test.
- Run the test, along with all other tests that have been implemented. Initially, you have not implemented the functionality so the new test will fail.
- Implement the functionality and re-run the test.
- Once all tests run successfully, you move on to implementing the next chunk of functionality.



Upcoming

- Lab 6 due Friday

References

- <https://junit.org/junit5/docs/current/user-guide/>

COSC 205

Configuration Management



Configuration Management

- Software systems are constantly changing during development and use.
- Configuration management (CM) is concerned with the policies, processes and tools for managing changing software systems.
- You need CM because it is easy to lose track of what changes and component versions have been incorporated into each system version.
- CM is essential for team projects to control changes made by different developers

Version

- An instance of a configuration item that differs, in some way, from other instances of that item
- Each version represents a specific snapshot of the software's state at a particular point in time, including its features, functionality, and configuration
- Versions always have a unique identifier

CM Activities

- **Version management**

- Keeping track of the multiple versions of system components and ensuring that changes made to components by different developers do not interfere with each other.

CM Activities

- **System building**

- The process of assembling program components, data and libraries, then compiling these to create an executable system.

CM Activities

- **Change management**

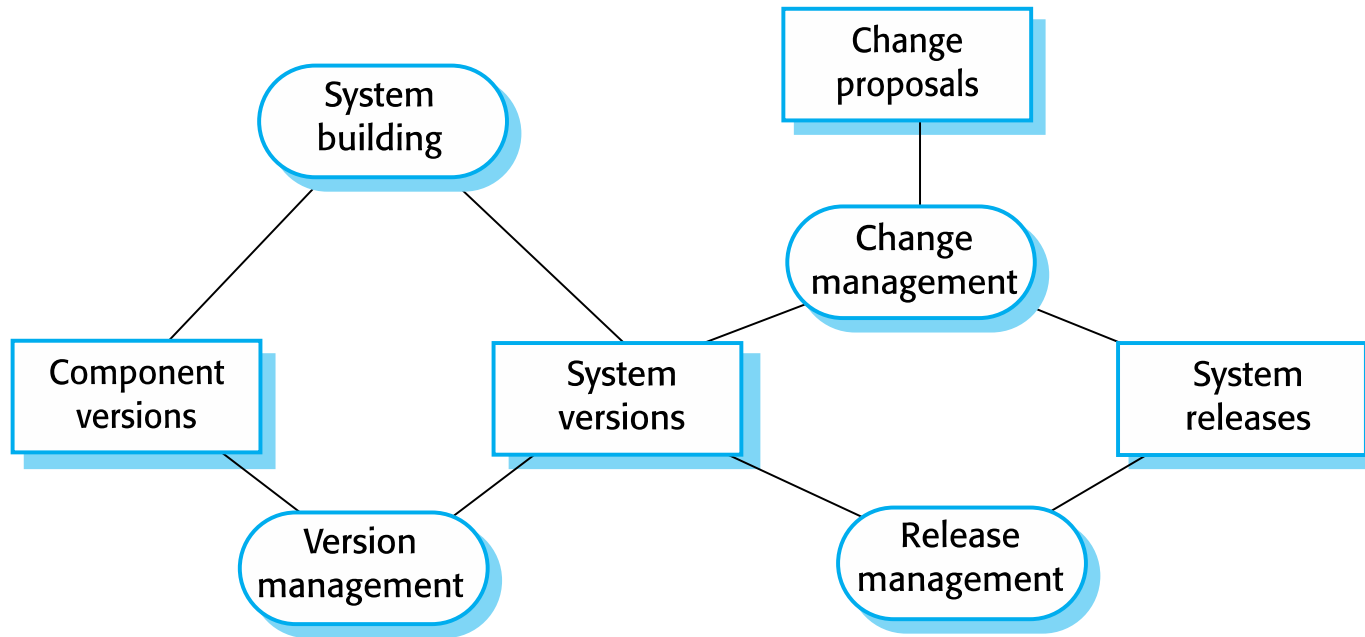
- Keeping track of requests for changes to the software from customers and developers, working out the costs and impact of changes, and deciding the changes should be implemented.

CM Activities

- **Release management**

- Preparing software for external release and keeping track of the system versions that have been released for customer use.

CM Activities



Agile Development and CM

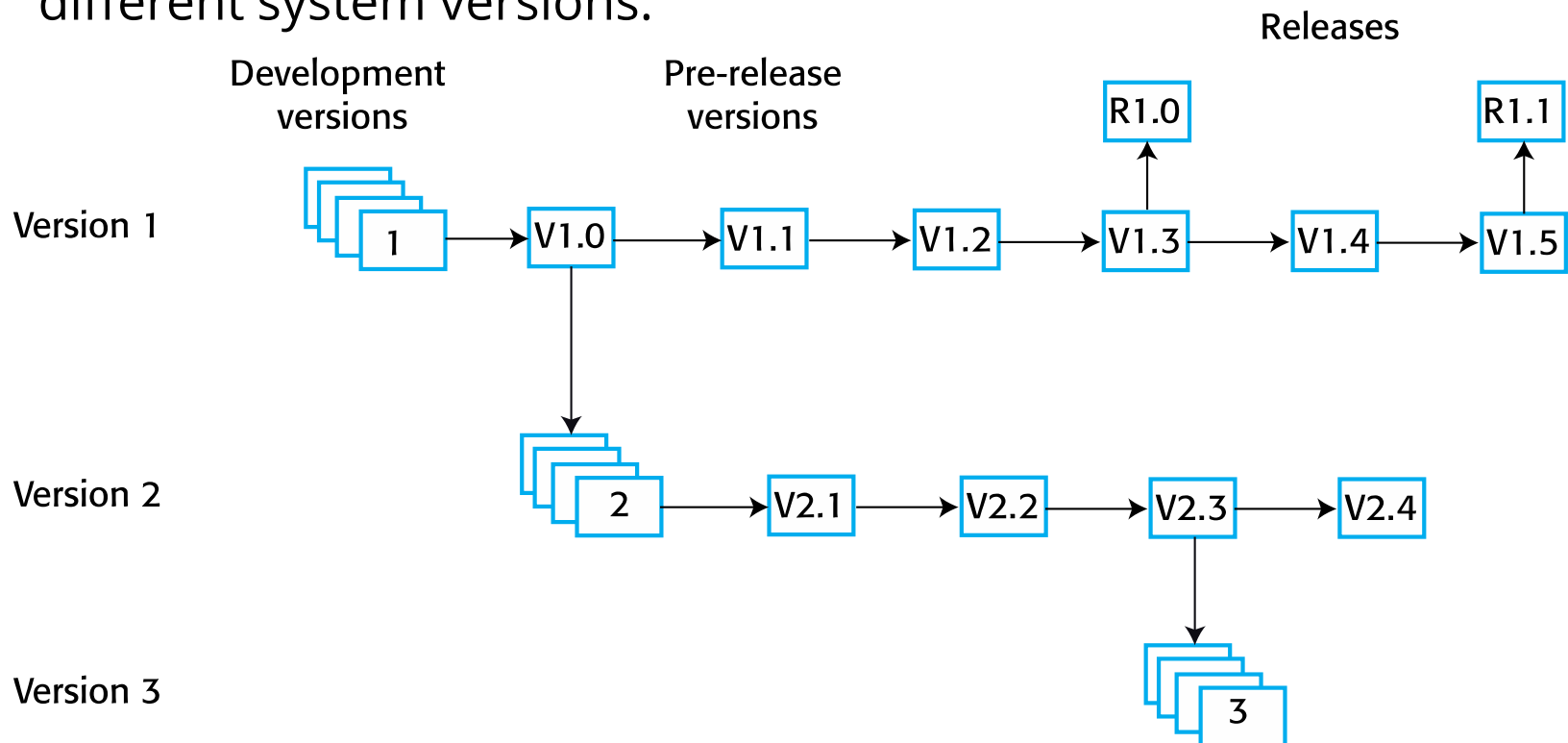
- Agile development, where components and systems are changed several times per day, is impossible without using CM tools.
- The definitive versions of components are held in a shared project repository and developers copy these into their own workspace.
 - I.e., GitHub
- They make changes to the code then use system building tools to create a new system on their own computer for testing.
 - Once they are happy with the changes made, they return the modified components to the project repository.

Development Phases

- A development phase where the development team is responsible for managing the software configuration and new functionality is being added to the software.
- A system testing phase where a version of the system is released internally for testing.
 - No new system functionality is added. Changes made are bug fixes, performance improvements and security vulnerability repairs.
- A release phase where the software is released to customers for use.
 - New versions of the released system are developed to repair bugs and vulnerabilities and to include new features.

Multi-version Systems

- For large systems, there is never just one 'working' version of a system.
- There are always several versions of the system at different stages of development.
- There may be several teams involved in the development of different system versions.



Version Management

- Version management (VM) is the process of keeping track of different versions of software components or configuration items and the systems in which these components are used.
- It also involves ensuring that changes made by different developers to these versions do not interfere with each other.
- Therefore, version management can be thought of as the process of managing codelines and baselines.

Configuration Item

- Configuration item or software configuration item (SCI)
 - Anything associated with a software project (design, code, test data, document, etc.) that has been placed under configuration control.
 - There are often different versions of a configuration item.
 - Configuration items have a unique name.

Codelines

- Distinct lines of development within a software project
- Each codeline represents a specific path of changes to the project's source code and related artifacts
 - Revisions to a configuration item
- A codeline is a sequence of versions of source code with later versions in the sequence derived from earlier versions.

Baseline

- Baselines represent a snapshot of the project's configuration at a specific point in time.
- The baseline therefore specifies **the component versions** that are included in the system plus a specification of the libraries used, configuration files, etc.
- Baselines serve as reference points for comparing and tracking changes throughout the project lifecycle.

Baselines

- Baselines may be specified using a configuration language, which allows you to define what components are included in a version of a particular system.
- Baselines are important because you often must recreate a specific version of a complete system.
 - For example, a product line may be instantiated so that there are individual system versions for different customers. You may have to recreate the version delivered to a specific customer if, for example, that customer reports bugs in their system that must be repaired.

Baselines and Codelines

- What are some examples of codelines and baselines in your project or that you can think of?

Baselines and Codelines

Codelines

- **Mainline:** The mainline codeline (ex: master branch) represents the primary development path
- **Feature Branches:** Codelines created to implement specific features or functionalities independently of the mainline.
 - Developers work on feature branches to isolate changes related to a particular feature, allowing for parallel development without interfering with the stability of the mainline.
- **Release Branches:** Release branches are created to prepare and stabilize the codebase for a new release.
- **Hotfix Branches:** Hotfix branches are created to address critical issues or bugs in a deployed version of the software.

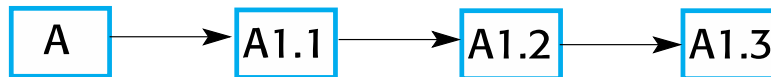
Baselines and Codelines

Baselines

- **Functional Baseline:** Captures the configuration of the project at a significant milestone, such as the completion of a major feature or module.
 - It serves as a reference for verifying the functionality and correctness of the software at specific points in development.
- **Configuration Baseline:** A configuration baseline captures the configuration of the project's artifacts, including source code, libraries, and configuration files. It provides a stable foundation for development, testing, and deployment activities, ensuring consistency across environments.
- **Release Baseline:** A release baseline represents the configuration of the project at the time of a software release.

Codelines and Baselines

Codeline (A)



Codeline (B)



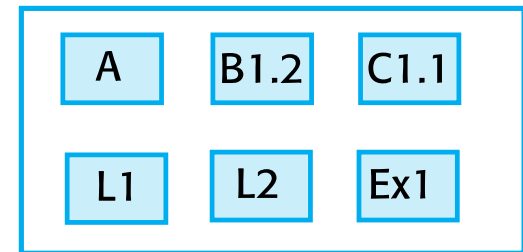
Codeline (C)



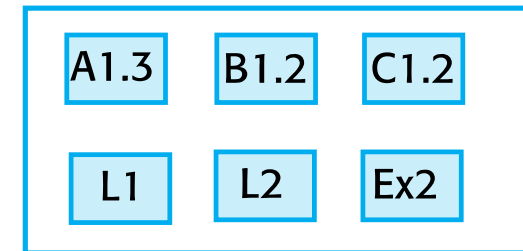
Libraries and external components



Baseline - V1



Baseline - V2

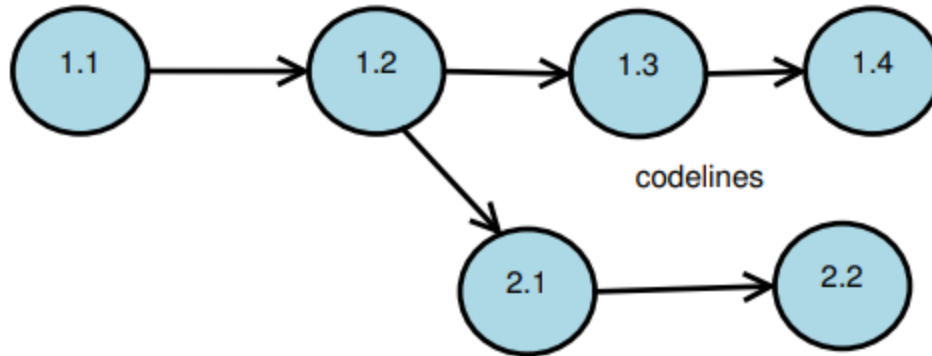


Mainline

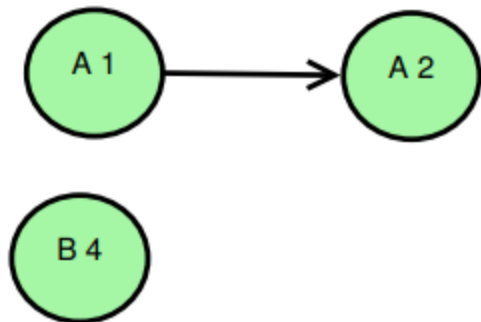
Codelines and Baselines - Exercise

- A company has 2 codelines for their system
 - Codeline 1, starting at version 1.1 and ending at 1.4
 - Codeline 2, which is built off version 1.2 and ends at 2.2
- There are 3 external libraries (A1, A2, B4)
 - A2 is built off of A1
- There are 2 different baselines/releases for their system
 - Windows release 2 uses the final component of Codeline 1, library B4, and the first library of A in their version of the system
 - Linux release 3 uses the final component of Codeline 2, and library A2 in their version of the system
- Model how these codelines, libraries and baselines would look

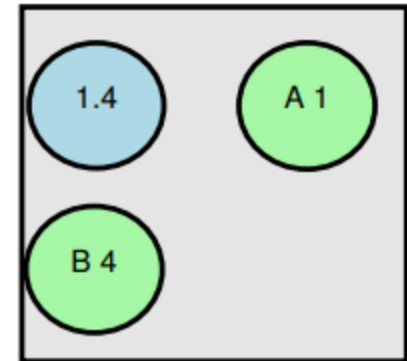
Codelines and Baselines - Exercise



External Libraries and components



Baseline: Windows Release 2



Baseline: Linux Release 3

- Reference:

<https://www.cs.odu.edu/~zeil/cs795SD/s13/Public/scm/scm-10-16x10.pdf>

Version Control Systems

- Version control (VC) systems identify, store and control access to the different versions of components. There are two types of modern version control system
 - **Centralized systems**, where there is a single master repository that maintains all versions of the software components that are being developed. Subversion is a widely used example of a centralized VC system.
 - **Distributed systems**, where multiple versions of the component repository exist at the same time. Git is a widely-used example of a distributed VC system.

Key Features of Version Control Systems

- Version and release identification
- Change history recording
- Support for independent development
- Project support
- Storage management

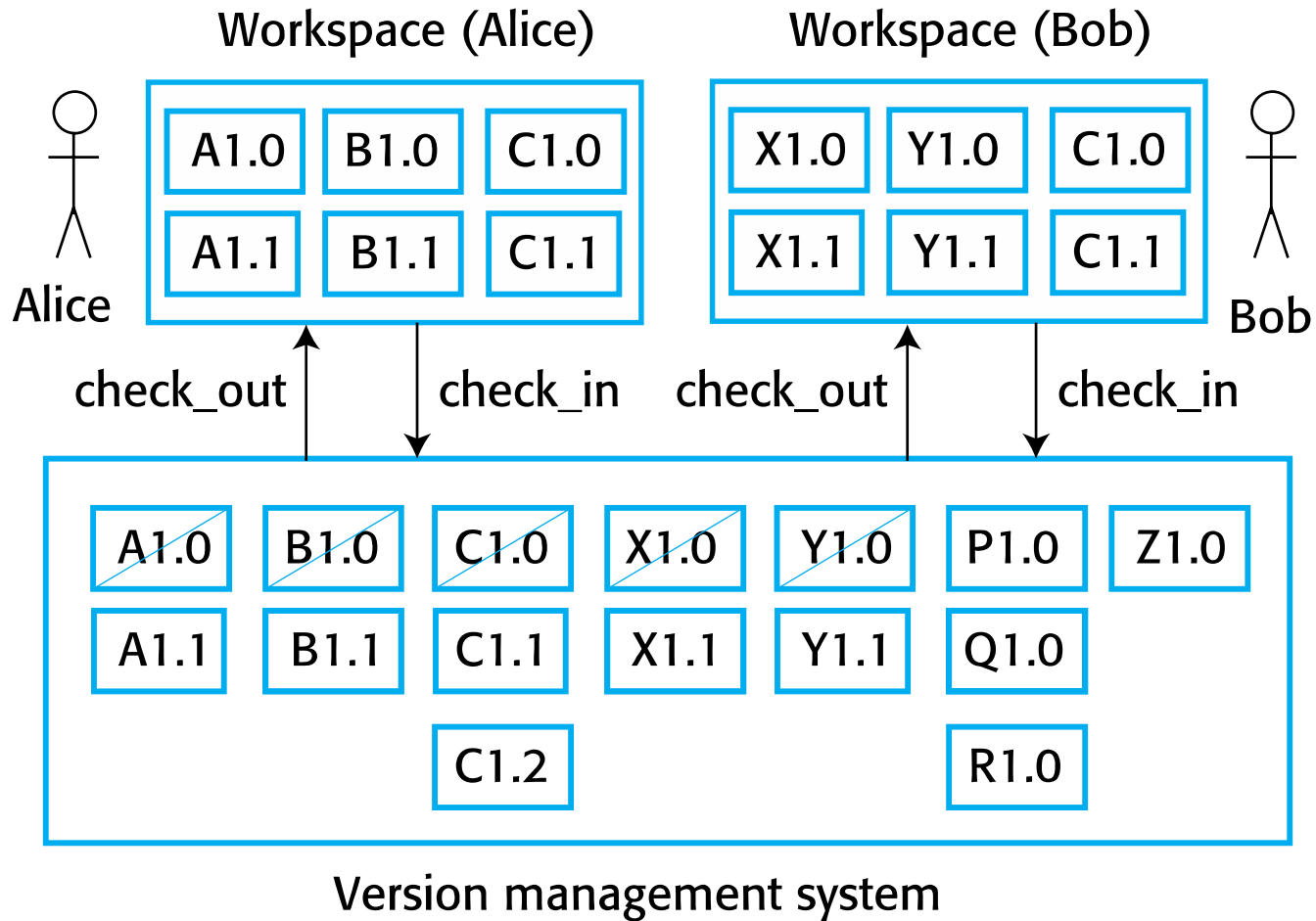
Public Repository and Private Workspaces

- To support independent development without interference, version control systems use the concept of a project repository and a private workspace.
- The project repository maintains the 'master' version of all components. It is used to create baselines for system building.
- When modifying components, developers copy (check-out) these from the repository into their workspace and work on these copies.
- When they have finished their changes, the changed components are returned (checked-in) to the repository.

Centralized Version Control

- Developers check out components or directories of components from the project repository into their private workspace and work on these copies in their private workspace.
- When their changes are complete, they check-in the components back to the repository.
- If several people are working on a component at the same time, each check it out from the repository.
 - If a component has been checked out, the VC system warns other users wanting to check out that component that it has been checked out by someone else.
- In centralized version control, there must be always a constant access to a central server

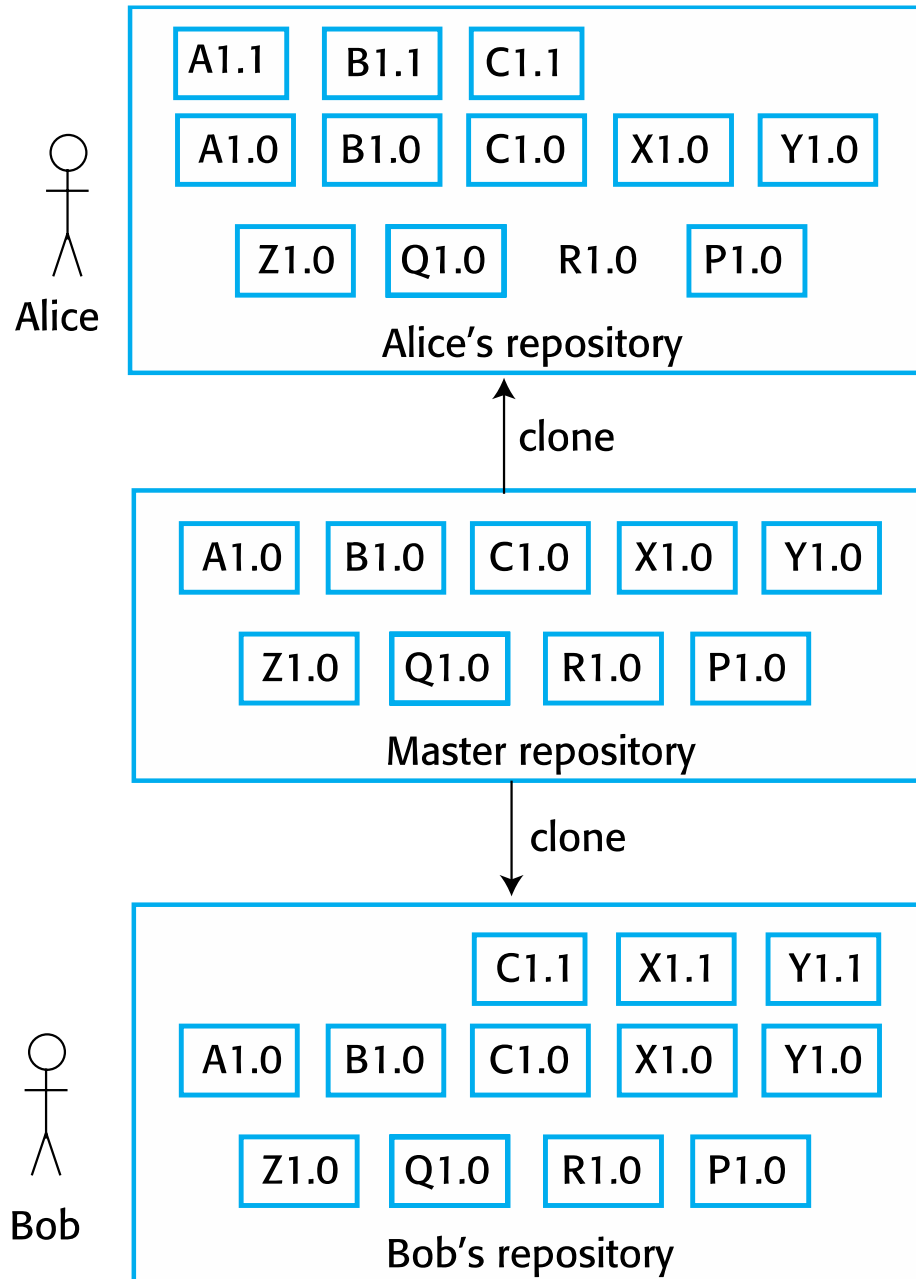
Repository Check-in/Check-out



Distributed Version Control

- A 'master' repository is created on a server that maintains the code produced by the development team.
- Instead of checking out the files that they need, a developer creates a clone of the project repository that is downloaded and installed on their computer.
- Developers work on the files required and maintain the new versions on their private repository on their own computer.
- When changes are done, they 'commit' these changes and update their private server repository. They may then 'push' these changes to the project repository.

Distributed Version Control – Repository Cloning



Question

- What are some benefits/disadvantages of Centralized Version Control System and Distributed Version Control Systems?

Benefits of Distributed Version Control

- It provides a backup mechanism for the repository.
 - If the repository is corrupted, work can continue and the project repository can be restored from local copies.
- It allows for off-line working so that developers can commit changes if they do not have a network connection.
- Project support is the default way of working.
 - Developers can compile and test the entire system on their local machines and test the changes that they have made.

Disadvantages of Distributed Version Control

- Complexity
- Increased Disk Space Usage
 - Since each developer maintains a full copy of the repository, DVCS can consume more disk space compared to CVCS, especially for large projects with extensive history or frequent branching.
- Security Risks with Distributed Access
 - DVCS introduces additional security considerations since each developer has a complete copy of the repository, potentially increasing the risk of unauthorized access, data breaches, or leaks if proper access controls are not implemented.

Benefits of Centralized Version Control

- Simplicity and Ease of Use
 - CVCS typically has a simpler setup and workflow compared to DVCS, making it easier for new developers to understand and adopt.
- Access Control
 - CVCS provides centralized access control mechanisms, allowing administrators to define permissions and restrict access to specific parts of the repository.

Disadvantages of Centralized Version Control

- Single Point of Failure
 - CVCS relies on a central server to store the repository, making it a single point of failure. If the central server experiences downtime or data loss, developers may lose access to version history and collaboration capabilities until the server is restored.
- Limited Offline Work
 - Unlike DVCS, CVCS typically requires continuous access to the central server for common operations like committing changes or viewing history.

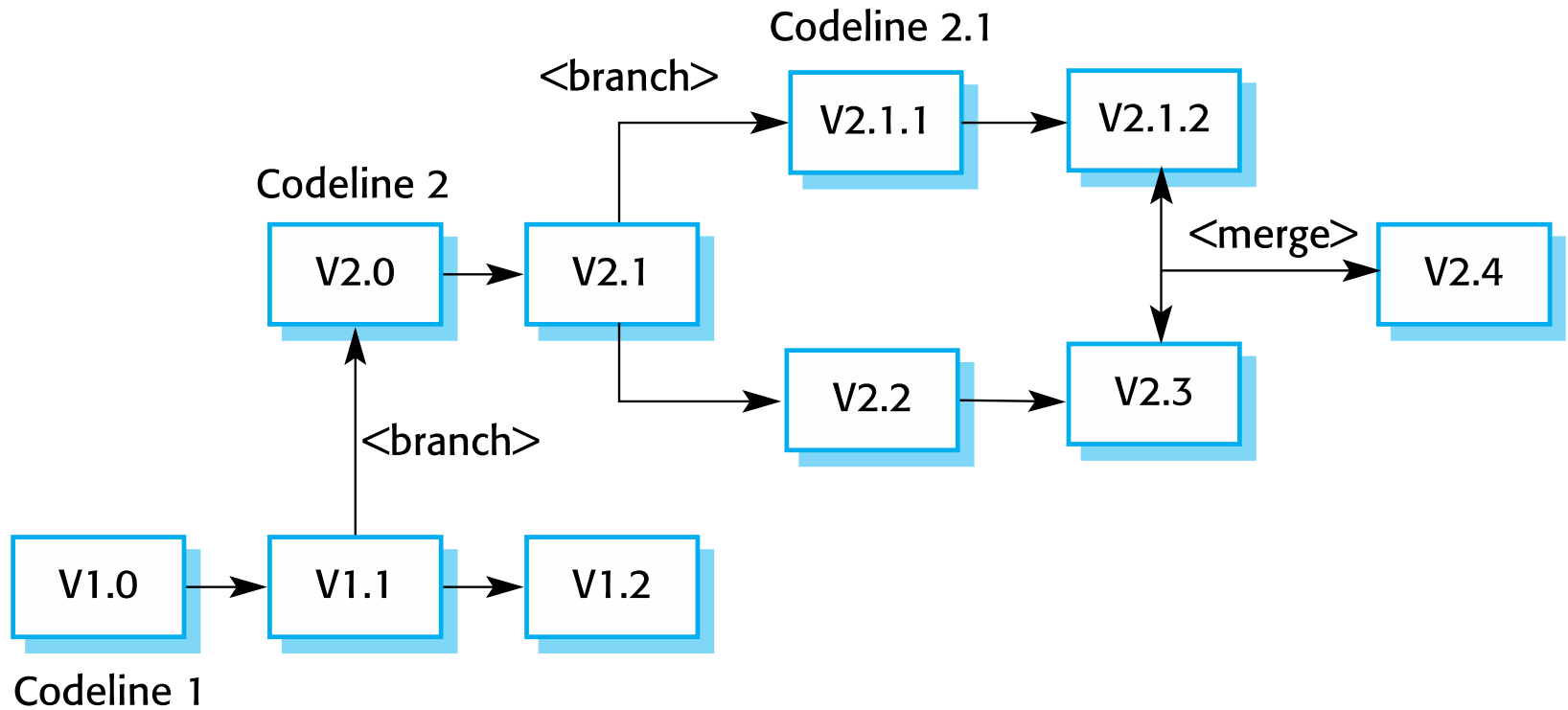
Branching and Merging

- Rather than a linear sequence of versions that reflect changes to the component over time, there may be several independent sequences.
 - This is normal in system development, where different developers work independently on different versions of the source code and so change it in different ways.
- **Branching**
 - The creation of a new codeline from a version in an existing codeline. The new codeline and the existing codeline may then develop independently.

Branching and Merging

- At some stage, it may be necessary to merge codeline branches to create a new version of a component that includes all changes that have been made.
- **Merging**
 - The creation of a new version of a software component by merging separate versions in different codelines. These codelines may have been created by a previous branch of one of the codelines involved.

Branching and Merging



Upcoming

- Lab 7
- Next Class:
 - System building, change management, release management

References

<https://www.cs.odu.edu/~zeil/cs795SD/s13/Public/scm/scm-10-16x10.pdf>

COSC 205

Configuration Management



Configuration Management

- Software systems are constantly changing during development and use.
- Configuration management (CM) is concerned with the policies, processes and tools for managing changing software systems.
- You need CM because it is easy to lose track of what changes and component versions have been incorporated into each system version.
- CM is essential for team projects to control changes made by different developers

CM Activities

- **Version management**

- Keeping track of the multiple versions of system components and ensuring that changes made to components by different developers do not interfere with each other.

- **System building**

- The process of assembling program components, data and libraries, then compiling these to create an executable system.

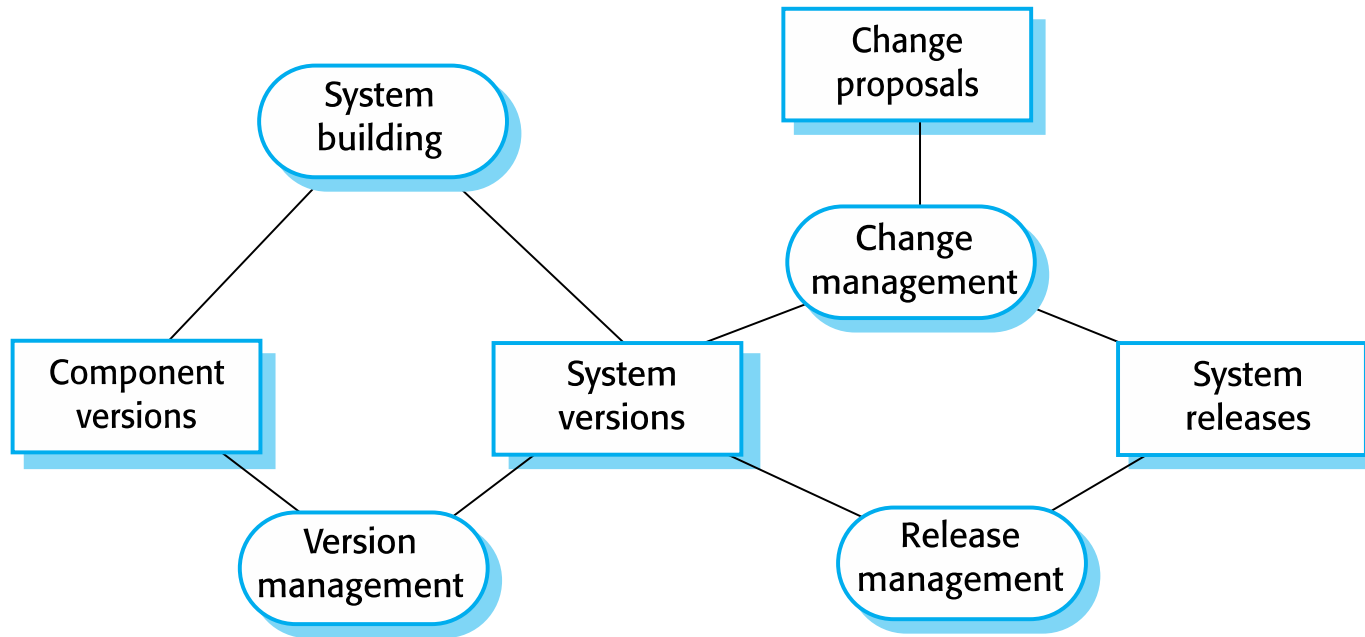
- **Change management**

- Keeping track of requests for changes to the software from customers and developers, working out the costs and impact of changes, and deciding the changes should be implemented.

- **Release management**

- Preparing software for external release and keeping track of the system versions that have been released for customer use.

CM Activities



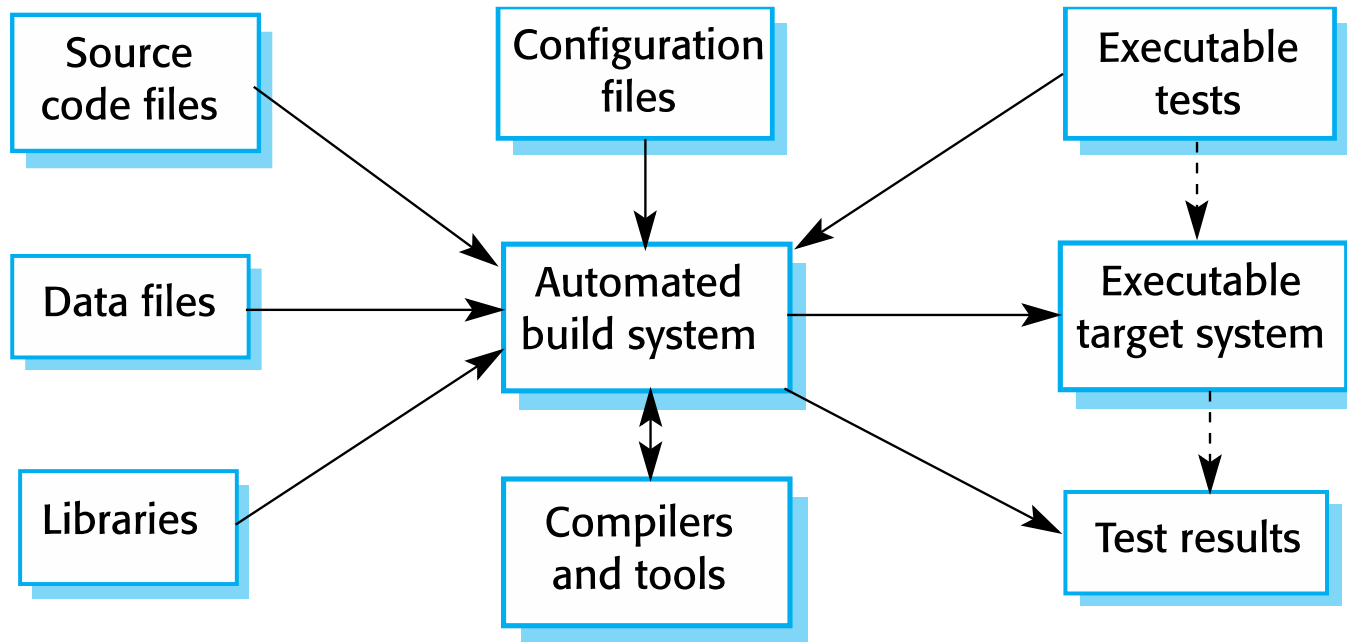
System Building

- System building is the process of creating a complete, executable system by compiling and linking the system components, external libraries, configuration files, etc.
- System building tools and version management tools must communicate as the build process involves checking out component versions from the repository managed by the version management system.
- The configuration description used to identify a baseline is also used by the system building tool.

Build Platforms

- The development system, which includes development tools such as compilers, source code editors, etc.
 - Developers check out code from the version management system into a private workspace before making changes to the system.
- The build server, which is used to build definitive, executable versions of the system.
 - Developers check-in code to the version management system before it is built. The system build may rely on external libraries that are not included in the version management system.
- The target environment, which is the platform on which the system executes.

System Building



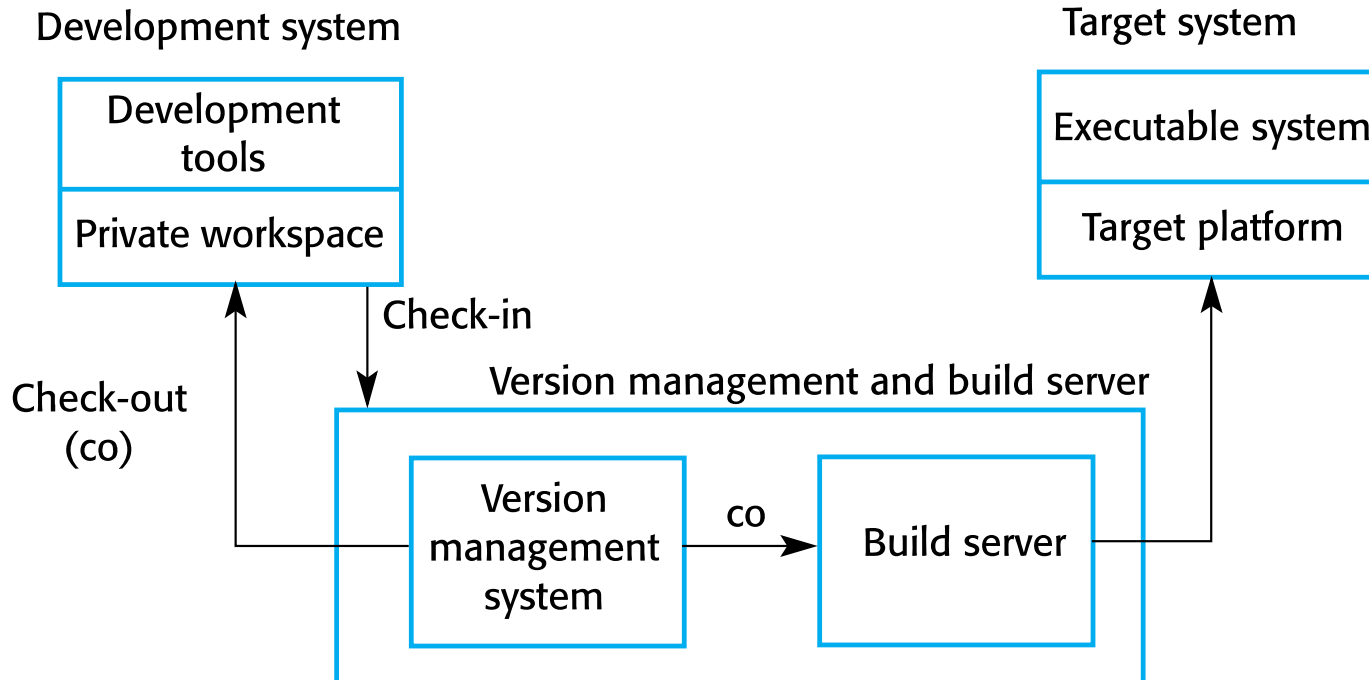
Build System Functionality

- Build script generation
- Version management system integration
- Minimal re-compilation
- Executable system creation
- Test automation
- Reporting
- Documentation generation

System Platforms

- The development system, which includes development tools such as compilers, source code editors, etc.
- The build server, which is used to build definitive, executable versions of the system. This server maintains the definitive versions of a system.
- The target environment, which is the platform on which the system executes.
 - For real-time and embedded systems, the target environment is often smaller and simpler than the development environment (e.g. a cell phone)

Development, build, and target platforms



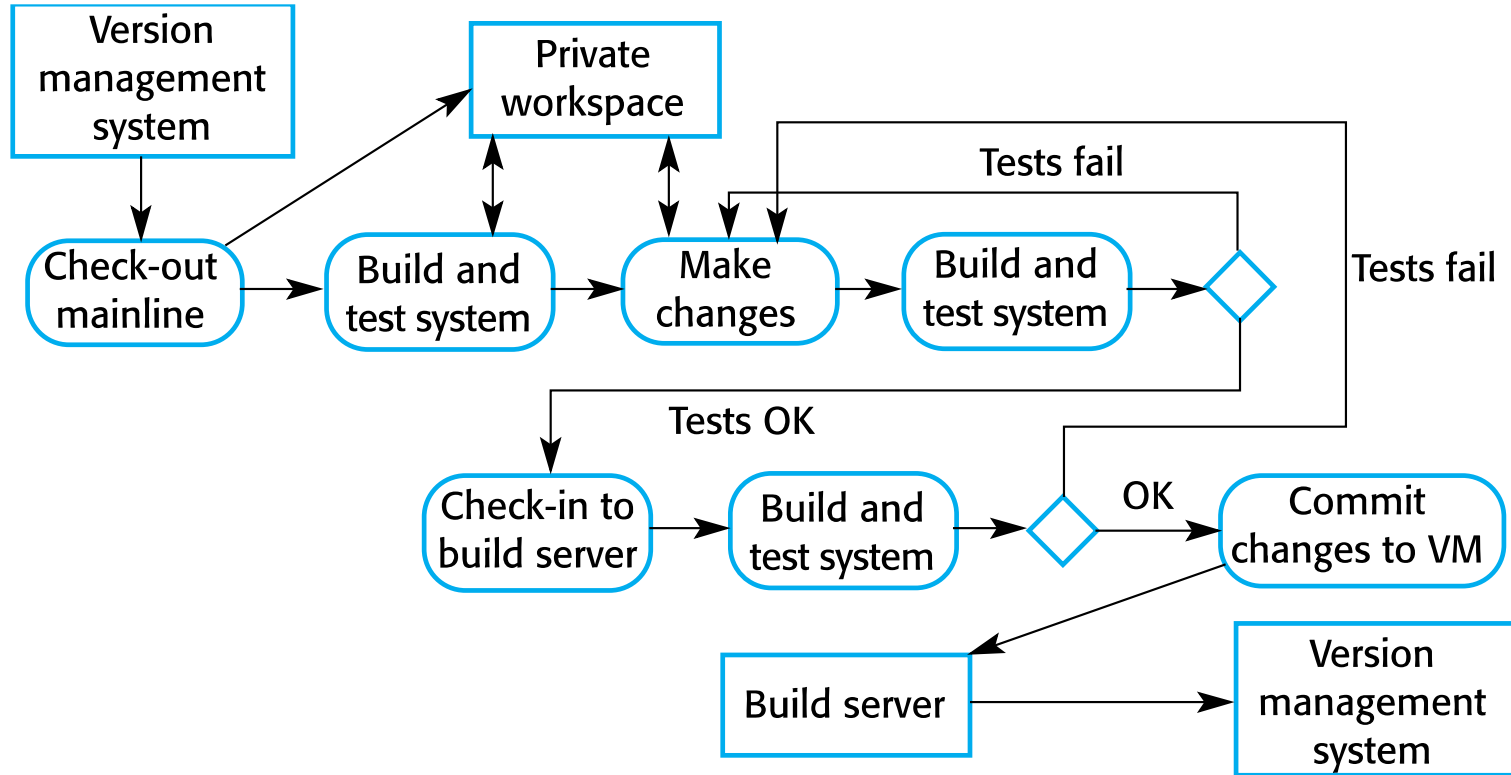
Agile Building

1. Check out the mainline system from the version management system into the developer's private workspace.
2. Build the system and run automated tests to ensure that the built system passes all tests. If not, the build is broken, and you should inform whoever checked in the last baseline system. They are responsible for repairing the problem.
3. Make the changes to the system components.
4. Build the system in the private workspace and rerun system tests. If the tests fail, continue editing.

Agile Building

5. Once the system has passed its tests, check it into the build system but do not commit it as a new system baseline.
6. Build the system on the build server and run the tests.
 - You need to do this in case others have modified components since you checked out the system.
 - If this is the case, check out the components that have failed and edit these so that tests pass on your private workspace.
7. If the system passes its tests on the build system, then commit the changes you have made as a new baseline in the system mainline.

Continuous Integration



Question

- What can you see as some pros and cons of continuous integration?

Pros and Cons of Continuous Integration

- Pros

- The advantage of continuous integration is that it allows problems caused by the interactions between different developers to be discovered and repaired as soon as possible.
- The most recent system in the mainline is the definitive working system.

- Cons

- If the system is very large, it may take a long time to build and test, especially if integration with other application systems is involved.
- If the development platform is different from the target platform, it may not be possible to run system tests in the developer's private workspace.

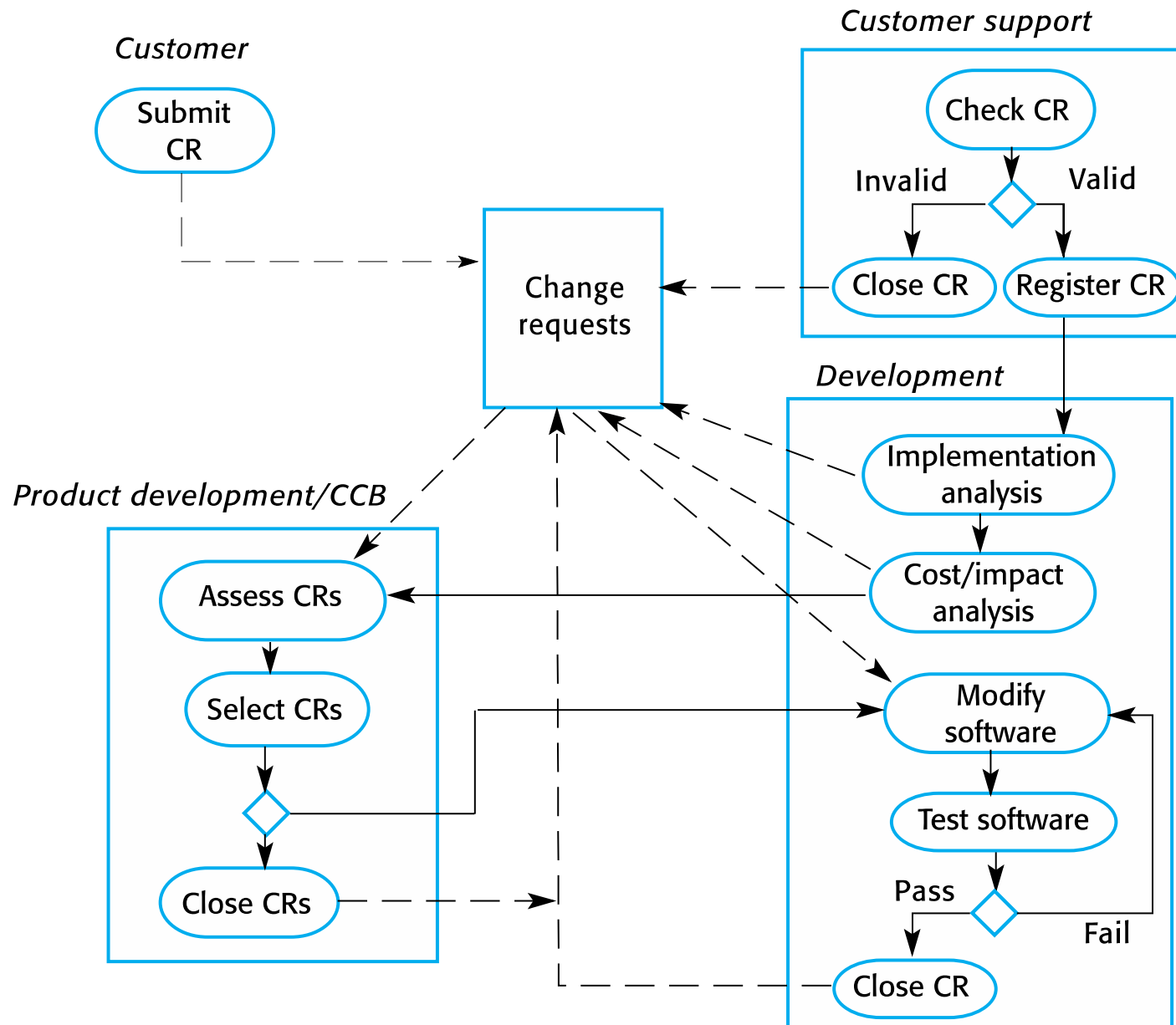
Daily Building

- The development organization sets a delivery time (say 2 p.m.) for system components.
 - If developers have new versions of the components that they are writing, they must deliver them by that time.
 - A new version of the system is built from these components by compiling and linking them to form a complete system.
 - This system is then delivered to the testing team, which carries out a set of predefined system tests
 - Faults that are discovered during system testing are documented and returned to the system developers. They repair these faults in a subsequent version of the component.

Change Management

- Organizational needs and requirements change during the lifetime of a system, bugs have to be repaired and systems have to adapt to changes in their environment.
- Change management is intended to ensure that system evolution is a managed process and that priority is given to the most urgent and cost-effective changes.
- The change management process is concerned with analyzing the costs and benefits of proposed changes, approving those changes that are worthwhile and tracking which components in the system have been changed.

Change Management Process



A partially completed change request form (a)

Change Request Form

Project: SICSA/AppProcessing

Number: 23/02

Change requester: I. Sommerville

Date: 20/07/12

Requested change: The status of applicants (rejected, accepted, etc.) should be shown visually in the displayed list of applicants.

Change analyzer: R. Looek

Analysis date: 25/07/12

Components affected: ApplicantListDisplay, StatusUpdater

Associated components: StudentDatabase

A partially completed change request form (b)

Change Request Form

Change assessment: Relatively simple to implement by changing the display color according to status. A table must be added to relate status to colors. No changes to associated components are required.

Change priority: Medium

Change implementation:

Estimated effort: 2 hours

Date to SGA app. team: 28/07/12

CCB decision date: 30/07/12

Decision: Accept change. Change to be implemented in Release 1.2

Change implementor:

Date of change:

Date submitted to QA:

QA decision:

Date submitted to CM:

Comments:

Factors in Change Analysis

- The consequences of not making the change
- The benefits of the change
- The number of users affected by the change
- The costs of making the change
- The product release cycle

Change Management and Agile Methods

- In some agile methods, customers are directly involved in change management.
- The propose a change to the requirements and work with the team to assess its impact and decide whether the change should take priority over the features planned for the next increment of the system.
- Changes to improve the software improvement are decided by the programmers working on the system.
- Refactoring, where the software is continually improved, is not seen as an overhead but as a necessary part of the development process.

Change Analysis - Example

- Your team has 2 weeks remaining in the project
- Your client has requested the following changes:
 - Add a new feature to allow users to attach files to tasks
 - There is a bug that when users go to delete tasks, the application occasionally crashes
 - Update the user interface to improve accessibility for users with disabilities
- In groups, discuss each change in terms of
 - Potential impacts of the change on the project
 - Priority
 - Change assessment (what to do to make change)
 - Your groups decision on whether to make the change or not

Revision Sheet - Example

Revision Sheet

Revision	Date	Brief Summary of Changes
Version 0.0(draft)	2023-10-01	Baseline document draft
Version 0.1	2023-10-02	formatting and some notes for future text
Version 0.2	2023-10-03	Writing for section 1
Version 0.3	2023-10-04	Writing for section 2, starting 3
Version 1.0	2023-10-06	Main body sections written. "Training" section is left for now as discussion needs to take place.
Version 1.1	2023-10-12	General editing, added training section, added diagram to baseline change process
Version 1.2	2023-10-26	Revised sections 1 through 6
Version 1.3	2023-11-05	Reviewed all sections
Version 1.4	2023-12-04	Modified some of the sections to reflect current setup. Reviewed the document and updated some links.

Release Management

- A system release is a version of a software system that is distributed to customers.
- For mass market software, it is usually possible to identify two types of release:
 - major releases which deliver significant new functionality, and
 - minor releases, which repair bugs and fix customer problems that have been reported.
- For custom software or software product lines, releases of the system may have to be produced for each customer and individual customers may be running several different releases of the system at the same time.

Release Components

- As well as the executable code of the system, a release may also include:
 - configuration files defining how the release should be configured for particular installations;
 - data files, such as files of error messages, that are needed for successful system operation;
 - an installation program that is used to help install the system on target hardware;
 - electronic and paper documentation describing the system;
 - packaging and associated publicity that have been designed for that release.

Release - Question

- What are some factors that could influence when a project team decides to release a system (either new or updated version)?

Factors Influencing System Release Planning

Factor	Description
Competition	For mass-market software, a new system release may be necessary because a competing product has introduced new features and market share may be lost if these are not provided to existing customers.
Marketing requirements	The marketing department of an organization may have made a commitment for releases to be available at a particular date.
Platform changes	You may have to create a new release of a software application when a new version of the operating system platform is released.
Technical quality of the system	If serious system faults are reported which affect the way in which many customers use the system, it may be necessary to issue a fault repair release. Minor system faults may be repaired by issuing patches (usually distributed over the Internet) that can be applied to the current release of the system.

Release Creation

- The executable code of the programs and all associated data files must be identified in the version control system.
- Configuration descriptions may have to be written for different hardware and operating systems.
- Update instructions may have to be written for customers who need to configure their own systems.
- Scripts for the installation program may have to be written.
- Web pages have to be created describing the release, with links to system documentation.
- When all information is available, an executable master image of the software must be prepared and handed over for distribution to customers or sales outlets.

Release Tracking

- In the event of a problem, it may be necessary to reproduce exactly the software that has been delivered to a particular customer.
- When a system release is produced, it must be documented to ensure that it can be re-created exactly in the future.
- This is particularly important for customized, long-lifetime embedded systems, such as those that control complex machines.
 - Customers may use a single release of these systems for many years and may require specific changes to a particular software system long after its original release date.

Release Reproduction

- To document a release, you have to record the specific versions of the source code components that were used to create the executable code.
- You must keep copies of the source code files, corresponding executables and all data and configuration files.
- You should also record the versions of the operating system, libraries, compilers and other tools used to build the software.

Release Planning

- As well as the technical work involved in creating a release distribution, advertising and publicity material must be prepared and marketing strategies put in place to convince customers to buy the new release of the system.
- Release timing
 - If releases are too frequent or require hardware upgrades, customers may not move to the new release, especially if they have to pay for it.
 - If system releases are too infrequent, market share may be lost as customers move to alternative systems.

Release Process Example

- An acceptance test will be created by the team and modified as necessary by clients.
- Prior to a release, the client will be presented with the acceptance test and sign it if they approve of the current iteration of the project.

Configuration Management Responsibilities

A few examples of what different roles could be responsible for in Configuration Management of a project

- Product Owner
 - Approving CM plan and release of software to client
 - Any change requests documented properly
- Scrum Master
 - Review and approve change requests (taking into account time remaining etc...)
- Developers
 - Give feedback on change requests
 - Ensure version control best practices are followed

Summary

- Configuration management is the management of an evolving software system. When maintaining a system, a CM team is put in place to ensure that changes are incorporated into the system in a controlled way and that records are maintained with details of the changes that have been implemented.
- The main configuration management processes are concerned with version management, system building, change management, and release management.
- Version management involves keeping track of the different versions of software components as changes are made to them.

Summary

- System building is the process of assembling system components into an executable program to run on a target computer system.
- Software should be frequently rebuilt and tested immediately after a new version has been built. This makes it easier to detect bugs and problems that have been introduced since the last build.
- Change management involves assessing proposals for changes from system customers and other stakeholders and deciding if it is cost-effective to implement these in a new version of a system.
- System releases include executable code, data files, configuration files and documentation. Release management involves making decisions on system release dates, preparing all information for distribution and documenting each system release.

Upcoming

- Lab 7
- Quiz

COSC 205

Software Evolution

Evolution processes

Legacy systems

Software maintenance



Summary From Last Class

- Configuration management is the management of an evolving software system. When maintaining a system, a CM team is put in place to ensure that changes are incorporated into the system in a controlled way and that records are maintained with details of the changes that have been implemented.
- The main configuration management processes are concerned with version management, system building, change management, and release management.
- Version management involves keeping track of the different versions of software components as changes are made to them.

Summary From Last Class

- System building is the process of assembling system components into an executable program to run on a target computer system.
- Software should be frequently rebuilt and tested immediately after a new version has been built. This makes it easier to detect bugs and problems that have been introduced since the last build.
- Change management involves assessing proposals for changes from system customers and other stakeholders and deciding if it is cost-effective to implement these in a new version of a system.
- System releases include executable code, data files, configuration files and documentation. Release management involves making decisions on system release dates, preparing all information for distribution and documenting each system release.

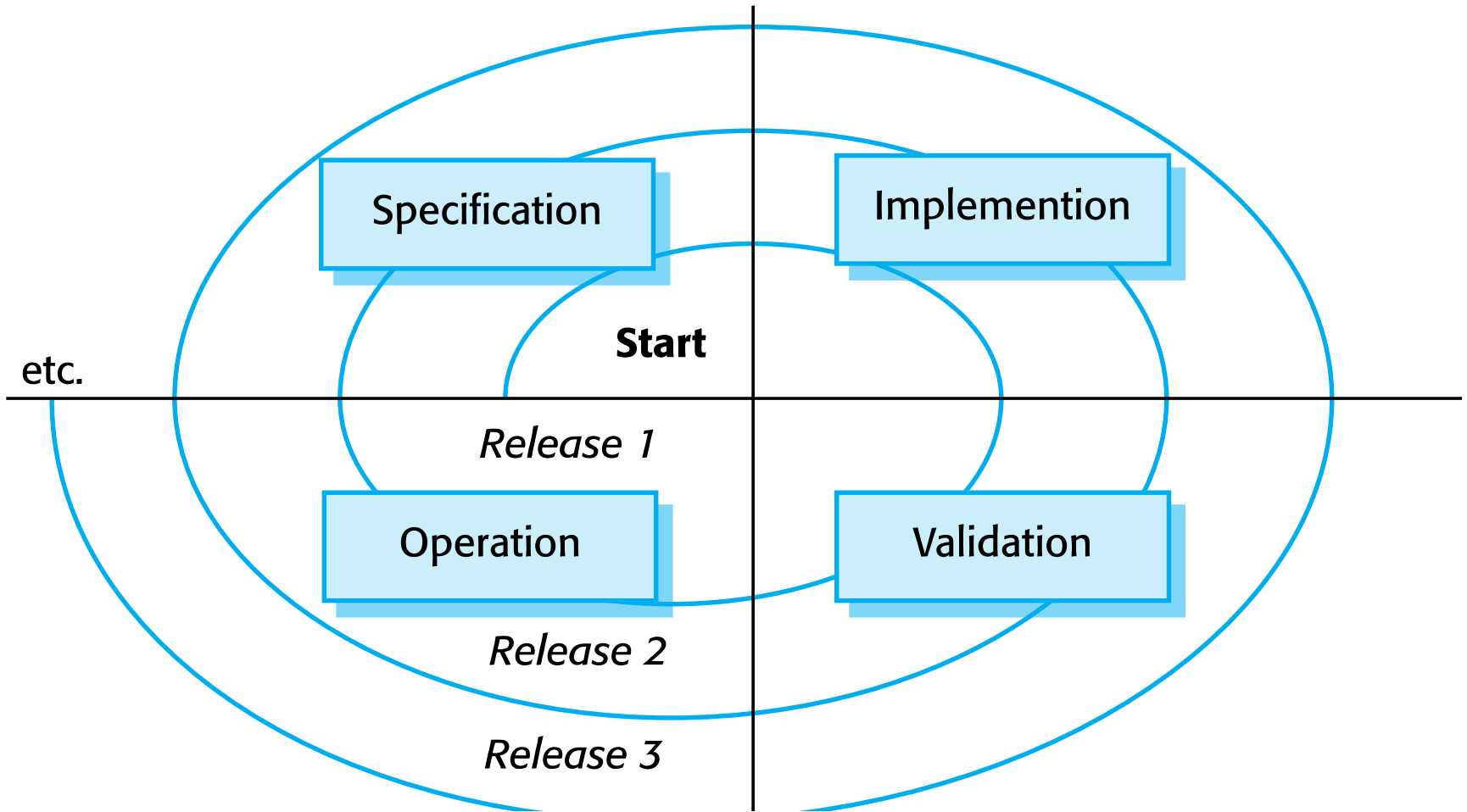
Software Change

- Software change is inevitable
 - New requirements emerge when the software is used;
 - The business environment changes;
 - Errors must be repaired;
 - New computers and equipment is added to the system;
 - The performance or reliability of the system may have to be improved.
- A key problem for all organizations is implementing and managing change to their existing software systems

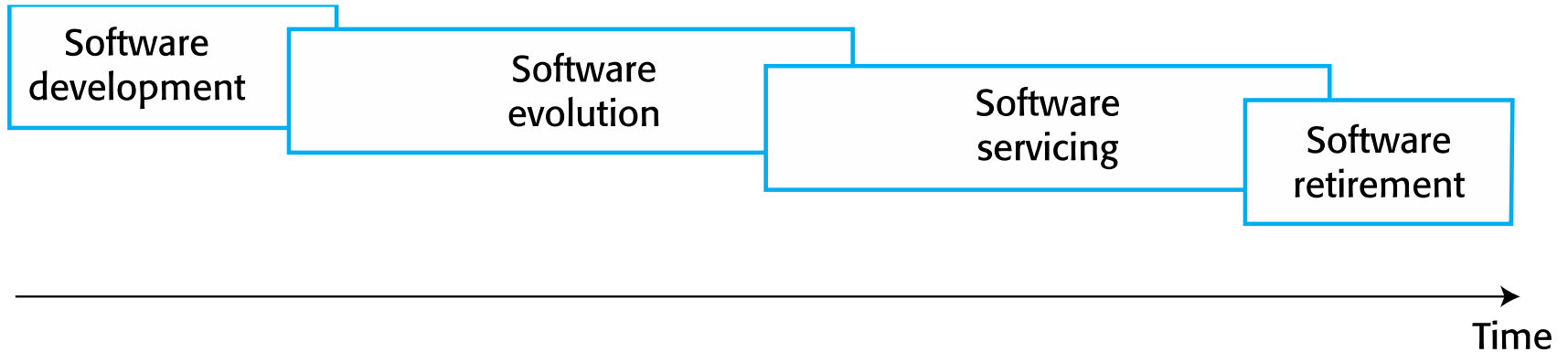
Importance of Evolution

- Organizations have huge investments in their software systems - they are critical business assets.
- To maintain the value of these assets to the business, they must be changed and updated.
- The majority of the software budget in large companies is devoted to changing and evolving existing software rather than developing new software.

A Spiral Model of Development and Evolution



Evolution and Servicing



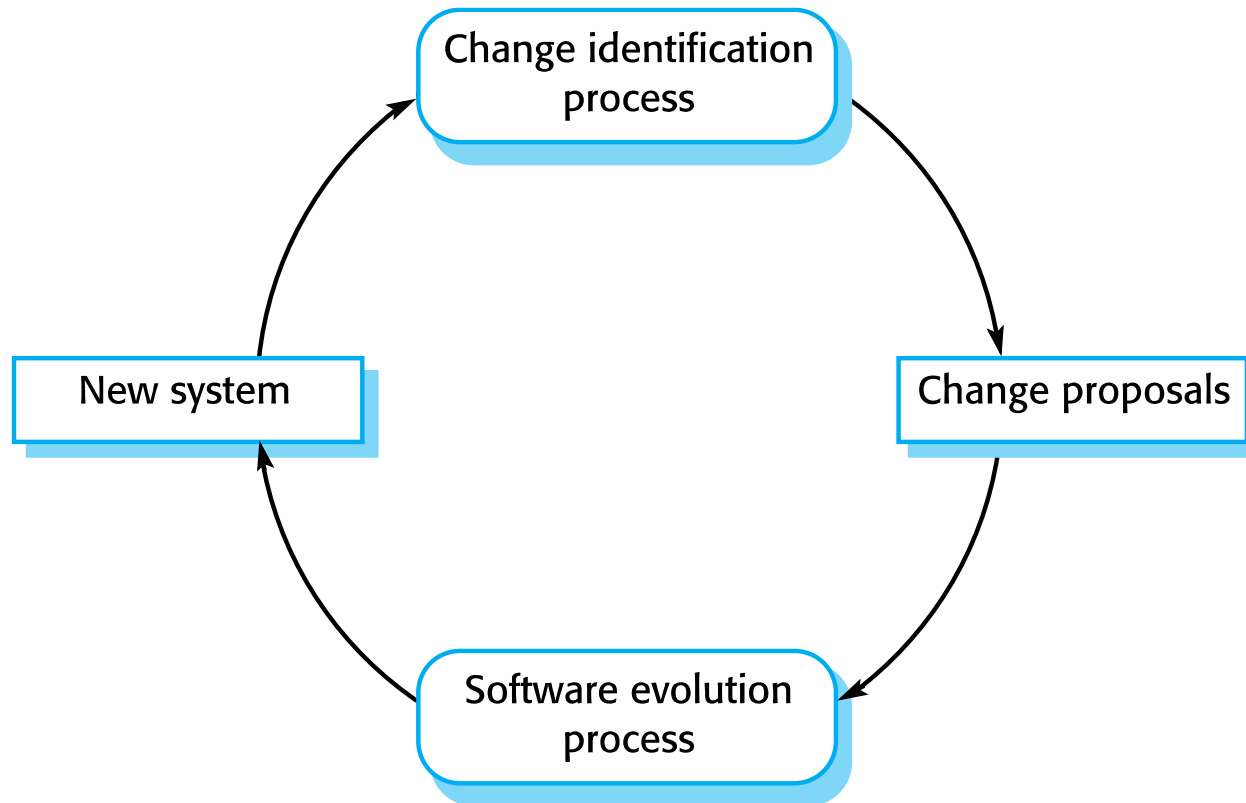
Evolution and Servicing

- Evolution
 - The stage in a software system's life cycle where it is in operational use and is evolving as new requirements are proposed and implemented in the system.
- Servicing
 - At this stage, the software remains useful, but the only changes made are those required to keep it operational i.e. bug fixes and changes to reflect changes in the software's environment.
 - No new functionality is added.
- Phase-out
 - The software may still be used but no further changes are made to it.

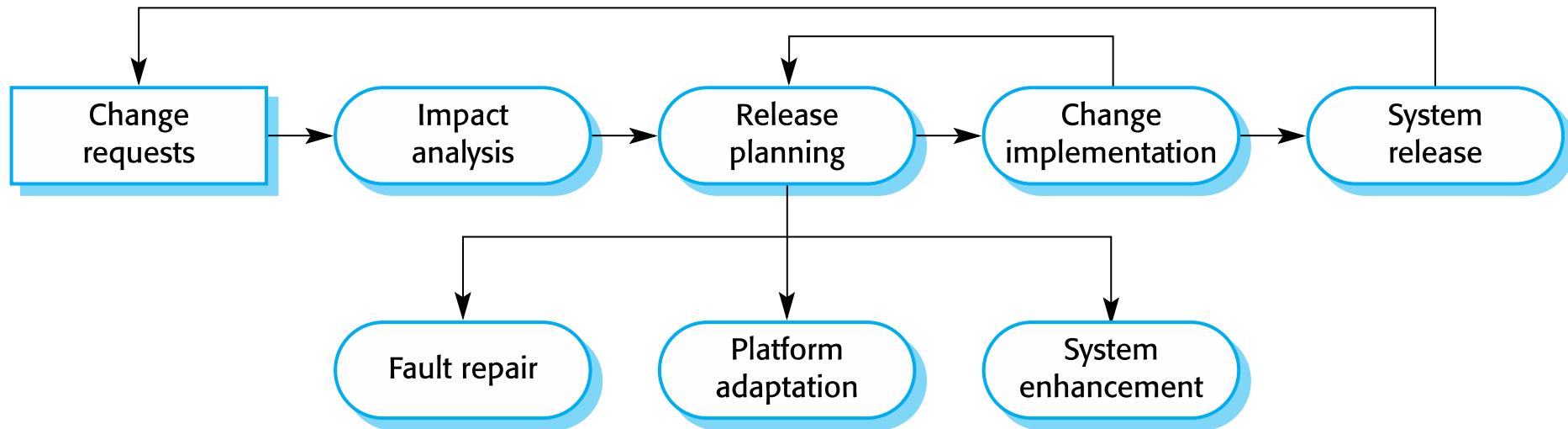
Evolution Processes

- Software evolution processes depend on
 - The type of software being maintained;
 - The development processes used;
 - The skills and experience of the people involved.
- Proposals for change are the driver for system evolution.
 - Should be linked with components that are affected by the change, thus allowing the cost and impact of the change to be estimated.
- Change identification and evolution continues throughout the system lifetime.

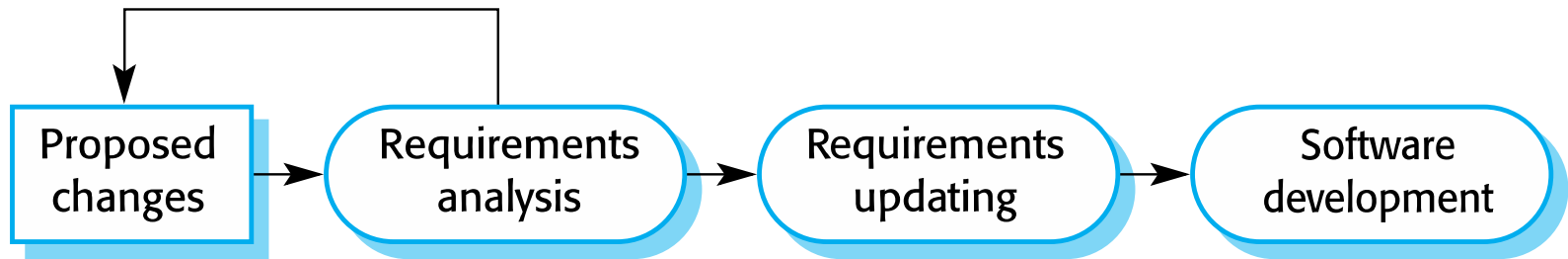
Change Identification and Evolution Processes



The Software Evolution Process



Change Implementation



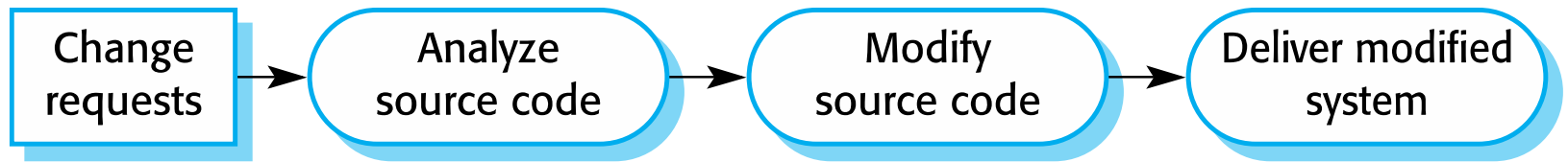
Change Implementation

- Iteration of the development process where the revisions to the system are designed, implemented and tested.
- A critical difference is that the first stage of change implementation may involve program understanding, especially if the original system developers are not responsible for the change implementation.
- During the program understanding phase, you have to understand how the program is structured, how it delivers functionality and how the proposed change might affect the program.

Urgent Change Requests

- Urgent changes may have to be implemented without going through all stages of the change process
 - If a serious system fault must be repaired to allow normal operation to continue;
 - If changes to the system's environment (e.g. an OS upgrade) have unexpected effects;
 - If there are business changes that require a very rapid response (e.g. the release of a competing product).

The Emergency Repair Process



Agile Methods and Evolution

- Agile methods are based on incremental development so the transition from development to evolution is a seamless one.
 - Evolution is simply a continuation of the development process based on frequent system releases.
- Automated regression testing is particularly valuable when changes are made to a system.
- Changes may be expressed as additional user stories.

Question

- Your project group has completed the project.
- The client then hires a new project team to take on the project
 - What are some issues that could arise if your development team used an agile approach but the evolution team prefers a more plan-based approach?
 - What if your group used a plan-based approach and the evolution team prefers an agile approach?

Handover Problems

- Where the development team have used an agile approach, but the evolution team is unfamiliar with agile methods and prefer a plan-based approach.
 - The evolution team may expect detailed documentation to support evolution, and this is not produced in agile processes.
- Where a plan-based approach has been used for development, but the evolution team prefer to use agile methods.
 - The evolution team may have to start from scratch developing automated tests and the code in the system may not have been refactored and simplified as is expected in agile development.

Legacy Systems

- Legacy systems are older systems that rely on languages and technology that are no longer used for new systems development.
- Legacy software may be dependent on older hardware, such as mainframe computers and may have associated legacy processes and procedures.
- Legacy systems are not just software systems but are broader socio-technical systems that include hardware, software, libraries and other supporting software and business processes.

Legacy Systems Components

- System hardware
 - Legacy systems may have been written for hardware that is no longer available.
- Support software
 - The legacy system may rely on a range of support software, which may be obsolete or unsupported.
- Application software
 - The application system that provides the business services is usually made up of a number of application programs.
- Application data
 - These are data that are processed by the application system. They may be inconsistent, duplicated or held in different databases.

Legacy Systems Components

- Business processes
 - These are processes that are used in the business to achieve some business objective.
 - Business processes may be designed around a legacy system and constrained by the functionality that it provides.
- Business policies and rules
 - These are definitions of how the business should be carried out and constraints on the business.
 - Use of the legacy application system may be embedded in these policies and rules.

Question

- Is anyone currently interacting with a legacy system in their projects right now? If so, what are some complications your group has had?

Legacy Systems Replacement

- Legacy system replacement is risky and expensive, so businesses continue to use these systems
- System replacement is risky for several reasons
 - Lack of complete system specification
 - Tight integration of system and business processes
 - Undocumented business rules embedded in the legacy system
 - New software development may be late and/or over budget

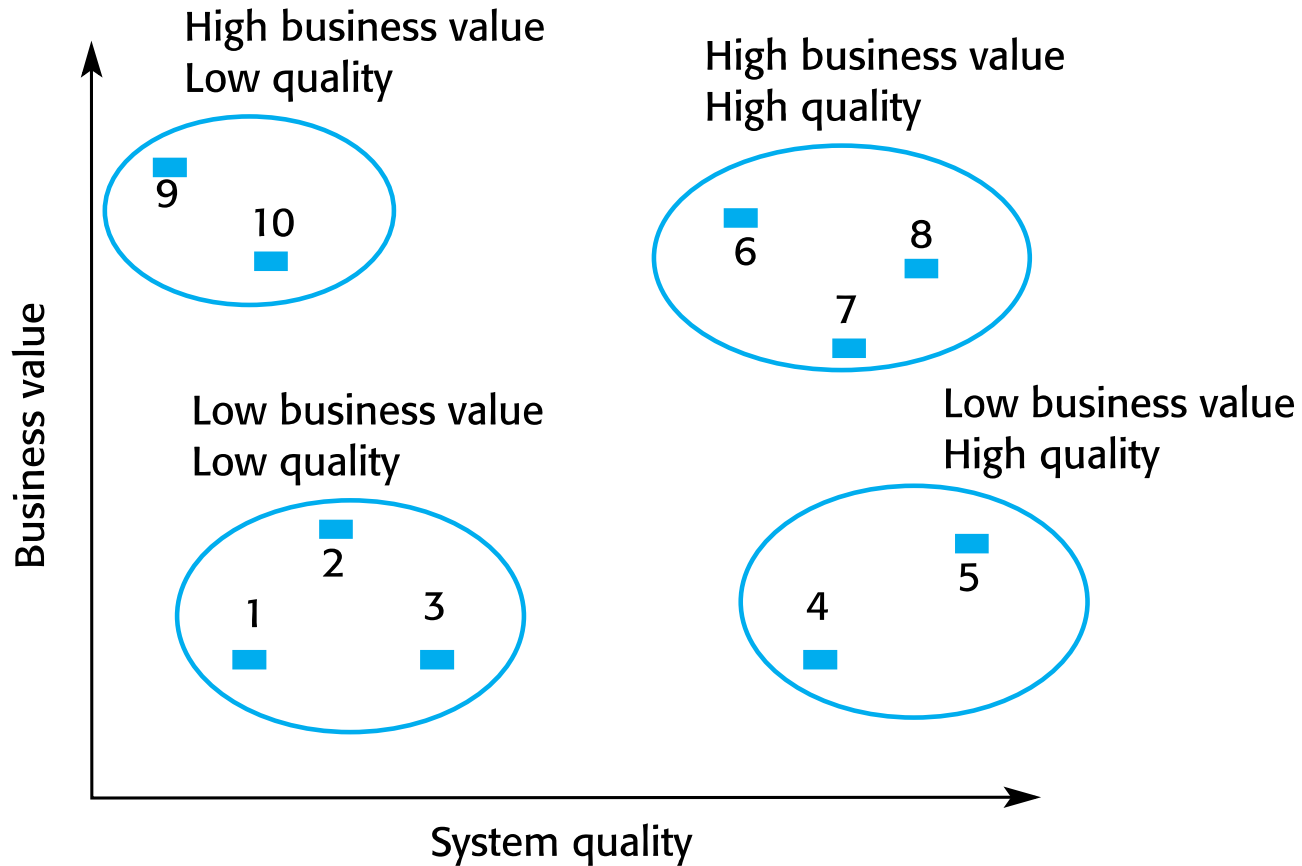
Legacy Systems Change

- Legacy systems are expensive to change for a number of reasons:
 - No consistent programming style
 - Use of obsolete programming languages with few people available with these language skills
 - Inadequate system documentation
 - System structure degradation
 - Program optimizations may make them hard to understand
 - Data errors, duplication and inconsistency

Legacy Systems Management

- Organizations that rely on legacy systems must choose a strategy for evolving these systems
 - Scrap the system completely and modify business processes so that it is no longer required;
 - Continue maintaining the system;
 - Transform the system by re-engineering to improve its maintainability;
 - Replace the system with a new system.
- The strategy chosen should depend on the system quality and its business value.

Example of Legacy System Assessment



Legacy Systems Categories

- Low quality, low business value
 - These systems should be scrapped.
- Low-quality, high-business value
 - These make an important business contribution but are expensive to maintain.
 - Should be re-engineered or replaced if a suitable system is available.
- High-quality, low-business value
 - Replace with commercial-off-the-shelf system, scrap completely or maintain.
- High-quality, high business value
 - Continue in operation using normal system maintenance.

Question

- In your groups discuss some real-world examples of legacy systems with:
 - High business value, high quality
 - High business value, low quality
 - Low business value, high quality
 - Low business value, low quality

Possible Examples

- High business value, high quality
 - Mainframe Banking System: A legacy banking system that has been in operation for several decades.
- High business value, low quality
 - Custom-built Inventory Management System: A legacy system developed in-house by a small manufacturing company to track inventory levels, orders, and shipments.
- Low business value, high quality
 - Academic Library Catalog System: A legacy library management system used by a university library to catalog books, journals, and other library materials. While the system may not directly generate revenue, it plays a crucial role in supporting the institution's academic mission by providing students and faculty with access to essential resources
- Low business value, low quality
 - Legacy Timekeeping System: An outdated timekeeping system used by a small retail business to track employee work hours and payroll

Business Value Assessment

- Assessment should take different viewpoints into account
 - System end-users;
 - Business customers;
 - Line managers;
 - IT managers;
 - Senior managers.
- Interview different stakeholders and collate results.

Issues in Business Value Assessment

- The use of the system
 - If systems are only used occasionally or by a small number of people, they may have a low business value.
- The business processes that are supported
 - A system may have a low business value if it forces the use of inefficient business processes.
- System dependability
 - If a system is not dependable and the problems directly affect business customers, the system has a low business value.
- The system outputs
 - If the business depends on system outputs, then the system has a high business value.

System Quality Assessment

- Business process assessment
 - How well does the business process support the current goals of the business?
- Environment assessment
 - How effective is the system's environment and how expensive is it to maintain?
- Application assessment
 - What is the quality of the application software system?

Business Process Assessment

- Use a viewpoint-oriented approach and seek answers from system stakeholders
 - Is there a defined process model and is it followed?
 - Do different parts of the organization use different processes for the same function?
 - How has the process been adapted?
 - What are the relationships with other business processes and are these necessary?
 - Is the process effectively supported by the legacy application software?

Factors Used in Environment Assessment

Factor	Questions
Supplier stability	Is the supplier still in existence? Is the supplier financially stable and likely to continue in existence? If the supplier is no longer in business, does someone else maintain the systems?
Failure rate	Does the hardware have a high rate of reported failures? Does the support software crash and force system restarts?
Age	How old is the hardware and software? The older the hardware and support software, the more obsolete it will be. It may still function correctly but there could be significant economic and business benefits to moving to a more modern system.
Performance	Is the performance of the system adequate? Do performance problems have a significant effect on system users?

Factors Used in Environment Assessment

Factor	Questions
Support requirements	What local support is required by the hardware and software? If there are high costs associated with this support, it may be worth considering system replacement.
Maintenance costs	What are the costs of hardware maintenance and support software licences? Older hardware may have higher maintenance costs than modern systems. Support software may have high annual licensing costs.
Interoperability	Are there problems interfacing the system to other systems? Can compilers, for example, be used with current versions of the operating system? Is hardware emulation required?

Factors Used in Application Assessment

Factor	Questions
Understandability	How difficult is it to understand the source code of the current system? How complex are the control structures that are used? Do variables have meaningful names that reflect their function?
Documentation	What system documentation is available? Is the documentation complete, consistent, and current?
Data	Is there an explicit data model for the system? To what extent is data duplicated across files? Is the data used by the system up to date and consistent?
Performance	Is the performance of the application adequate? Do performance problems have a significant effect on system users?

Factors Used in Application Assessment

Factor	Questions
Programming language	Are modern compilers available for the programming language used to develop the system? Is the programming language still used for new system development?
Configuration management	Are all versions of all parts of the system managed by a configuration management system? Is there an explicit description of the versions of components that are used in the current system?
Test data	Does test data for the system exist? Is there a record of regression tests carried out when new features have been added to the system?
Personnel skills	Are there people available who have the skills to maintain the application? Are there people available who have experience with the system?

System Measurement

- You may collect quantitative data to assess the quality of the application system
 - The number of system change requests; The higher this accumulated value, the lower the quality of the system.
 - The number of different user interfaces used by the system; The more interfaces, the more likely it is that there will be inconsistencies and redundancies in these interfaces.
 - The volume of data used by the system. As the volume of data (number of files, size of database, etc.) processed by the system increases, so too do the inconsistencies and errors in that data.
 - Cleaning up old data is a very expensive and time-consuming process

Upcoming

- Lab 8

COSC 205

Software Maintenance

Refactoring



Software Maintenance

- Modifying a program after it has been put into use.
- The term is mostly used for changing custom software
 - Generic software products are said to evolve to create new versions.
- Maintenance does not normally involve major changes to the system's architecture.
- Changes are implemented by modifying existing components and adding new components to the system.

Types of Maintenance

- Corrective Maintenance
- Adaptive Maintenance
- Perfective Maintenance
- Preventive Maintenance

Corrective Maintenance

- Corrective maintenance involves identifying and fixing defects or errors in the software.
- These defects may include bugs, glitches, or malfunctions that cause the software to behave unexpectedly or produce incorrect results.
- Corrective maintenance aims to restore the software to its intended functionality and ensure that it operates reliably and efficiently.
- Examples of corrective maintenance activities include bug fixes, error diagnosis, and software patches or updates.

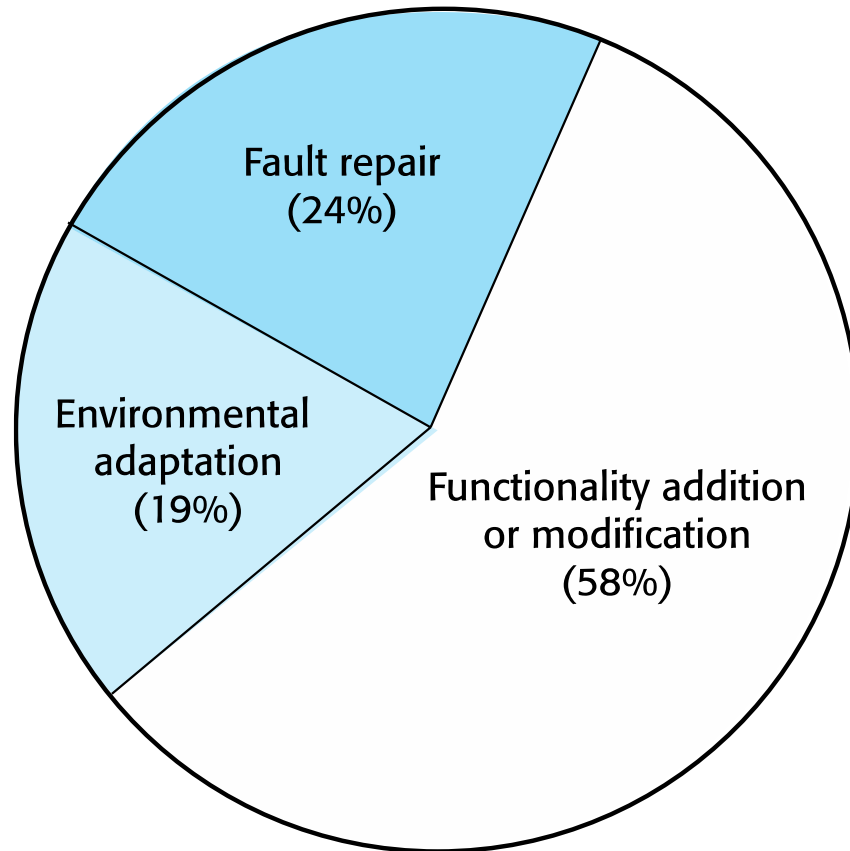
Adaptive Maintenance

- Adaptive maintenance involves modifying the software to adapt to changes in the environment, such as new hardware platforms, operating systems, or regulatory requirements.
- As technology evolves and business needs change, software systems may need to be updated or modified to remain compatible with the latest technologies and standards.
- Environmental adaptation
 - Maintenance to adapt software to a different operating environment
 - Changing a system so that it operates in a different environment (computer, OS, etc.) from its initial implementation.

Perfective Maintenance

- Perfective maintenance involves enhancing the software to improve its performance, reliability, or usability.
- These enhancements are intended to add new features, optimize existing functionality, or address usability issues based on user feedback or evolving business requirements.
- Perfective maintenance aims to enhance the overall quality and user experience of the software, making it more valuable and competitive in the marketplace.
- Examples of perfective maintenance activities include adding new features, optimizing algorithms or data structures, and improving user interface design and usability.

Maintenance Effort Distribution



Question

- In groups discuss:
- If you think it is more expensive to add new features to a system during the maintenance cycle, than to add the same features during the development cycle

Maintenance Costs

- It is usually more expensive to add new features to a system during maintenance than it is to add the same features during development
 - A new team must understand the programs being maintained
 - Separating maintenance and development means there is no incentive for the development team to write maintainable software
 - Program maintenance work is unpopular
 - Maintenance staff are often inexperienced and have limited domain knowledge.
 - As programs age, their structure degrades, and they become harder to change

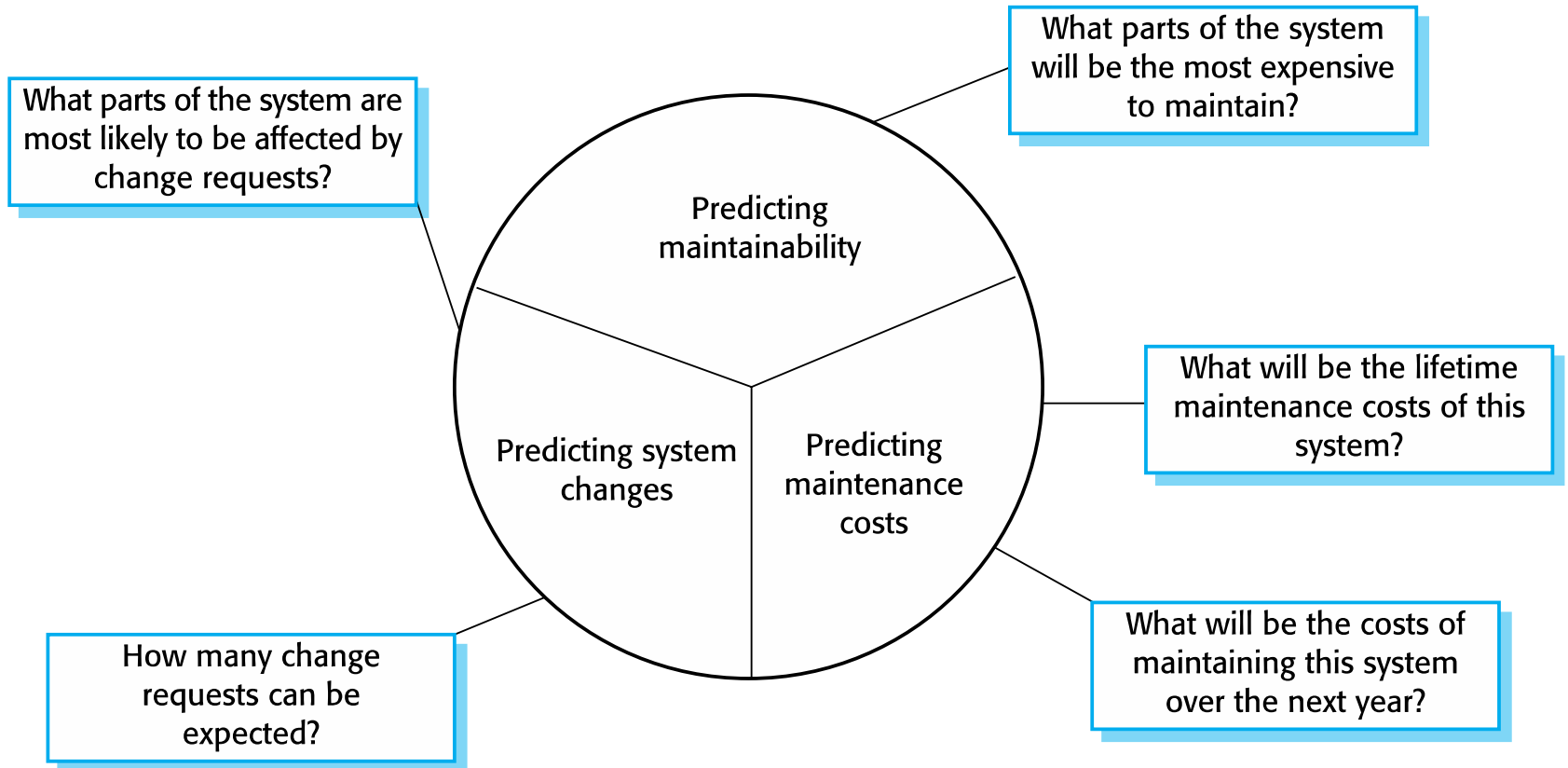
Maintenance Costs

- Usually greater than development costs
- Affected by both technical and non-technical factors.
- Increases as software is maintained. Maintenance corrupts the software structure so makes further maintenance more difficult.
- Ageing software can have high support costs (e.g. old languages, compilers etc.).

Maintenance Prediction

- Maintenance prediction is concerned with assessing which parts of the system may cause problems and have high maintenance costs.
 - Change acceptance depends on the maintainability of the components affected by the change;
 - Implementing changes degrades the system and reduces its maintainability;
 - Maintenance costs depend on the number of changes and costs of change depend on maintainability.

Maintenance Prediction



Change Prediction

- Predicting the number of changes requires an understanding of the relationships between a system and its environment.
- Tightly coupled systems require changes whenever the environment is changed.
- Factors influencing this relationship are
 - Number and complexity of system interfaces;
 - Number of inherently volatile system requirements;
 - The business processes where the system is used.

Complexity Metrics

- Predictions of maintainability can be made by assessing the complexity of system components.
- Studies have shown that most maintenance effort is spent on a relatively small number of system components.
- Complexity depends on
 - Complexity of control structures;
 - Complexity of data structures;
 - Object, method (procedure) and module size.

Process Metrics

- Process metrics may be used to assess maintainability
 - Number of requests for corrective maintenance;
 - Average time required for impact analysis;
 - Average time taken to implement a change request;
 - Number of outstanding change requests.
- If any or all of these is increasing, this may indicate a decline in maintainability.

Preventive Maintenance

- Preventive maintenance involves proactively identifying and addressing potential issues or risks in the software to prevent future problems or failures.
- By identifying and mitigating potential sources of technical debt, vulnerabilities, or performance bottlenecks early on, preventive maintenance helps minimize the likelihood of costly downtime, security breaches, or system failures in the future.
- Examples of preventive maintenance activities include code refactoring, performance tuning

Refactoring

- Refactoring is the process of making improvements to a program to slow down degradation through change.
- You can think of refactoring as 'preventative maintenance' that reduces the problems of future change.
- Refactoring involves modifying a program to improve its structure, reduce its complexity or make it easier to understand.
- When you refactor a program, you should not add functionality but rather concentrate on program improvement.

Bad Smells in Program Code

- Duplicate code
 - The same or very similar code may be included at different places in a program. This can be removed and implemented as a single method or function that is called as required.
- Long methods
 - If a method is too long, it should be redesigned as several shorter methods.
- Switch (case) statements

Code Reviews

- Part of the software development process where peers or team members systematically examine each other's code to ensure quality, adherence to coding standards, and identify potential issues or improvements.
- One or more people check a program, mainly by viewing and reading parts of its source code, either after implementation or as an interruption of implementation.

Code Reviews Purpose

- Improve code quality
 - By having multiple sets of eyes review code, errors and potential improvements can be identified early in the development process.
- Knowledge sharing
 - Code reviews provide an opportunity for team members to learn from each other and gain insights into different approaches and best practices.
- Ensure consistency
 - Code reviews help enforce coding standards and maintain consistency across the codebase.
- Foster collaboration
 - Through constructive feedback and discussion, code reviews encourage collaboration and teamwork among developers.

Code Reviews Process

- Preparing for review
 - The developer submits their code for review, along with any relevant documentation or context
- Reviewing the code
 - Reviewers carefully examine the code, looking for errors, inefficiencies, and areas for improvement
- Providing feedback
 - Reviewers provide feedback to the developer, pointing out any issues or suggesting improvements
- Iterating
 - The developer addresses the feedback received during the review and makes any necessary changes to the code.
 - This process may involve multiple iterations until the code meets the required standards and quality criteria.
- Approving the code
 - Once the code has been reviewed and approved, it can be merged into the main codebase.

Sample Review Criteria

- Readability:
 - Is the code well-structured and easy to understand?
- Functionality:
 - Does the code meet the requirements specified in the task or user story?
- Performance:
 - Are there any potential optimizations that could be made to improve performance?
- Security:
 - Are security best practices followed, such as input validation and sanitization?
- Code Style and Standards:
 - Does the code adhere to the established coding standards and style guidelines?
- Testing:
 - Are there any missing or inadequate test cases that need to be added?

Code Reviews Format

- Outline of a code review

Code Review

Reviewer: [Name(s) goes here]

Created by: [Who created the piece of code (multiple names if needed)]

File Name: _____

Code block:

[Insert code block being reviewed here]

Suggestions and comments based on review. Ensure to reference proper lines segments and sections of the code go here.

Changes made by original creators go here. Be detailed showing exactly where the changes were made and leave comments about what changes were made for the comments provided.

Exercise – Code Reviews

- Pair up with a partner
- Both of you individually write a function to compute the factorial of a number (10 minutes)
 - Remember: Factorial of 5 is $5*4*3*2*1 = 120$
- Share the method you wrote with your partner
- Have your partner conduct a short code review (5 minutes)
- Implement the changes your partner suggested (5 minutes)

Example Factorial Solution if needed

```
public class FactorialCalculator {  
  
    public static void main(String[] args) {  
        int n = 5; //example input  
        int s = compute(n);  
        System.out.println(s); }  
  
    public static int compute(int n) {  
        int r = 1;  
        for (int i = 1; i <= n; i++) {  
            r = r * i; }  
        return r; }  
}
```


Summary

- Software development and evolution can be thought of as an integrated, iterative process that can be represented using a spiral model.
- For custom systems, the costs of software maintenance usually exceed the software development costs.
- The process of software evolution is driven by requests for changes and includes change impact analysis, release planning and change implementation.
- Legacy systems are older software systems, developed using obsolete software and hardware technologies, that remain useful for a business.

Summary

- It is often cheaper and less risky to maintain a legacy system than to develop a replacement system using modern technology.
- The business value of a legacy system and the quality of the application should be assessed to help decide if a system should be replaced, transformed or maintained.
- There are four types of software maintenance: corrective maintenance (bug fixing), adaptive maintenance (modifying software to work in a new environment), perfective maintenance (implementing new or changed requirements), and preventive maintenance (improving the software to prevent future problems).

Upcoming

- Lab 8
- Quiz

COSC 205

Developer's Guide



Developer's Guide

- Comprehensive resource designed to assist software developers in understanding, navigating, and contributing to a software project
- It serves as a roadmap or reference manual that provides essential information, guidelines, and best practices tailored specifically for developers working on a particular software system or application

Target Audience

- The primary audience for a developer's guide is software developers, programmers, and technical personnel involved in the development, customization, and maintenance of the software project
- It provides detailed technical information, guidelines, and best practices tailored to assist developers in understanding, contributing to, and working with the software codebase

Developer's Guide Purposes

- **Onboarding New Developers**

- A developer's guide helps onboard new team members by providing them with a clear understanding of the project's architecture, design principles, coding standards, and development workflow.
- This accelerates their integration into the development team and reduces the learning curve associated with understanding the project's intricacies.

Developer's Guide Purposes

- **Guidance and Best Practices**

- It serves as a central repository of guidelines, best practices, and conventions that developers should adhere to during the development process.
- This includes coding standards, naming conventions, code formatting rules, and design principles, ensuring consistency and maintainability across the codebase.

Developer's Guide Purposes

- **Understanding Project Structure**

- A developer's guide provides insights into the project's architecture, components, modules, and dependencies.
- It includes architectural diagrams, system design documentation, and module descriptions to help developers comprehend the project's structure and relationships between its various components.

Developer's Guide Purposes

- **Troubleshooting and Debugging**

- It often includes troubleshooting guides, debugging tips, and solutions to common issues encountered during development.
- This empowers developers to identify and resolve problems efficiently, minimizing downtime and accelerating the development cycle.

Developer's Guide Purposes

- **Continuous Learning and Improvement**

- A developer's guide fosters a culture of continuous learning and improvement within the development team.
- Developers can refer to the guide to stay updated on project updates, new features, and evolving best practices, ensuring that their skills remain relevant and aligned with the project's goals.

Building a Developer's Guide

- Title Page
- Revision history
- Table of contents
- Project overview
- Installation instructions
- Design and architecture
- Testing instructions
- Coding standards

Project Overview

- High-level understanding of the project, including its objectives, scope, and target audience
- Any references made to other documents
- Required skills of developers working on the software

Installation Instructions

- Majority of document
- Detail the steps required to set up the development environment, including software dependencies, configurations, and any specific setup instructions
- Information on the project's codebase
- Include troubleshooting tips for common installation issues to facilitate a smooth setup process for developers
- Detailed as possible, explaining all items necessary for complete setup and understanding of the software
 - Screenshots are very helpful in the document as well

Design and Architecture

- This section delves into the project's architecture, design principles, and key components
- UML diagrams or architectural diagrams, to visualize the project's structure

Testing Instructions

- Guidance to developers on how to effectively test the software to ensure its quality, reliability, and functionality
- What testing was performed
- Frameworks used
- Setting up test environment
- Executing tests

Coding Standards

- Define coding standards, including naming conventions, code formatting rules, and best practices relevant to the project's programming languages and technologies
- Provide examples and code snippets demonstrating adherence to the coding standards outlined
- Benefits of enforcing coding standards, such as improved code readability, maintainability, and collaboration among team members

Example Development Guides

- <https://borealisdata.ca/guides/en/latest/developers/index.html>
- <https://developers.google.com/gdata/docs/developers-guide>
- <https://developer.mozilla.org/en-US/docs/Web/Guide>

Upcoming

- Quiz (about Developer's Guide)
- Lab 9
- Next class
 - User Manual
- Presentation next week.

Final Project Presentation

- Around 30-minute presentation
- Team A B C next Monday
- Team D E F next Wednesday
- If the presentation score is higher than the Mini Spike presentation score, it will replace the previous score.

Final Project Presentation Outline

Similar format to Spike Final Presentation

- Project introduction
- Project plan
- Project demo
- Challenges of the project
- Future enhancement

Project Introduction

- Introduction to project team
- Introduction to the client or company (if any)
- Project description and objectives
- Etc...

Project Plan

- Review the plan your group made at the beginning of the project
- Compare what was expected to be done with what was actually accomplished
- Etc...

Project Demo

- Demo of the software
- Most of the time allotted during the presentation
- Can talk about how things were built during the demo as well
 - What was used etc...
- Etc...

Challenges of the Project

- Technical challenges
- Group challenges, team collaboration challenges
- Roadblocks met along the way
- Did you hit any of the risks outlines in your risk management plan? How did you go about mitigating the risk or overcoming it?
- Etc...

Future Enhancement

- Future improvements
 - Functionality, future roadmap etc...
- Where can client go in the future from this version of the product?
- Areas that need to be improved
- Etc...

Final Project Presentation Outline

- This is a rough outline, if your team wants to add more to the project please do so
- Just ensure it is within the time frame
 - Don't be too short either
 - Optimal time frame 25-30 minutes
- Ensure all team members present

COSC 205

User Manual



User Manual

- A document or guide that provides instructions, information, and guidance to end-users on how to effectively use a software application, product, or device
- It serves as a comprehensive reference resource that helps users understand the functionalities, features, and capabilities of the software or product, as well as how to perform various tasks and operations.
- User manuals contain detailed, step-by-step instructions for the end user and allow for some support in troubleshooting

Target Audience

- The primary audience for a user manual is end-users or non-technical individuals who will interact with the software application or system
- They are not meant to be read from cover to cover, but as reference books, so a table of contents should always be included in a user manual.

User Manual Purposes

- **Guidance and Instructions**

- A user manual provides users with clear and comprehensive guidance on how to use the software application
- It includes step-by-step instructions, tutorials, and explanations to help users navigate the interface, perform tasks, and utilize features effectively.

User Manual Purposes

- **Onboarding and Training**

- The user manual serves as a valuable resource for onboarding new users and training them on how to use the software
- It introduces users to the software's functionalities, workflows, and best practices, reducing the learning curve and accelerating their proficiency with the application.

User Manual Purposes

▪ Reference Material

- Users can refer to the user manual as a reference guide when they encounter questions, issues, or uncertainties while using the software
- It provides quick access to information about features, options, settings, and procedures, empowering users to find solutions independently.

User Manual Purposes

- **Enhancing User Experience**

- A well-crafted user manual contributes to a positive user experience by providing users with the information and support they need to accomplish their goals effectively and efficiently
- It instills confidence, reduces frustration, and fosters satisfaction with the software application.

User Manual Purposes

- **Reducing Support Requests**

- By empowering users with self-service resources and solutions, the user manual helps reduce the volume of support requests and inquiries directed to customer support or helpdesk teams
- Users can find answers to their questions and resolve issues independently, saving time and resources for both users and support personnel.

Elements of a Good User Manual

- Plain language
 - Writing for the users not the developers
- Simple
 - Crowding your documentation with complicated illustrations, and dense blocks of text will give the sense that the user guide is too complex and inaccessible
- Visuals
 - Images, screenshots clarify a concept
- Logical flow
 - Use headings, subheadings appropriately (maintain structure)
 - Step-by-step instructions

How to know if you built a good user manual?

- Test instructions alongside the product using naive users
 - An important step in the process of writing a user manual is the testing.
 - The choice of who to test on can change the results dramatically
 - Ideally, testing should be performed on individuals who have never used your product or viewed the manual before
 - Observe them working through the process and make note of where they get stuck as they progress through the user manual
 - Your testers should be able to navigate the use of the product with only the support of the user manual
 - They should not need to reach out for additional support.
 - Everything they need should be in the user guide itself

Building a User Manual

- Title Page
- Revision history
- Table of contents
- Introduction
- Pre-requisites
- Step-by-step Instructions
- Appendix

Table of Contents

- A table of contents appears at the beginning of an article and lays out all the sections contained within the document
- Customers can click through to the section they feel is most relevant to them rather than having to read the whole article from beginning to end
- Saves your customers time in navigation, by being able to quickly find if what they are looking for is included or not

Introduction

- Brief overview of the document and what it's purpose is, intended audience

Pre-requisites

- Outline any minimum hardware or software specifications necessary for running the software system
 - Operating system, device etc..
- Network requirements
 - Internet connectivity etc...
- Pre installation steps if any

Step-by-step Instructions

- This isn't necessarily a section labelled "Step-by-step instructions", more so the part of the document where all the instructions will be placed into for end users to use
- Can have different sections outline instructions for different parts
 - Ex: Section for creating an account, logging in, editing account, etc...
- Main part of the document
 - Crucial to have separate sections/sub sections to allow for users to easily navigate to the appropriate part of the document, using the table of contents

Appendix

- Supplementary section that is added to the end of a document,
- The purpose of an appendix is to provide additional information, data, or details that are relevant to the main content but may not fit seamlessly within the main body of the document
- Any additional references to documentation, more support information etc...

Example User Manuals

- <https://borealisdata.ca/guides/en/latest/user/index.html#>
- https://www.google.com/help/hc/images/android/android_ug_50/Android-Lollipop-Quick-Start-Guide.pdf
- <https://docs.github.com/en/get-started>
- <https://helpx.adobe.com/ca/photoshop-elements/user-guide.html>

Upcoming

- Quiz
- Lab 9
- Next week
 - Final Presentation
- The week after next
 - Code Breaking Exercise

References

- <https://www.techsmith.com/blog/user-documentation/>
- <https://document360.com/blog/user-documentation/>

COSC 205 - Winter 2026

Midterm Exam Review

Overview

- **Part A:** Multiple Choice (20 marks)
- **Part B:** Short Answer (15 marks)
- **Part C:** Long Answer (15 marks)
- **Total:** 50 marks

Part A - Multiple Choice

20 questions covering key concepts

Question 1

What are the three components of a user story?

- a) Role, Feature, Reason
- b) Requirement, User, Implementation
- c) Role, Task, Reason
- d) Role, Feature, Iteration

Answer: a) Role, Feature, Reason

Question 2

Why is it important to add acceptance criteria to a user story?

- a) To assign tasks to team members
- b) To estimate the time required for development
- c) To create additional documentation for the project
- d) To provide a clear definition of when the story is complete

Answer: d) To provide a clear definition of when the story is complete

- Acceptance criteria ensure everyone knows when a story is “done”
- Provides measurable completion standards

Question 3

In Pair Programming, what are the roles of the “Driver”? (Multiple answers)

- a) Observing and guiding
- b) Controlling keyboard and mouse
- c) Not paying attention
- d) Responsible for creating the code
- e) Actively talking to the co-pilot

Answer: b, d, e

- Driver controls the keyboard/mouse
- Creates the code
- Actively communicates with the navigator

Question 4

What is the primary sequence of steps in Test Driven Development (TDD)?

- a) Write tests, code, refactor
- b) Write code, refactor, write tests
- c) Refactor, write tests, code
- d) Code, write tests, refactor

Answer: a) Write tests, code, refactor

Question 5

Which UML diagram is best suited for representing the flow of activities and actions?

- a) Use Case Diagram
- b) Statechart Diagram
- c) Sequence Diagram
- d) Activity Diagram

Answer: d) Activity Diagram

- Shows workflow and decision points
- Represents business processes and algorithms

Question 6

Functional requirements represent _____

- a) How well the system provides the functional requirements
- b) What the system does or is expected to do
- c) How well the system allows the user to complete tasks
- d) The performance of a system

Answer: b) What the system does or is expected to do

- Functional requirements define system behavior
- Non-functional requirements define quality attributes

Question 7

Which of the following best describes the purpose of story points?

- a) To measure the time required to complete a task
- b) To track the progress of team members
- c) To allocate resources for a project
- d) To estimate the complexity and effort required for a user story

Answer: d) To estimate the complexity and effort required for a user story

- Story points are relative measures of effort
- Not tied to specific time units

Question 8

What has the client signed up for on the project? (Multiple answers)

- a) Initial meetings to give you further details on requirements
- b) Coding help
- c) Negotiate how much work is done from sprint to sprint
- d) Availability to answer questions
- e) Meetings every day of the week to discuss the project

Answer: a, c, d

- Client provides requirements and clarification
- Participates in sprint planning
- Available for questions, but not daily meetings

Question 9

_____ is an abstraction that connects two classes

- a) Inheritance
- b) Multiplicity
- c) Association
- d) Navigability

Answer: c) Association

- Association represents relationships between classes
- Can have multiplicity and navigability properties

Question 10

The _____ represents the stakeholders and is the voice of the customer

- a) Professor
- b) Development Team
- c) Scrum Master
- d) Product Owner

Answer: d) Product Owner

- Product Owner prioritizes the backlog
- Represents customer interests
- Makes final decisions on requirements

Question 11

Activity Diagram - Which action sequences are possible? (Multiple answers)

- a) $A \rightarrow B$
- b) $A \rightarrow C \rightarrow B \rightarrow D \rightarrow F$
- c) $A \rightarrow C \rightarrow D \rightarrow F$
- d) $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$
- e) $A \rightarrow C \rightarrow D \rightarrow B \rightarrow E \rightarrow F$
- f) $A \rightarrow B \rightarrow E$

Answer: b, d

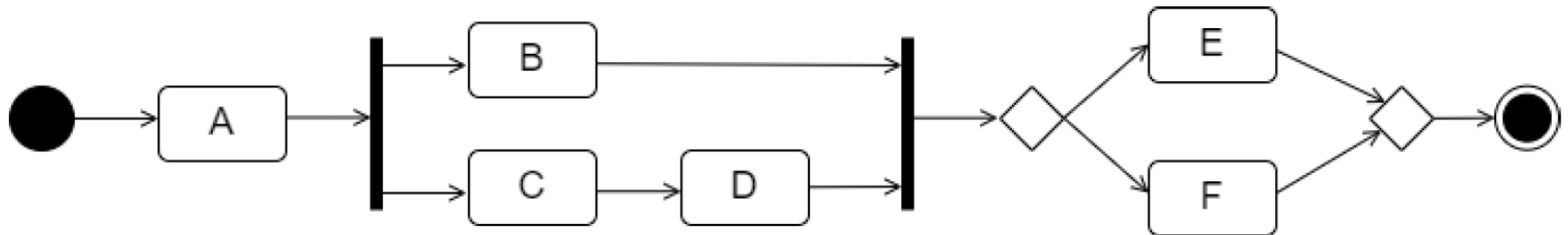
- Must follow the flow paths in the diagram
- Consider decision points and merge points

Fork

Join

Decision

Merge



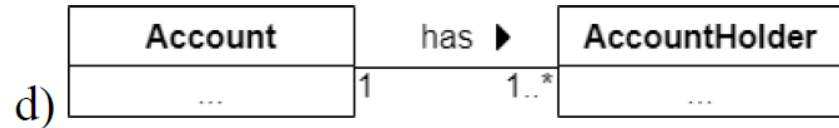
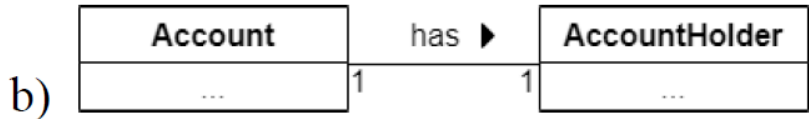
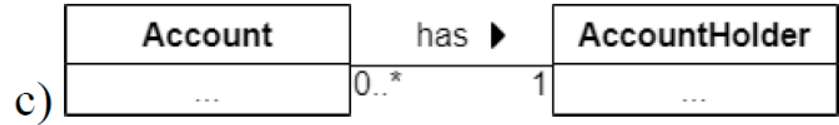
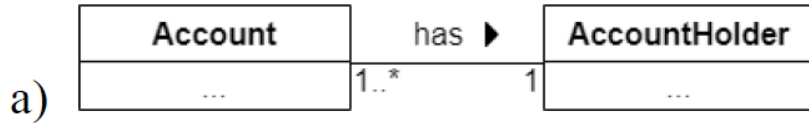
Question 12

Which class diagram correctly represents the scenario?

An account has one and only one account holder. An accountholder has one or more accounts.

Answer: a

- Account has multiplicity 1 to AccountHolder
- AccountHolder has multiplicity 1..* to Account
- Navigate from Account to AccountHolder



Question 13

The vertical dimension in a sequence diagram represents _____

- a) a timeline
- b) an object
- c) all objects involved in the interactions
- d) messages

Answer: a) a timeline

- Time flows from top to bottom
- Horizontal dimension shows different objects
- Messages flow horizontally across time

Question 14

Which of the following are Extreme Programming practices? (Multiple answers)

- a) User stories
- b) Test-first development
- c) Pair programming
- d) Story points
- e) Client meetings

Answer: a, b, c

- XP includes user stories, TDD, and pair programming
- Story points are from Scrum
- Client meetings are general practice, not XP-specific

Question 15

In Agile methodology, what is the primary purpose of a sprint retrospective?

- a) To assign tasks for the next sprint
- b) To review and improve the team's processes
- c) To demonstrate completed work to stakeholders
- d) To conduct user acceptance testing

Answer: b) To review and improve the team's processes

- Retrospective focuses on team improvement
- What went well? What can be improved?
- Sprint review is for demonstrating work

Question 16

What is a primary advantage of using Agile methodologies?

- a) Strict adherence to a fixed plan
- b) Reduced need for customer involvement
- c) Increased flexibility and adaptability to changing requirements
- d) Longer development cycles

Answer: c) Increased flexibility and adaptability to changing requirements

- Agile embraces change
- Short iterations allow quick pivots
- Customer feedback drives development

Question 17

What are some benefits of initiating a project with a pre-planned schedule? (Multiple answers)

- a) Helps with price estimates
- b) Have significant more customer involvement and availability
- c) Get a quick overview of the milestones of the project
- d) Better estimation of project timelines and resource allocation
- e) Increased flexibility in adapting to changing requirements

Answer: a, c, d

- Pre-planned schedules help with budgeting
- Clear milestones and timeline visibility
- Better resource planning
- Less flexibility than Agile

Question 18

What are the steps in an effective risk management process?

- a) Risk identification, Risk monitoring, Risk analysis, Risk planning
- b) Risk identification, Risk analysis, Risk planning, Risk monitoring
- c) Risk analysis, Risk identification, Risk planning, Risk monitoring
- d) Risk analysis, Risk planning, Risk identification, Risk monitoring

Answer: b) Risk identification, Risk analysis, Risk planning, Risk monitoring

1. Identify risks
2. Analyze probability and impact
3. Plan mitigation strategies
4. Monitor throughout project

Question 19

Product risks affect _____

- a) schedule or resources
- b) the organization developing or procuring the software
- c) the quality or performance of the software being developed

Answer: c) the quality or performance of the software being developed

- Product risks: quality issues, performance problems
- Project risks: schedule, budget, resources
- Business risks: organizational impact

Question 20

Which of the following describes a task-oriented person?

- a) The motivation for doing the work is a means to an end
- b) The motivation for doing the work is the presence of co-workers
- c) The motivation for doing the work is the actions of co-workers
- d) The motivation for doing the work is the work itself

Answer: d) The motivation for doing the work is the work itself

- Task-oriented: intrinsically motivated by the work
- Self-oriented: work as means to personal goals
- Interaction-oriented: motivated by social aspects

Part B - Short Answer

5 questions requiring explanations

Question 1: Testable User Stories (3 marks)

Are these user stories testable?

1. “As a user, I want to be able to log in to the system using my email and password.”
2. “As a user, I want the system to be intuitive and easy to use.”
3. “As a user, I want the system to be fast.”

Question 1: Answers

Story 1: TESTABLE ✓

- Describes a specific action (logging in)
- Clear criteria: using email and password
- Can verify: valid credentials work, invalid don't

Story 2: NOT TESTABLE ✗

- “Intuitive” and “easy to use” are subjective
- No measurable success criteria
- Needs specific usability metrics

Story 3: NOT TESTABLE ✗

- “Fast” is subjective without metrics
- Need specific performance targets
- E.g., “loads in under 2 seconds”

- As a user, I want to be able to log in to the system using my email and password.
- Explanation: This user story is testable because it describes a specific action (logging in) and provides clear criteria for success (using email and password).
- As a user, I want the system to be intuitive and easy to use.
- This user story is not testable because it lacks specific actions or criteria for success. "Intuitive" and "easy to use" are subjective terms that are difficult to measure objectively.
- As a user, I want the system to be fast.
- This user story is not testable because "fast" is a subjective term. It does not provide clear criteria for success or specific actions to test.

Question 2: External Sponsor vs Real-World Client (2 marks)

List at least four differences:

Question 2: Answer

1. **Solicitation:** Instructors/college solicited the client's involvement
 2. **Competition:** Client wasn't shopping around for best development team
 3. **Service relationship:** Client is performing a service for students
 4. **Expectations:** Different expectations between students and customer
 5. **Choice:** Students don't have as much say in who their client is
- Key point:** Educational context vs commercial context

Question 3: Story Points vs Time Estimates (2 marks)

What are the distinctions? How does planning poker help?

Question 3: Answer

Story Points:

- Units of measure for overall effort
- Relative sizing (fibonacci: 1, 2, 3, 5, 8, 13...)
- Include complexity, risk, and uncertainty
- Team-specific, not time-based

Planning Poker:

- Gamified estimation technique
- Uses cards to vote on story points
- Everyone reveals simultaneously (no anchoring)
- Discussion on outliers leads to consensus
- More engaging and accurate than individual estimates

- Story points are units of measure for expressing an estimate of the overall effort required to fully implement a product backlog item or any other piece of work.
- Scrum poker, also known as “planning poker” and “pointing poker”, is a gamified technique that development teams use to guess the effort of project management tasks.
- These estimations are based on the entire group’s input and consensus, making them more engaging and accurate than other methods.
- To help gauge the number of story points for the relevant tasks, teams use planning poker cards, which are like poker cards.

Question 4: Agile vs Plan-Driven (4 marks)

Key differences and when to use each approach

Question 4: Answer - Plan-Driven

Plan-Driven Development:

- Separate development stages planned in advance
- Outputs defined upfront for each stage
- Iteration occurs within activities
- Requirements fixed early

Best for:

- Construction projects (building bridges)
- Regulated industries (medical devices, aerospace)
- Fixed requirements and specifications
- High cost of change

Question 4: Answer - Agile

Agile Development:

- Specification, design, implementation, testing inter-leaved
- Outputs decided through negotiation
- Requirements evolve throughout development
- Frequent delivery of working software

Best for:

- E-commerce platforms (changing market demands)
- Mobile apps (user feedback driven)
- Startups (pivoting based on market)
- Uncertain or evolving requirements

- Plan-driven development
 - A plan-driven approach to software engineering is based around separate development stages with the outputs to be produced at each of these stages planned in advance.
 - Iteration occurs within activities.
- Agile development
 - Specification, design, implementation and testing are interleaved and the outputs from the development process are decided through a process of negotiation during the software development process.

Question 5: Daily Scrum Meetings (4 marks)

Purpose and three required questions

Question 5: Answer

Purpose:

- Daily synchronization point
- Scrum Master facilitates
- Tracks backlog and progress
- Quick status updates (15 minutes)
- Identifies blockers early

Three Questions Each Member Answers:

- 1. What have you done since yesterday?**
- 2. What are you planning to do today?**
- 3. Any impediments/stumbling blocks?**

- The 'Scrum master' is a facilitator who arranges daily meetings, tracks the backlog of work to be done, records decisions, measures progress against the backlog and communicates with customers and management outside of the team.
 - What have you done since yesterday?
 - What are you planning to do today?
 - Any impediments/stumbling blocks?

Part C - Long Answer

E-commerce platform scenario

Scenario Context

Your software development team has been tasked with building a new **e-commerce platform** for a client.

The project requirements are expected to **evolve over time** due to:

- Changing market demands
- Customer feedback

Question 1: Initial Client Meeting (2 marks)

What information should your team gather from the first meeting?

Question 1: Key Discussion Points

- **Project vision and goals:** What problem are we solving?
- **Target users:** Who will use the platform?
- **Core features:** Must-have functionality
- **Timeline expectations:** Deadlines and milestones
- **Budget constraints:** Resources available
- **Success metrics:** How to measure project success
- **Stakeholders:** Who makes decisions?
- **Existing systems:** Integration requirements
- **Technical constraints:** Platform preferences, security needs

Question 2: Before Second Meeting (2 marks)

What should your team complete before the follow-up meeting?

Question 2: Team Preparation

- **Review and document** meeting notes
- **Research** similar e-commerce platforms
- **Draft initial user stories** based on requirements
- **Prepare questions** for clarification
- **Create rough timeline** and sprint plan
- **Identify technical risks** early
- **Assign preliminary roles** within team
- **Prepare mockups or wireframes** if possible
- **Estimate story points** for initial backlog

Question 3: TDD and Pair Programming (2 marks)

How could each be used in the e-commerce project? Provide examples.

Question 3: Answer - TDD

Test-Driven Development:

- Write tests before implementing features
- Ensures code quality and test coverage

Example Scenario:

1. Shopping cart functionality

- Write test: “Add item to cart increases quantity by 1”
- Implement minimal code to pass
- Refactor for clean code

2. Payment processing

- Write test: “Invalid credit card rejected”
- Implement validation logic
- Refactor and optimize

Question 3: Answer - Pair Programming

Pair Programming:

- Two developers, one workstation
- Driver codes, Navigator reviews

Example Scenario:

1. Complex checkout flow

- Driver: Implements multi-step process
- Navigator: Catches edge cases, reviews logic
- Switch roles every 25 minutes

2. Security features

- Driver: Codes authentication
- Navigator: Reviews for vulnerabilities
- Real-time code review prevents bugs

Question 4: Risk Management Plan (5 marks)

Identify at least two risks with:

- Risk type
- Probability
- Effects
- Management strategy

Risk 1: Example - Requirements Change

Type: Project Risk

Probability: High (market demands change frequently)

Effects:

- Schedule delays
- Scope creep
- Budget overrun
- Team burnout

Management Strategy:

- Use Agile methodology with short sprints
- Regular client check-ins
- Maintain flexible backlog
- Change control process
- Buffer time in schedule

Risk 2: Example - Third-Party Payment Gateway

Type: Product Risk

Probability: Medium (external dependency)

Effects:

- Payment processing failures
- Security vulnerabilities
- Customer trust issues
- Revenue loss

Management Strategy:

- Research multiple payment providers
- Implement fallback options
- Thorough integration testing
- Monitor service status
- Have contingency provider ready

Risk 3: Example - Team Member Availability

Type: Project Risk

Probability: Medium

Effects:

- Knowledge silos
- Missed deadlines
- Uneven workload
- Quality issues

Management Strategy:

- Cross-training team members
- Documentation of all work
- Pair programming for knowledge sharing
- Regular code reviews
- Backup person for each role

Question 5a: Bob's Engagement Issues (2 marks)

For this question, assume you are appointed as the project manager. You've noticed a concerning pattern with one of your team members, Bob. Bob has been consistently arriving late to meetings and failing to complete his assigned tasks on time for the past few weeks. Recognizing the importance of addressing this issue promptly, you decide to have a conversation with Jimmy, another member of your team who has a close relationship with Bob. During your discussion, Jimmy revealed some insightful information about Bob's perspective. It appears that Bob is feeling disheartened because he perceives his assigned tasks as overly time-consuming and lacking in clear future value. Moreover, he feels a sense of envy towards his peers, believing that they are engaged in more stimulating and rewarding assignments.

Scenario: Bob arrives late, misses deadlines, feels tasks are time-consuming and lack value, envies others' assignments.

What implications might arise for the rest of the group?

Question 5a: Answer

Potential Implications:

1. Team Morale Decline

- Other members become frustrated
- Loss of cohesion and trust

2. Project Delays

- Missed milestones
- Dependencies blocked

3. Quality Compromise

- Rushed work to compensate
- Incomplete deliverables

4. Workload Redistribution

- Others pick up slack
- Resentment builds

5. Team Dynamics

- Conflict between members
- Reduced productivity overall

Question 5b: Addressing Bob's Concerns (2 marks)

As project manager, what steps could you take?

Question 5b: Answer

Step 1: Private Meeting with Bob

- Listen actively and non-judgmentally
- Understand his perspective
- Validate his contributions
- Build trust and respect

Step 2: Adjust Workload

- Align tasks with Bob's interests/skills
- Redistribute responsibilities if needed
- Provide more engaging assignments
- Explain value/purpose of tasks

Question 5b: Answer (continued)

Step 3: Foster Open Communication

- Regular check-ins with all team members
- Encourage feedback sessions
- Create supportive environment
- Team problem-solving approach

Step 4: Professional Development

- Offer skill-building opportunities
- Rotate interesting tasks
- Provide visibility on career value
- Connect work to learning outcomes

Step 5: Set Clear Expectations

- Define accountability measures
- Establish consistent standards
- Follow up on commitments
- Balance support with responsibility

Key Takeaways

From This Exam:

- User stories need clear, testable criteria
- Agile embraces change; plan-driven needs stability
- TDD and pair programming improve quality
- Risk management requires proactive planning
- Team dynamics affect project success
- Communication is critical at all levels

Study Tips for Future Exams

- 1. Understand concepts, don't just memorize**
- 2. Practice drawing UML diagrams**
- 3. Know the Agile ceremonies and their purposes**
- 4. Understand when to use different methodologies**
- 5. Think about real-world applications**
- 6. Review risk management frameworks**
- 7. Study team dynamics and conflict resolution**

Questions?

Thank you for your attention!